

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Vehicle routing problems with slack
induction by string removals**

Master's Thesis

MARTIN PAJERSKÝ

Brno, Spring 2025

**MASARYK
UNIVERSITY**

FACULTY OF INFORMATICS

**Vehicle routing problems with slack
induction by string removals**

Master's Thesis

MARTIN PAJERSKÝ

Advisor: doc. Mgr. Hana Rudová, Ph.D.

Department of Machine Learning and Data Processing

Brno, Spring 2025



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source. In the preparation of this thesis, I used Grammarly for syntax and grammar checking, and for improving my writing style. I declare that I used these tools according to the principles of academic integrity. I checked the content and take full responsibility for it.

Martin Pajerský

Advisor: doc. Mgr. Hana Rudová, Ph.D.

Acknowledgements

I would like to thank my supervisor, doc. Hana Rudová, Ph.D., for her professional, thoughtful guidance and constructive feedback. I am also grateful to RNDr. Václav Sobotka for his assistance and practical insights and Prof. Greet Vanden Berghe for kindly sharing details about their paper via email. Lastly, I am deeply thankful to my parents and siblings for their love and support.

Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic.

Abstract

Vehicle routing problems aim to efficiently serve customer requests while respecting constraints such as vehicle capacities and delivery time windows. When it is not possible to fulfill all requests, orienteering problems are considered, where only a subset of locations is visited to maximize the total collected reward. This thesis implements the Slack induction by string removals heuristic for several representative vehicle routing problem variants, and its effectiveness is confirmed by reproducing the original results. The approach is also extended to orienteering problems, with specific changes to handle their unique requirements. The developed solver achieves performance comparable to the best available methods across multiple problem variants.

Keywords

Vehicle routing problem, orienteering problem, slack induction by string removals, optimization, heuristics

Contents

1	Introduction	1
2	Problem description	3
2.1	Classical vehicle routing problems	3
2.1.1	Vehicle routing problem variants	4
2.2	Orienteering problems	5
2.2.1	Orienteering problem variants	6
2.3	Formal model	6
2.4	Solution approaches	10
2.4.1	Exact methods	10
2.4.2	Heuristics	11
2.4.3	Metaheuristics	12
2.4.4	Hybrid solvers	16
2.5	Existing implementations	18
3	Slack induction by string removals	20
3.1	Large neighborhood search	20
3.2	Destroy heuristic	22
3.3	Repair heuristic	27
3.4	Fleet minimization	29
3.5	Parameter settings	32
3.6	Adaptations to problem variants	32
3.6.1	Time windows	32
3.6.2	Pickup and deliveries	33
4	Approaches for orienteering problems	35
4.1	Large neighborhood search	35
4.1.1	Initial solution construction	36
4.2	Destroy heuristic	36
4.3	Repair heuristic	36
4.3.1	Sorting methods	38
4.3.2	Shaw removal inspired shuffle	39
4.4	Parameter settings	40
4.5	Adaptations to problem variants	41
4.5.1	TOPTW	41
4.5.2	CTOP	41

5	Experiments	42
5.1	Classical Vehicle routing problems	42
5.1.1	Datasets	43
5.1.2	Evaluation	44
5.2	Orienteering problems	52
5.2.1	Datasets	52
5.2.2	Parameter tuning	54
5.2.3	Evaluation	60
6	Conclusion	66
	Bibliography	68
A	Archive	76
B	SISR parameters	77
C	Detailed tabular results	79

1 Introduction

Every day, companies deliver goods, transport people, or provide services that require moving between locations. Planning these routes thoroughly is crucial, as it saves money, reduces fuel use, and improves customer satisfaction. Even small improvements in routing can make a substantial difference in lowering costs.

These tasks are studied as vehicle routing problems (VRP), a traditional field in operations research. The goal is to assign routes to a fleet of vehicles so that all requests are fulfilled as efficiently as possible. The classical routing problems focus on optimizing routes while considering practical constraints, such as limited vehicle capacities or delivery time windows. When constraints make it impossible to serve all customers, orienteering problems (OP) become relevant. In OP, each request is assigned a reward, and the objective is to maximize the total collected reward within the limits. This extends the classical routing problems by adding a decision step of selecting a subset of locations to visit.

Routing problems are complex, and an exact method cannot achieve the performance needed for real-world instances. This led to the use of heuristics. Over the long history of vehicle routing research, increasingly complicated methods have been developed to solve specific problem variants [1]. It is then difficult for others to reuse even parts of these approaches or to adapt them to different constraints. Simpler and more flexible approaches are easier to adjust to practical constraints [2]. Slack induction by string removals (SISR) created by Christiaens and Vanden Berghe in 2020 [3] has achieved highly competitive results, especially for large-scale problems. It stands out not just for its performance but also for its ability to handle various problem variants with only minor modifications. Despite its success, there is no publicly available implementation of SISR. In addition, efficient and open-source solvers with support for multiple variants are generally very limited.

This thesis has two main goals. First, it implements the SISR heuristic for representative variants of classical VRP and compares the performance with the original paper. Second, it demonstrates the capabil-

ities of both the SISR algorithm and the developed implementation by adapting this heuristic to the family of orienteering problems.

The content of the work is structured as follows. Chapter 2 introduces the core routing problems considered in the thesis, VRP, OP, and their representative variants. The chapter also discusses common solution approaches and briefly surveys existing solvers. Chapter 3 presents the SISR heuristic for VRPs in detail, following the structure of the original paper. This chapter is enriched with insights to more fully describe the implementation process. Chapter 4 builds on the previous chapter and explains our adaptation of SISR for orienteering problems. Chapter 5 evaluates the performance of the implemented solvers. For VRPs, the results are directly compared with those reported in the original SISR paper. For OPs, parameter tuning is discussed and followed by a comparison to state-of-the-art solutions. Chapter 6 summarizes the main contributions of the thesis, reflects on the limitations of the current approach, and outlines directions for future research.

2 Problem description

The *vehicle routing problem* (VRP) was first proposed by Dantzig and Ramser in 1959 [4]. It generalizes the *traveling salesman problem* (TSP) with the goal of determining routes for multiple vehicles. The vehicles deliver goods to a set of customers while the total traveled distance is minimized. A closely related class of routing problems is the *orienting problem* (OP), introduced by Golden et al. in 1987 [5]. OP also derives from the TSP but with a key difference: not all customer nodes must be visited. Instead, the goal is to determine a subset of served customers that maximize a reward for visiting them.

This chapter describes VRP and OP separately, briefly explaining their formulations, objectives, and basic *variants* (*extensions*). Subsequently, we discuss solution approaches, including both exact and heuristic methods, and conclude with a short overview of existing open-source implementations.

2.1 Classical vehicle routing problems

In general, the VRP concerns the distribution of goods from a depot to a set of customers using a fleet of vehicles. Each customer is located at a specific location and puts in a request for the delivered supplies. Vehicles start and end their routes at the depot, each serving at most one route. Every customer must be visited exactly once, splitting their request among multiple vehicles is not allowed.

VRP is usually defined by a complete graph, where nodes correspond to the customers and depot, while the edges yield a cost for traversing between locations. The cost can represent the amount of used fuel, traveled distance, or the time spent moving and serving customers. A single unit, such as weight or number of pallets, describes customer demand. A feasible route is a sequence of customer nodes visited by a vehicle fulfilling all constraints. Route size (cardinality) refers to the number of customers visited (served) by the route. Formal mathematical models of VRP can be found in a widely recognized book by P. Toth and D. Vigo [6, p. 4]. In Section 2.3, we provide a formal model for OP variant and VRP can be understood as a special

case, with a different objective function and the requirement that all customers must be visited.

The goal of VRP is not just to find a set of routes that collectively fulfill all customer requests. As an optimization problem, we want to discover the best of all candidate solutions concerning an objective. Multiple objectives have been studied, and their goals are often conflicting [7]. The most common objective is to minimize the total cost. In real-life scenarios, aiming to minimize the number of vehicles is also frequent. Since using more vehicles requires more drivers, it leads to higher route-related fixed costs. Other standard objectives are balancing the travel time of the routes or the vehicle load. Additionally, the objectives are often combined hierarchically, introducing new opportunities for defining problems.

2.1.1 Vehicle routing problem variants

Directly motivated by real-world applications, many VRP variants have been introduced and researched. These variants modify problem attributes, objectives, or constraints to fit various operational settings [8]. Here, we briefly describe three of the most studied variants.

- *Capacitated VRP* (CVRP) is the most fundamental and extensively researched variant [9]. The vehicles have a predefined capacity that cannot be exceeded. When deciding which customers to visit with each vehicle, total transported goods must add up to a maximum of this capacity constraint. In the simplest version of CVRP, all vehicles have homogeneous capacity.
- *VRP with time windows* (VRPTW) assigns to each node a time window in which the service must start [10]. All customers must be visited within their time windows, but the vehicles can wait if they arrive before the window opens. Alongside the start and end of the time window, a service time is usually specified, representing the duration required to complete the delivery at the customer's location. The depot itself can also have a time window, resulting in limitation on the duration of routes. In the literature, VRPTW is often associated with a hierarchical objective. First, the goal is to minimize the number of vehicles used, and only then to minimize the total cost.

- *Pickup and delivery problem with time windows* (PDPTW) is a direct extension of VRPTW. Instead of transporting goods from the depot to customers, requests are generalized to start and end at any pair of nodes [11]. Vehicles load goods in a pickup node and transfer it to a corresponding delivery node. This order must be respected, and each pickup-delivery pair must be served by the same vehicle. Vehicles can start either empty or partially loaded from the depot. As the vehicle visits pickup and delivery locations, its load changes non-monotonically, depending on the order in which customers are visited.

2.2 Orienteering problems

With a limited time budget, serving all customers may become impossible. This leads to the definition of the OP, where an optimal route is found only for a subset of requests, leaving remainder *absent*. Customers are assigned a *score* (*reward/profit*), which is earned for serving them. The basic version of the problem uses only a single vehicle, and each customer can be visited at most once. The multi-vehicle OP, more closely aligning with the VRP, is described as a fundamental OP variant.

The mathematical definitions of OP are similar to those of VRP and are based on graph representations. We provide the mathematical formulation of the problem in Section 2.3. A modified version of OP, where the edges of a graph represent customers and are assigned rewards, was also studied and is used for different applications [12].

The measure of collected scores introduces a new primary objective: maximizing the total sum of scores from visited customers. After optimizing the rewards, it is traditionally also desirable to minimize expenses to save resources. The costs known from classical VRP remain important and can be used as secondary objectives. Another studied combination of objectives is maximization of the difference between scores and costs, or optimization of one measure, with the other one bounded [13].

2.2.1 Orienteering problem variants

In this section, similar to VRP, we briefly describe the most common variants of OP. Numerous extensions have been formulated, with a survey paper by Gunawan et al. [14] providing one of the most recent and comprehensive overviews.

- *Team OP* (TOP) defines a fixed number of vehicles greater than one and the objective is to maximize the total score collected by the whole fleet/team. The limit on the fleet size must be satisfied. Allowing an unbounded number of vehicles would make the scoring system meaningless. In that situation, the solution could trivially maximize the score, reducing the problem to a variant of VRP. Additionally, the length of each route is typically also limited.
- *TOP with time windows* (TOPTW) builds upon the principles of TOP and incorporates time window constraints identical to those described for VRPTW, where customers must be served within the specified time intervals.
- *Capacitated TOP* (CTOP) similarly to CVRP, requests have associated demands, and the capacity of the vehicles must be respected. In the literature, the team orienteering problem with both time windows and capacity constraints is referred to as *selective vehicle routing problem with time windows* (SVRPTW) [15].

2.3 Formal model

In this section, we present a formal model for solving team orienteering problem with time windows. We will describe TOPTW as it is the most complicated variant from ones introduced in the previous section. The model is based on formulation from [16]. At the end, we also briefly describe how models for TOP and CTOP differ from this formulation. The notation is summarized in Table 2.1.

In our model, we consider a set of customer nodes N , where each customer $i \in N$ is assigned a positive score p_i .

The set of all vehicles is denoted K , each serving a single route. A constant $T_{\max}$ limits the length of all routes. Each route $k \in K$ starts in

Notation	Description
G	network graph
V	set of all nodes in G , including customers and depots
E	set of edges in G
N	set of customer nodes
τ^+	node of the origin depot
τ^-	node of the destination depot
K	set of vehicles
Q	weight capacity of vehicles
$T_{\max}$	time limit of each route
t_{ij}	travel time between nodes $i \in V$ and $j \in V$
l_i	lower bound of the time window of node $i \in V$
u_i	upper bound of the time window of node $i \in V$
s_i	service time at node $i \in V$
p_i	score of customer $i \in N$
d_i	demand of customer $i \in N$
x_{ijk}	binary decision variable, set to 1 only if vehicle $k \in K$ visits node $j \in V$ immediately after node $i \in V$
y_{ik}	binary decision variable, set to 1 only if vehicle $k \in K$ visits node $i \in V$
S_{ik}	continuous decision variable denoting start of service at node $i \in V$ in route $k \in K$

Table 2.1: Model notation.

a origin depot τ^+ for and ends in a destination depot τ^- . The depots are identical for all routes and also represented as nodes. Only some of the traditional OP datasets consider origin and destination depots to have physically different locations, but the model allows for cases where $\tau^+ = \tau^-$.

The problem is defined on a complete, undirected graph $G = (V, E)$, where the set of nodes $V = N \cup \{\tau^+, \tau^-\}$ includes customer locations as well as the depots. The reason for the graph being undirected is that in the literature, the travel time between nodes $i, j \in V$ is considered to be symmetrical. The travel time is denoted t_{ij} and usually computed using Euclidean distance. If required for real-world

application, the model can be modified to consider directed arcs. For example, a model of the directed TOP can be found in [6, p. 284].

Each node $i \in V$ is associated with a time window $[l_i, u_i]$. Additionally, service at each location takes a non-negative duration s_i . Note that the service times and time windows also apply to depot nodes, not just customers. The upper bound τ^- of the depot's time window, effectively limits the route similarly to $T_{\max}$. Other time windows-related values for depots are usually zero.

The model uses three decision variables. A binary variable $x_{ijk} \in \{0, 1\}$ is equal to 1 if, in route $k \in K$, node $j \in V$ is visited immediately after node $i \in V$ and zero otherwise. Secondly, binary variable $y_{ik} \in \{0, 1\}$ is equal to 1 if node $i \in V$ is visited by vehicle $k \in K$, and zero otherwise. Third variable S_{ik} is continuous and denotes the service start time at location $i \in V$ for vehicle $k \in K$.

The objective function maximizes sum of scores of visited customers across all routes

$$\text{maximize } \sum_{i \in N} \sum_{k \in K} p_i y_{ik}. \quad (2.1)$$

To limit the number of routes to be exactly $|K|$, each with start at τ^+ and end in τ^- , we define constraint

$$\sum_{k \in K} \sum_{j \in N} x_{\tau^+ j k} = \sum_{k \in K} \sum_{i \in N} x_{i \tau^- k} = |K|. \quad (2.2)$$

Following pair of constraints is used to guarantee the connectivity of the paths

$$\sum_{i \in N \cup \{\tau^+\}} x_{ijk} = y_{jk} \quad \forall j \in N, \forall k \in K, \quad (2.3)$$

$$\sum_{j \in N \cup \{\tau^-\}} x_{ijk} = y_{ik} \quad \forall i \in N, \forall k \in K. \quad (2.4)$$

Each customer must be visited at most once, this is ensured by constraint

$$\sum_{k \in K} y_{ik} \leq 1 \quad \forall i \in N. \quad (2.5)$$

The limit on distance of each route $T_{\max}$ is ensured by constraint

$$\sum_{i \in N} (s_i y_{ik} + \sum_{j \in N} t_{ij} x_{ijk}) \leq T_{\max} \quad \forall k \in K. \quad (2.6)$$

Variables S_{ik} construct a timeline of route $k \in K$, where the time continuity of services of any two subsequently visited nodes is ensured by constraint

$$S_{ik} + s_i + t_{ij} - S_{jk} \leq M(1 - x_{ijk}) \quad \forall i, j \in V, \forall k \in K. \quad (2.7)$$

Where M is a constant large enough to ensure that when $x_{ijk} = 0$ the constraint will not restrict the values of S_{ik} and S_{jk} .

Finally, each customer visit must fit into its time window, which is ensured by constraint

$$l_i \leq S_{ik} \leq u_i \quad \forall i \in V, \forall k \in K. \quad (2.8)$$

We now show the necessary changes and additions to transform this TOPTW formulation to capture other representative variants of OP.

Models for TOP

The formulation for TOP can be found in [17]. The model does not differ significantly from the one we presented. The majority of the constraints remain the same, but constraints 2.7 and 2.8 regarding time windows are omitted. Removing those constraints means that a new constraint to eliminate subtours (i.e., cycles without depots) needs to be added. Additionally, if $|K| = 1$, the model corresponds to plain OP.

Capacity constraint

Capacity constraints are introduced by defining a maximum vehicle capacity Q and a non-negative demand d_i for each customer node $i \in N$. The vehicles are prevented from exceeding their capacity by implementing a constraint

$$\sum_{i \in N} d_i y_{ik} \leq Q \quad \forall k \in K. \quad (2.9)$$

A self-contained formulation for CTOP can be found in [18].

2.4 Solution approaches

Thanks to their shared roots, the core solution techniques for both VPR and OP families involve searching for optimal routes. OP represents a broader domain of combinatorial problems since it introduces an additional decision. Even though the objectives and constraints differ, selecting the subset of requests to maximize the score and planning the optimal path are usually done simultaneously when solving OP [6, p. 13]. Thanks to this, methods used in solvers significantly overlap for both problems. In this section, we describe solving techniques for VPR and OP jointly, noting key differences and specifics if needed.

2.4.1 Exact methods

Exact solving methods use rigorous mathematical techniques and systematic exploration of all possible solutions to ensure they find the best possible solution to the problem. They are valuable for benchmarking heuristic algorithms and for applications where finding the optimal solution is critical.

Initial approaches to solving the VRP in the 1960s were built on advances made in solving the traveling salesman problem [2]. Since then, the algorithms have been steadily improved. The most effective exact algorithms currently rely on the branch-and-bound technique [19]. Combined with other approaches, it forms widely used branch-and-cut or branch-cut-and-price methods. In principle, these techniques systematically divide the problem into smaller subproblems (branching) and then eliminate infeasible or suboptimal solutions in the search for the optimal solution (pruning).

*VRPSolver*¹, a CVRP exact solver released in 2019, reaches state-of-the-art results [20]. More recently, solver *RouteOpt*², using a learning-based branching strategy, achieved a 47% speedup over *VRPSolver* [21].

Exact solvers of OP and its variants also use branching techniques mentioned for VRP. Methods additionally involve means to determine which customers to include in a route while respecting the constraints [15].

1. <https://vrpsolver.math.u-bordeaux.fr>

2. <https://github.com/Zhengzhong-You/RouteOpt>

Despite their theoretical advantages and optimality guarantees, the exact methods cannot efficiently handle increasingly complex problems. As ascendants of the TSP, both VRP and OP are NP-hard problems [22, 23]. With a growing number of customers, it is more challenging to find optimal solutions. CVRP instances of around 200 customers are solved in a matter of hours, and only a few instances of more than 300 customers were solved. TOP instances with up to 100 customers and shorter routes were solved optimally [24], but the complexity increases even more steeply with the number of customers.

Variants introduce additional considerations to the solvers. For instance, while the hierarchical objective in VRPTW is mainly used in heuristics [6, p. 135], it can still be modeled in a single exact formulation by introducing a cost penalty to the first customer in a route [25]. Then, the same methods, such as branch-and-cut, can be applied. However, when the number of vehicles is fixed, even finding a feasible solution to VRPTW also becomes an NP-complete problem [26]. PDPTW is particularly challenging too, the solvers struggle with problems of fewer than 100 requests [27].

These aspects, along with the growing real-life applications, have led to the fact that as many as 81% of relevant VRP solution method articles published between 2009 and 2015 used heuristics or meta-heuristics [9].

2.4.2 Heuristics

In real-life settings, heuristics have gained popularity as they are often better suited for practical applications. Their advantages lie in low or adjustable computational cost and the ability to handle larger problems that are beyond the scope of exact solvers. Heuristics also allow for better generalization and often provide more flexibility to contain new constraints or changes in the objective function.

Constructive heuristics are methods used to create an initial solution, starting from scratch. These methods greedily apply predefined rules to build routes and quickly converge into a feasible solution. For instance, the *savings* algorithm [28] is a simple but often used approach. The algorithm begins by creating an initial solution where each customer has its own route, going from the depot to the customer and back. In each step, one pair of routes is merged using a saving

criterion. The savings are calculated as the reduction of the overall cost in the case of the merge. A feasible pair of routes with the highest saving is merged. The process is repeated until no more merges are possible. The savings concept can also be applied to TOP [29]. Overall, the constructive heuristics are usually approximately 15% worse than the optimum [30]. Thanks to their speed and relatively good quality of delivered solutions, constructive heuristics are used as an input to improvement heuristics [6, p. 88]. Nowadays, they are also utilized as foundational components in more advanced algorithms.

Improvement heuristics aim to modify existing solutions to obtain better results. They take the solution and perform operations that alter it in some way. The solution itself does not change significantly and is referred to as a *neighbor* or a *local change*. The cost of the new solution is then calculated, and in case of improvement, the process can be repeated. For example, the *relocate* heuristic picks a customer and moves it to a different position inside its route [31]. More complex heuristics have been developed to simultaneously modify multiple routes, allowing the exploration of a larger solution space and potentially leading to better results.

2.4.3 Metaheuristics

Metaheuristics are higher-level strategies used to control the problem-specific heuristics. They are general approaches designed to guide the search process and are applicable to different optimization problems. Metaheuristics often bring mechanisms that help escape local optima, which is typically not achievable with standalone improvement heuristics. The field of metaheuristics is vast, and there are many ways to classify them. We will distinguish them based on their approach to exploring the solution space. Specifically, VRP solving methods can be classified into single-solution and population-based.

Single-solution based metaheuristics

Single-solution based methods explore neighborhoods by making changes to a current solution. Starting from an initial solution, the algorithm selects only one candidate at each iteration. The process of replacing the current solution is referred to as the candidate being

accepted. The initial solution is generated randomly or created using constructive heuristics, such as the savings method discussed in the previous section.

Hill climbing is the simplest metaheuristic. It iteratively improves the solution by exploring its neighbors and accepting only those that bring improvement. The search terminates when no improving neighbor exists. Hill climbing can only find local optimum solutions. To address this limitation, a *multistart* algorithm is used to bring diversity into search. The whole search is restarted again using a new initial solution.

Iterated local search (ILS), introduced by Lourenço et al. [32], first applies local search to improve the solution, similar to hill climbing. Instead of restarting the whole search after reaching a local optimal solution, ILS makes a large change to the solution, known as a perturbation. This potentially allows access to neighborhoods that were previously unreachable. Finally, accepting criteria must be met to move into the perturbed solution. This whole process is iteratively repeated. The intensity of the perturbation mechanism and acceptance criteria must be chosen carefully to balance between completely random moves and insufficient diversity. Gunawan et al. applied ILS to TOPTW and its real-world application [33]. Thanks to careful tuning of local search operators, the authors were able to improve multiple best-known solutions.

Simulated annealing (SA) is a probabilistic method that allows the escape of the local optimum by possibly accepting a worsening solution [34]. The process is controlled by temperature. During each iteration, a neighbor solution is generated, and its objective function is calculated. The candidate is accepted if its objective function has increased or, based on the probability determined by the current temperature, slightly declined. The probability of accepting a worsening neighbor decreases over time as the temperature is gradually reduced in each iteration. The best overall solution found is stored and returned at the end. Lin combined SA with a multistart approach, using simple neighborhood operators to develop solvers for TOP [35] and TOPTW [36]. Their work achieved competitive results on established benchmarks, improving several of the best-known solutions. These studies highlighted the practical value of SA, mainly its effective convergence and efficient memory usage.

Tabu search prevents the search from plateauing by temporarily discarding neighbors that have been recently visited. Instead, it navigates to the best neighbor that is not present in the tabu list. Many different variants and extensions of this method have been proposed for VRP, e.g., [37].

Common heuristics such as relocate, or k-opt [38] make small changes and explore a close neighborhood quickly and in detail. However, this may not be sufficient for exploring solution spaces effectively. A *large neighborhood search* (LNS) for VRP was created to address these limitations [39]. LNS considers much bigger neighborhoods, which are too broad to be explored in detail. This metaheuristic relies on *destroy* (*ruin*) and *repair* (*recreate*) operators. A subset of customers is first removed from the solution, destroying a part of its structure. Subsequently, the solution is repaired by reinserting the nodes back, ideally in a near-optimized order. A large number of operators have been developed to improve the performance of LNS or to address specific problem variants. Some of these operators include radial, random, sequential, and demand-based removals, as well as optimal, greedy, and random reinsertions [40]. More complex variants provide opportunities to combine the operators. For example, Schmid et al. [41] based their operators for TOPTW on multiple customer evaluation criteria, which were then scaled relative to each other.

Slack induction by string removals for vehicle routing problems (SISR) algorithm [3], developed by J. Christiaens et al., uses the LNS framework with a novel destroy and greedy repair method. Thanks to the simplicity of the operators, it was adapted to numerous constraints in different problem variants. SISR has achieved outstanding performance across multiple classical VRP variants, most notably PDPTW. In Chapter 3, we will examine this heuristic in detail.

Adaptive large neighborhood search (ALNS) is a framework created by Ropke and Pisinger [42]. It effectively combines multiple repair and destroy heuristics, and simulated annealing controls the acceptance of solutions. Before, LNS solvers typically relied on single but more complicated specialized heuristics. In ALNS, the pair of destroy and repair methods used in each iteration is chosen independently, usually by a roulette wheel selection principle. This approach encourages diverse exploration of the neighborhood. The weights of each heuristic are updated dynamically based on their performance history. The

adaptive selection reduces the need for extensive parameter tuning and adjusts for properties or structures of individual problem instances. ALNS can serve as a general framework for implementing solvers for various problem variants with a fitting choice of heuristics. In the original work [42], the authors used two fast greedy repair and five diverse destroy heuristics, achieving competitive results on the PDPTW variant. Later, the framework was successfully applied to solve multiple other variants [43].

Population-based metaheuristics

Population-based methods use a set of solutions that are iteratively improved. Working with populations maintains diversity and allows for sharing information (e.g., exchanging route segments) to improve solutions. This approach enables the search to proceed on both global and local scales. Since multiple solutions are usually evaluated simultaneously, population-based metaheuristics benefit from parallel computing.

Genetic algorithms (GA), inspired by natural evolution, manage populations through processes like selection, crossover, and mutation of individuals. The crossover operation combines parts of two parent routes to create new ones. Different variants of this operator have been applied when solving VRP [44]. Other adaptations of GA techniques include ensuring diversity by random mutation, which permutes parts of a solution [45], or adaptive parent selection [46], which adjusts algorithm choices based on the current state of the search.

Swarm intelligence is a subset of population-based metaheuristics, where solutions emerge from the collective behavior of individuals (e.g., ants or particles) rather than directly being a combination of the previous solutions. *Ant colony optimization* (ACO) is inspired by the behavior of ants that leave pheromones to guide other individuals to the location of food. At first, an ant constructs a path that appears to be random. As more ants traverse it, the pheromones become reinforced. Over time, this process may lead to the identification of the shortest path to the target. This is suitable for problems where the ant can traverse between graph nodes, creating solutions represented as a path in a graph [47, p. 422]. ACO, combined with local search, found its applications for solving both VRP and OP, e.g., [48].

Particle swarm optimization (PSO) is a variant of swarm intelligence where each particle in the swarm is a numerical vector representing the solution. The search space is explored by iterative movement of the particles based on velocity. The velocity of each particle is updated based on both personal and global best position found. Dang et al. [49] designed a PSO-based heuristic for TOP, introducing multiple sub-swarms, an adaptive velocity update mechanism, and hybridization with local search, among other improvements.

2.4.4 Hybrid solvers

Concepts of various metaheuristics are combined together to create hybrid metaheuristics. These methods take the strengths of different approaches while suppressing their weaknesses. Hybrid solution approaches are becoming more complicated and problematic even to classify. Different approaches have been combined in many ways, and it is impossible to capture all of them. Here, we describe two of the relevant hybrid solvers achieving exceptional results.

Most population-based methods in VRP literature can be considered hybrid, as they commonly rely on local search components to strengthen the search. *Hybrid genetic search* (HGS) by Vidal et al. [50] uses hybridization of genetic algorithms, where a local search plays an important role in finding optimized solutions. In HGS, the solutions are not well-defined sets of routes, but a *giant tour* is used. A giant tour is a plain sequence of all customers, ignoring the depots. It is accompanied by a split operator, which is an essential component. Split optimally finds the shortest paths to reconstruct individual feasible routes, now employing the depots. This representation makes crossovers from GA more general, as they do not have to consider particular routes. The population consists of feasible and adaptively penalized infeasible groups, augmenting the search space. When creating new individuals, HGS uses advanced evaluation that considers not just total cost but also looks at the diversity of the population. If the solutions stagnate, an adaptive diversification mechanism is employed.

HGS has proven effective for various VRP variants [50, 51], leading Vidal et al. to generalize these approaches into the *unified* HGS (UHGS) [52]. The methods were extended to be general and indepen-

dent of specific problem variants. In case a problem-specific strategy is needed, modular components are used. For example, a problem without time windows is treated as unconstrained. The start and the end of the windows are set to $-\infty$ and ∞ , respectively. A single implementation of the UHGS solver was applied to solve 29 VRP variants. It was benchmarked against multiple problem sets, often achieving state-of-the-art performance. However, pickup and delivery problems are significant challenge for HGS, and to our knowledge, no adaptations addressing variants with this type of constraint have been developed. An implementation of HGS for CVRP written in C++ is publicly available³.

Hybridizing methods go far beyond applying different (meta) heuristics at once, they can also be combined with mathematical programming solvers. Being able to isolate smaller combinatorial problems and then solve them to optimality might bring an advantage.

Hybrid adaptive large neighborhood search (HALNS) for TOP by Hammami et al. [53] uses multiple hybridizations. Destroy and repair ALNS with simulated annealing acceptance criterion is used as an underlying metaheuristic. When a new solution is accepted, a series of local search procedures is applied. Subsequently, if the solution is the best found so far, a newly designed sub-route optimization procedure (SROP) is applied. The SROP is solved using integer programming techniques in a multi-threaded commercial exact solver CPLEX⁴. In the end, a set packing problem (SPP) on all discovered routes is solved to optimality, again using a CPLEX. The authors addressed the importance of SROP and SPP, determining both necessary for achieving the maximal scores. Four classical destroy heuristics are used, namely random, sequence, route, and saving removal, supplemented with OP-specific lowest score removal. Nodes to be re-inserted are selected based on one of four heuristics: random, highest score, score to cost increase ratio, and last removed, first inserted.

HALNS reached or improved best-known solutions on all of the studied instances (more than 700 in total), ranging from 64 to 400 requests. The reported computational time is comparable with other

3. <https://github.com/vidalt/HGS-CVRP>

4. <https://www.ibm.com/analytics/cplex-optimizer>

methods. Solving both SROP and SPP subproblems was limited to 60 seconds. Other OP variants were not discussed in the paper.

More recently, the algorithm was adapted for capacitated TOP [54]. The approach was extended with variant-specific operators, such as largest demand removal, and the SROP is no longer mentioned. The method outperformed the compared heuristics and improved several best-known solutions.

2.5 Existing implementations

The field of vehicle routing is subject to active research, but the availability of high-quality, open-source implementations is limited. Many papers introduce new heuristics or increasingly complex variants. Authors often do not share their code, making reproducibility and further development difficult. Additionally, as the real-world problem instances grow in size, the performance is very critical. The solvers implemented in high-level languages are often not sufficient for extensive applications.

Several state-of-the-art solvers exist for classical variants, mainly CVRP and VRPTW. A notable example is the high-performance C++ implementation, HGS-CVRP, which was mentioned in the previous section. Another example is a Python-based library PyVRP⁵ created by Wouda et al., which uses C++ for performance-critical parts and currently supports five problem variants. PyVRP is also based on the hybrid genetic search but lacks support for other widely important variants, such as PDPTW. In the paper [55], the authors also briefly review other open-sourced projects, stating their shortcomings.

Competitions such as the DIMACS⁶ or EURO Meets NeurIPS 2022⁷ attracted many contenders and created ground for developing strong VRP solvers, from which some were released publicly. However, the resulting solvers are high-performing for their respective tasks but are usually not created with the goal of modularity or extensibility.

Other implementations of VRP algorithms are available, but many have not reached state-of-the-art performance. Despite the potential of

5. <https://pyvrp.org/index.html>

6. <http://dimacs.rutgers.edu/programs/challenge/vrp/>

7. <https://euro-neurips-vrp-2022.challenges.ortec.com/>

SISR, to the best of our knowledge, there is only one publicly available implementation, written in Python⁸. No detailed results on established benchmark sets are reported, making it difficult to evaluate the correctness or performance of the code. The repository includes an example of a single instance from a well-known VRPTW benchmark, and no other problem variants are mentioned. We executed the solver on this example as provided in the repository, but an infeasible solution was produced. The code found a solution using 18 vehicles, while the known optimal solution requires a minimum of 20. We also note the existence of a modular Python library ALNS⁹, designed for solving combinatorial optimization problems, not limited to routing. It supports various acceptance and stopping criteria or operator selection schemes. Users have to implement their own destroy and repair methods to use the algorithm for a desired problem. One of the library's official examples is for CVRP and includes a part of the destroy operator from SISR. Even though this example is for illustrative purposes, we mention this to note a public reference to SISR.

8. <https://github.com/rinogrego/SISR-for-VRP/tree/main>

9. <https://alns.readthedocs.io/en/latest/index.html>

3 Slack induction by string removals

Slack induction by string removals for vehicle routing problems (SISR), developed by J. Christiaens and G. Vanden Berghe [3], is a state-of-the-art heuristic based on the adaptive large neighborhood search framework. The design of SISR is simple compared to the complexity of many modern VRP metaheuristics. Additionally, it is very flexible, as the authors successfully applied SISR with slight adjustments to multiple problem variants. It has consistently outperformed general-purpose and specialized algorithms. SISR has shown that accuracy must not always be sacrificed for simplicity.

In this chapter, we present SISR in detail, starting with an underlying LNS metaheuristic, using simulated annealing, which is applied for the distance minimization task. Next, three main contributions of the original paper are described: a destroy heuristic based on adjacent string removal, a greedy repair method with blinks, and an absence-based fleet minimization procedure. We also briefly comment on the core parameter settings. The base versions of the methods are presented for solving CVRP, but we also review adaptations that extend SISR to representative VRP variants.

Unless stated otherwise, this chapter is inspired by the original publication. Slight formal changes have been made in the presentation of the algorithms to provide a more detailed explanation of the approach. Those modifications were mainly motivated by challenges encountered during the implementation.

A solution $s = \{R, A\}$ to the VRP, consists of a set of routes R and a set of absent customers A . The customers in A are not visited by any route in R . All methods presented in this chapter work with solutions in this format and s serves as their input.

3.1 Large neighborhood search

SISR, as a method from the LNS family, explores a large portion of the solution space by iteratively destroying and repairing solutions. It employs only a single pair of destroy and repair methods, omitting the adaptive mechanism of more recent ALNS. However, simulated annealing metaheuristic is still incorporated to guide the search and

```

1: procedure LARGENEIGHBORHOODSEARCH( $s$ )
2:    $s_{\text{best}} \leftarrow s$ 
3:    $T \leftarrow T_0$ 
4:   while not reached  $\text{maxIteration}$  do
5:      $s' \leftarrow s$ 
6:     DESTROY( $s'$ )
7:     REPAIR( $s'$ , false)
8:     if  $f(s') < f(s) - T \ln(U(0,1))$ 
9:        $s \leftarrow s'$ 
10:    if  $f(s') < f(s_{\text{best}})$ 
11:       $s_{\text{best}} \leftarrow s'$ 
12:     $T \leftarrow cT$ 

```

Figure 3.1: Pseudo-code of the large neighborhood search heuristic for the distance minimization objective.

generate neighboring solutions. We have briefly described the SA in Section 2.4.3, but as it is a core part of the search, we now approach it more formally. SA was inspired by physics and is guided using a notion of temperatures. The system is first heated to an initial temperature of T_0 . The final temperature is denoted T_{final} , both are parameters of the search.

In Figure 3.1, the structure of the LARGE_{NEIGHBORHOOD}SEARCH meta-heuristic is detailed. An initial solution is an input to this procedure. First, the best solution s_{best} and the temperatures are initialized (lines 2 and 3). The initial solution is constructed in a naive manner, where each customer is assigned to a separate route. More sophisticated initialization procedures for SISR have not been applied.

The core of the algorithm consists of maxIteration iterations (lines 4-12). Each iteration starts by finding the neighbor solution s' . The neighbor originates from the current solution s , on which DESTROY and REPAIR methods are applied (lines 5-7). Those methods are detailed in the following sections. The obtained solution must be feasible, therefore the REPAIR is not allowed to return an incomplete solution.

The current solution is updated if s' meets the acceptance criterion (lines 8 and 9). When the objective function is lower than the best-

known solution so far, it is stored as a new s_{best} (lines 10 and 11). Finally, the temperature is decreased (line 12). The result of the search is held in the s_{best} at the end of the procedure.

Based on a number of iterations n , a cooling constant c is calculated as follows:

$$c = \left(\frac{T_{\text{final}}}{T_0} \right)^{\frac{1}{n}}. \quad (3.1)$$

The current temperature in any given iteration is a multiplication of the temperature from the previous step and the cooling factor

$$T_{k+1} = cT_k \quad \forall k \in \{0, \dots, n-1\}. \quad (3.2)$$

The continual temperature decrease controls the acceptance of worse solutions. Whether a new solution is accepted depends on a change (perturbation) in energy. The objective function $f(s)$ represents the total cost (or distance) of the solution s , i.e., $f(s) = \text{cost}(s)$. Since VRP is a minimization problem, the energy change is calculated as a difference between the objective values of a neighboring candidate s' and the current solution s :

$$dE = f(s') - f(s). \quad (3.3)$$

Consider a given iteration with temperature T . The neighbor is accepted to replace the current solution if a random value from a uniform distribution $\mathcal{U}(0, 1)$ is less than the energy threshold given by a Boltzmann factor $e^{(-dE/T)}$. This leads to the acceptance criterion

$$f(s') < f(s) - T \ln(\mathcal{U}(0, 1)). \quad (3.4)$$

3.2 Destroy heuristic

The purpose of the destroy heuristic is to remove customers from a solution, leaving some of them absent. This prepares space for the following repair. First, we explain the main idea behind SISR's DESTROY, which is designed to remove *adjacent strings* of customers from a solution. We refer to *string* as a consecutive sequence of customer requests

in a route. Strings contain at least one and at most all served customers in route.

To simplify search for the adjacent strings, SISR's destroy uses an *adjacency table*. For each customer, the table contains a list of all customers, sorted by ascending distance. This table can be precomputed once before the beginning of the search, as it remains unchanged throughout the execution.

Authors of SISR introduced a term *spatial slack*, a property concerning routing problems. A reachable area of a vehicle can be defined based on a limited travel distance. This area forms elliptical shapes around the edges of the route, with edge endpoints as the focal points of the ellipse. When a customer is removed (no longer visited), this area expands. Even though the basic CVRP does not limit the travel distance, the spatial slack of an altered route can be considered in relation to its original. For example, after removing a customer, the spatial slack increases but is limited by the travel distance of the original route. Another, more straightforward type of slack is a *capacity slack*, which refers to the unutilized capacity of a vehicle.

Introducing both described types of slack is important in a search for improving solutions. Removing customers that are spread across the solution (for example, random removal) inefficiently disperses the capacity and spatial slack across the solution. This motivates the adjacency in the DESTROY.

On the contrary, removal based solely on customers being geographically close to each other may fail to capture detours that reach more distant customers. To address this, the simplest idea would be just to increase radius of the removed region. However, this approach would explode the number of removed customers, damaging routes beyond what is needed. Due to this, strings of customers are being removed by DESTROY.

Since DESTROY is based on string removal, it has to balance between the number of affected routes and the number of customers removed per route to introduce required amount of slack. Let s be a solution that is going to be destroyed. The method is led by parameter $\bar{c}$, which can be seen roughly as an average number of removed customers. Number of removed customers from a single route $l_{\max}$ is capped by average route size and a parameter $L_{\max}$:

$$l_{\max} = \min \left\{ L_{\max}, \overline{|r \in R|} \right\}. \quad (3.5)$$

To decide how many routes should be affected by the removal, a maximal number of changed routes $k_{\max}$ is calculated:

$$k_{\max} = \frac{4\bar{c}}{1 + l_{\max}}. \quad (3.6)$$

The authors of SISR did not comment on the constant four in the numerator. However, in the standard parameter configuration (discussed later), it plays an essential role by effectively setting the range of removed routes to a value between 1 and 3. Note also that in special cases, $k_{\max}$ can exceed $|R|$, i.e., a number of routes in the solution. This must be considered to avoid undefined behavior.

To ensure diversity, the final number of ruined routes k is randomly selected from a uniform distribution and rounded down:

$$k = \lfloor \mathcal{U}(1, k_{\max}) \rfloor. \quad (3.7)$$

Knowing the number of affected routes, the decision on how many customers will be removed from a specific route remains. Let $r \in R$ be a route from s . The amount of removed customers l_r from r is limited by previously computed $l_{\max}$ and the size of the route $|r|$. The value of l_r is randomly sampled from a uniform distribution considering those bounds:

$$l_r = \lfloor \mathcal{U}(1, \min\{|r|, l_{\max}\} + 1) \rfloor. \quad (3.8)$$

A *removal operator* is a helper heuristic designed to ruin a single route r . It selects l_r customers from r . The choice is centered around pivot customer $c \in r$, which is an additional input. Two removal operators based on strings of customers are used:

STRING(r, l_r, c) returns a single random string r' within route r . The string is sampled so it has a length equal to l_r and contains the pivot customer c .

SPLITSTRING(r, l_r, c) introduces an additional parameter $\beta \in [0, 1]$, called the *split depth*. Similar to the **STRING** operator, first a random string r' containing c is selected. However, the split depth allows

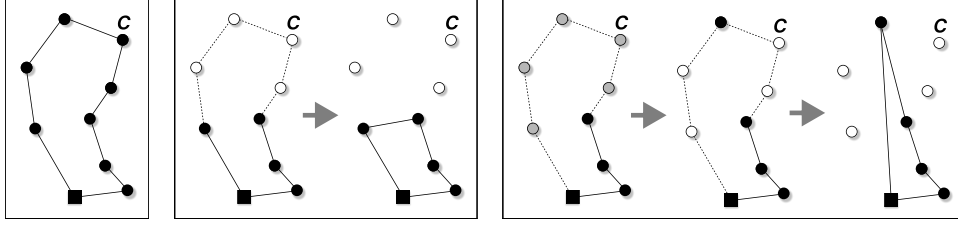


Figure 3.2: Example of the string removal operators selecting four customers.

for $m \in \mathbb{N}$ consecutive customers from r' to be preserved in the route. Due to this, the length of the initial r' is increased to $l_r + m$. To have an effect, m must be greater than zero and, together with the removed string, must not exceed the route length, i.e., $m \in [1, |r| - l_r]$. The value of m starts at one and is obtained iteratively. In each iteration, if a randomly generated number from $\mathcal{U}(0, 1)$ is greater than β , m is increased by one. If the inequality does not hold or if the upper bound is reached, the process terminates.

The starting position of preserved customers within r' is random. If the beginning or end of r' is included, the `SPLITSTRING` returns a single string, otherwise it cuts r' into two strings.

Example usage of the removal operators with $l_r = 4$ is shown in Figure 3.2. Note that the only one of multiple possible results of both operators is depicted. An example route with a pivot customer c is shown on the left. Customers marked in black are planned to be served by the route. The `STRING` removal is applied in the middle. Customers depicted in white are selected and ultimately removed. On the right, `SPLITSTRING` with $m = 1$ first selects a longer candidate string r' (colored in grey), from which one random customer is preserved in the next step, while the remainder is selected for removal.

The structure of the `DESTROY` operator is presented in Figure 3.3. First, the number of ruined routes k is determined based on calculations from Equations 3.5, 3.6 and 3.7 (line 2). Note that k also represents the number of removed strings, as an assumption is made that exactly one string is removed from each affected route. Next, the set of ruined routes is initialized as an empty set (line 3).

```

1: procedure DESTROY( $s$ )
2:   calculate  $l_{\max}, k_{\max}, k$  using Equations 3.5, 3.6 and 3.7
3:    $R^- \leftarrow \emptyset$ 
4:    $c_{\text{seed}} \leftarrow \text{RANDOMCUSTOMER}(s)$ 
5:   foreach  $c \in \text{adjacency}(c_{\text{seed}})$  such that  $c \notin A$ 
6:      $r \leftarrow$  route in  $R$  such that  $c \in r$ 
7:     if  $r \notin R^-$ 
8:       calculate  $l_r$  using Equation 3.8
9:       if  $\mathcal{U}(0, 1) < \alpha$ 
10:         $A \leftarrow A \cup \text{STRING}(r, l_r, c)$ 
11:       else
12:         $A \leftarrow A \cup \text{SPLITSTRING}(r, l_r, c)$ 
13:        $R^- \leftarrow R^- \cup \{r\}$ 
14:       if  $|R^-| = k$ 
15:         break

```

Figure 3.3: Pseudo-code of the DESTROY procedure.

The whole execution of a single destroy operator is centered around a single customer, c_{seed} . Function `RANDOMCUSTOMER` samples c_{seed} from all customers, including possibly absent ones (line 4). The adjacency list corresponding to a c_{seed} row in the adjacency table is then iterated according to the given order (lines 5-15). Customer c from the list is considered only if it is served in s . The route $r \in R$ that visits c is identified (line 6). If r has not yet been ruined, the algorithm proceeds to do so (line 7).

The length of the removed string, l_r , is calculated employing Equation 3.8 (line 8). The choice of a removal operator is random, based on parameter $\alpha \in [0, 1]$, referred to as *split rate* (line 9). Removal operators return a set of newly removed customers, which is added to the absent customers set (lines 10 and 12). Finally, r is added to ruined routes set (line 13).

If the limit of k ruined routes gets reached, the procedure breaks out of the loop (lines 12 and 13). After the completion of the procedure, s is a destroyed solution with newly added absent customers.

3.3 Repair heuristic

The repair heuristic in SISR extends widely used greedy insertion. A purely greedy insertion heuristic considers all feasible positions for all absent customers. A customer is placed in a position with the smallest increase in cost, repeating until all customers are served. However, inserting consistently at the overall best place has a significant weakness. It tends to postpone inserting the most expensive requests, ending with very few poor options when most routes are already full [42]. To overcome this problem, SISR first orders absent customers using a predefined sorting rule. Then, the absent customers are inserted into the solution in this fixed order rather than relying solely on the most cost-effective positions. The sorting method is chosen randomly from four predefined options:

RANDOM shuffles the customers.

DEMAND orders customers by descending demand.

FAR orders customers in decreasing distance from the depot.

CLOSE orders customers in increasing distance from the depot.

These sorting methods are selected with respective weights of 4, 4, 2, and 1, where a higher weight corresponds to a higher probability of selection. The choice of these exact values is not further discussed in the paper, and they are not treated as parameters. Therefore, it is unclear whether they were chosen through a tuning process or empirically.

The REPAIR procedure is detailed in Figure 3.4. Input to the method is a solution and a boolean variable *allowAbsence*. The purpose of this flag is to distinguish whether s should be maintained feasible or if customers are allowed to be unserved. The role of such incomplete solution is explained in Section 3.4, describing the fleet minimization algorithm. This distinction between the two types of REPAIR was not explicitly addressed in the SISR paper.

The procedure begins by sampling a sorting method based on the predefined weights and ordering the absent customers accordingly (lines 2 and 3). In order to not interfere with A , a new set of absent customers is initialized A_{new} as an empty set (line 4).

```

1: procedure REPAIR( $s, allowAbsence$ )
2:   pick sorting algorithm
3:   SORT( $A$ )
4:    $A_{\text{new}} = \emptyset$ 
5:   foreach  $c \in A$ 
6:      $p \leftarrow \text{none}$ 
7:     foreach  $r \in R$ , such that  $r$  has enough capacity for  $c$ 
8:       foreach  $p_r \in r$ 
9:         if  $1 - \gamma < \mathcal{U}(0, 1)$ 
10:          continue
11:         if  $p = \text{none}$  or  $\text{COSTAT}(c, (r, p_r)) < \text{COSTAT}(c, p)$ 
12:            $p \leftarrow (r, p_r)$ 
13:       if  $p \neq \text{none}$ 
14:         insert  $c$  at  $p$ 
15:       else
16:         if  $allowAbsence$ 
17:            $A_{\text{new}} \leftarrow A_{\text{new}} \cup \{c\}$ 
18:         else
19:            $R \leftarrow R \cup \{\text{new route with } c\}$ 
20:    $A \leftarrow A_{\text{new}}$ 

```

Figure 3.4: Pseudo-code of the REPAIR procedure.

The procedure iterates through all customers c in the given order and tries to find a suitable insertion position p in the existing routes (lines 5-19). Insertion position is a tuple consisting of a route and a specific location in that route. Initially, p is unassigned (line 6).

All routes $r \in R$ with sufficient remaining capacity are iterated as candidates for insertion (lines 7-12). Within each eligible route, all insertion positions p_r are considered (lines 8-12). There are $|r| + 1$ possible locations within r .

Blink rate $\gamma \in [0, 1]$ brings another non-determinism into the search. It is a probability (typically close to zero) of skipping a position (lines 9 and 10). As a result, the best possible spot is selected with a probability $1 - \gamma$. Alternatively, with probability γ , it gets blinked over, resulting in a different choice. Selecting the second best position

has probability $\gamma(1 - \gamma)$ and so on. The blinks are performed without knowledge of the actual quality of the insertion, as each is evaluated independently. Authors show in the paper’s appendix that the blink rate is a computationally less demanding alternative to *heuristic-biased stochastic sampling* [56], a closely related method that first ranks all positions and then samples from them, with selection biased toward lower-ranked positions.

The insertion position p is updated whenever a lower-cost placement is found or if $p = (r, p_r)$ is the first to reach the eligible position (lines 11 and 12). Note that for CVRP, the capacity is the only constraint, so the eligibility was assured in line 7. For other problem variants, the corresponding additional constraints need to be checked. Function `CostAt` calculates the increase in total cost when inserting customer c at p .

The customer is inserted if the valid position was found while iterating routes (lines 13 and 14). However, a new rearrangement of the nodes may result in a situation where none of the existing routes have enough capacity to include the customer. This case is handled depending on the *allowAbsence* parameter. If absent customers are allowed, c will not be served and gets inserted into A_{new} (lines 16 and 17). In case of denied absences, a new route that visits only c is created and added to the solution (lines 18 and 19). Finally, the absence list is actualized (line 20). At the end of the procedure, s contains the repaired solution.

3.4 Fleet minimization

As stated in Chapter 2, heuristics for variants such as VRPTW are traditionally used to solve a hierarchical objective: 1) minimize the number of vehicles and 2) minimize total cost. To resolve these conflicting objectives, SISR uses a two-stage approach. In the first phase, an absences-based acceptance criterion is used as an optimization strategy for minimizing the number of vehicles. Then, distance minimization is performed as described before, under the additional condition that solutions using more vehicles are always rejected.

For the fleet minimization phase, the authors of SISR designed a new approach. Before a detailed description, we first describe the

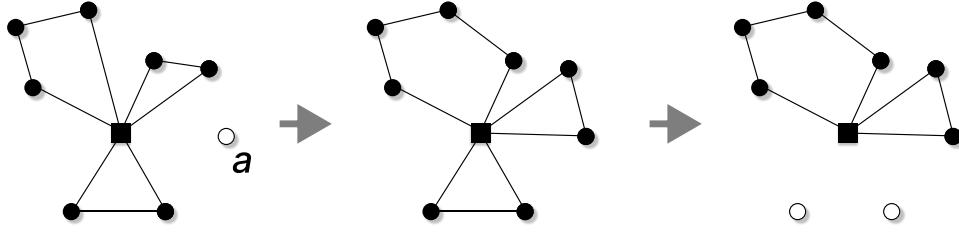


Figure 3.5: Example iteration of the `FLEETMINIMIZATION` procedure.

main concept of the method. The `FLEETMINIMIZATION` procedure is a large neighborhood search heuristic similar to the one described in Section 3.1. It strictly limits the number of vehicles used in solutions and allows for the possibility of not serving all customers. To explore the neighborhood, the procedure uses previously introduced `DESTROY` and `REPAIR` methods. A newly designed acceptance criterion prioritizes solutions serving customers that were previously often absent. Finally, finding a solution that serves all customers will result in lowering the routes limit.

An example iteration of the `FLEETMINIMIZATION` procedure is shown in Figure 3.5. The solution on the left serves three routes and contains an absent customer *a*. After performing a pair of the `DESTROY` and `REPAIR` methods, all customers happen to be served in the middle. One of the routes gets removed on the right, and the procedure continues by exploring solutions with just two routes.

The `FLEETMINIMIZATION` procedure is detailed in Figure 3.6. The procedure begins by initializing solution s_{best} , representing a feasible solution with so far the lowest number of vehicles (line 2). An absences counter `ABSENCE[c]` for each customer c in the problem is set to zero (lines 3 and 4). It represents the number of solutions explored during the entire search in which customer c was absent.

The main loop of the search is set to perform *maxIterationFleet* iterations. Each iteration begins by copying the current solution $s = (R, A)$ into s' and using `DESTROY` and `REPAIR` procedures to create a neighbor solution $s' = (R', A')$ (lines 6-8). During the repairing step, the solution must not use additional vehicles, but the absences are allowed.

```

1: procedure FLEETMINIMIZATION( $s$ )
2:    $s_{\text{best}} \leftarrow s$ 
3:   foreach  $c \in s.\text{customers}$ 
4:      $\text{ABSENCE}[c] \leftarrow 0$ 
5:   while not reached  $\text{maxIterationFleet}$  do
6:      $s' \leftarrow s$ 
7:     DESTROY( $s'$ )
8:     REPAIR( $s'$ , true)
9:     if  $|A'| < |A|$  or  $\text{SUMABS}(A') < \text{SUMABS}(A)$ 
10:       $s \leftarrow s'$ 
11:     if  $A' = \emptyset$ 
12:       $s_{\text{best}} \leftarrow s'$ 
13:      remove  $r \in R$  with the lowest  $\text{SUMABS}(r)$ 
14:     foreach  $c \in A'$ 
15:        $\text{ABSENCE}[c] \leftarrow \text{ABSENCE}[c] + 1$ 

```

Figure 3.6: Pseudo-code of the FLEETMINIMIZATION procedure.

The current solution is updated to s' if the absences-based acceptance criterion is satisfied: either s' has fewer absent customers than s , or the total sum of absences for A' is lower than for A (lines 9 and 10). The function SUMABS calculates the sum of absences with respect to the current ABSENCE counter, where $\text{SUMABS}(A) = \sum_{c \in A} \text{absence}[c]$.

When A' is empty, all customers are served by R' , making s' feasible. This implies that the new minimum number of vehicles is $|R'|$, and s_{best} is updated (lines 11 and 12). In subsequent iterations, solutions using $|R'| - 1$ vehicles should be explored. To achieve this, the route $r \in R$ with the lowest sum of absences of its customers $\text{SUMABS}(r)$ is removed from the solution (line 13). As a result, customers previously visited by r are now absent, and s becomes infeasible again.

At the end of each iteration, the absence counters are increased based on A' (lines 14 and 15). After the completion of the procedure, s_{best} is a feasible solution with the lowest number of vehicles found.

3.5 Parameter settings

All procedures presented in this chapter contain parameters that play a key role in the performance of the algorithms. Values were stated in the SISR paper for reproducibility. For some of the parameters, the authors performed a more extensive tuning. Different combinations were found to yield better results depending on the problem variant. Nevertheless, the authors unified the parameters when conducting experiments. For completeness, in Appendix B, we summarize those values and briefly comment what effect the different values can have on the search. For more details about the parameter tuning approaches and complete results, we refer the reader to the original paper [3].

3.6 Adaptations to problem variants

The authors briefly presented and evaluated various modifications to SISR to address different problem variants. In this section, we examine two adjustments in greater depth as representative examples. We use the time windows and the pickup and delivery problem to highlight important adaptations of the methods presented in this chapter. Additionally, we connect these modifications to problem variants VRPTW and PDPTW presented in Section 2.1.1.

3.6.1 Time windows

The REPAIR procedure needs to be extended by a feasibility check emerging from the time windows. In more detail, vehicles are assumed to depart from customers as soon as possible, which is the best choice from a feasibility point of view. Then, for each position in the route, the largest possible increase in the departure time (*time slack*) is computed in linear time, proportional to the route size. Using these precomputed values, the insertion feasibility itself is checked in constant time [57]. The time slack values are recomputed only after a customer is inserted or removed from the route.

Thanks to the additional variables of time windows, new possibilities for customer sortings in REPAIR emerge. SISR authors constructed the following three additional sorting methods:

TW-LENGTH orders customers by increasing time window length.

TW-START orders customers by increasing time window start.

TW-END orders customers by decreasing time window end.

Each of these new sorting methods has a corresponding weight of 1, while the sortings introduced in Section 3.3 are also used and their weights remain the same. The weights are not originally provided in the SISR paper. However, one of the authors, G. Vanden Berghe (personal communication, November 28, 2024), shared those values.

Those modifications directly corresponds to the VRPTW variant and allows the algorithm to solve it.

3.6.2 Pickup and deliveries

In the **DESTROY** procedure, it may happen that after removing requests, their corresponding pickup-delivery pair remains in the route. The method needs to be changed so that any unpaired requests are removed after the application of the removal operator. This modification effectively increases the number of total removed requests, which is not addressed in the paper, so we assume that parameters do not reflect this change.

The **REPAIR** procedure has to consider non-monotonous changes of vehicle capacity throughout the route. The calculation of varying capacity at each node is analogous to the time slack described above, again allowing for a constant time check of the capacity constraint.

The **REPAIR** is also modified to consider the pickup-delivery node pair. After finding a feasible place for the pickup node, the positions for its corresponding delivery must also be explored. This introduces an additional nested loop before the insertions cost evaluation.

Additionally, new sorting methods in **REPAIR** are used. Even though they are not presented in the SISR paper, G. Vanden Berghe (personal communication, November 28, 2024) disclosed that three additional sortings were used, each with weight of 1:

PD-DISTANCE orders requests by decreasing distance between pickup and delivery nodes.

CLOSE-PICKUP orders requests by increasing distance of the pickup location from the depot.

CLOSE-DELIVERY orders requests by increasing distance of the delivery customer from the depot.

To apply SISR on the widely researched PDPTW variant, both time windows and pickup and delivery modifications have to be applied simultaneously. This adds one additional layer of complexity, as a time windows feasibility check has to be nested for pickup-delivery pairs. Each time a pickup location is fixed, the time slacks inside the route change and must be recalculated before finding a suitable spot for the delivery.

4 Approaches for orienteering problems

This chapter explains how the original SISR was adapted to implement the solver for orienteering problems. We follow the structure of Chapter 3, explaining the key differences in the methods. The most important changes occur in the REPAIR procedure, for which updated pseudo-code is provided. Fleet minimization is not discussed, as the number of vehicles is fixed. The methods presented are applicable for (team) orienteering problems, but we also describe specific changes to the representative OP variants and summarize newly introduced parameters.

4.1 Large neighborhood search

As was stated in the problem formulation, the objective dramatically changes in the case OP. The quality of the solution strictly depends on its total score, and the goal of maximizing this value drives the search. The underlying metaheuristic LARGE NEIGHBORHOOD SEARCH from Section 3.1 can be used for OP even though it was formerly defined for distance minimization tasks. This is realized by generalizing the objective function f from the acceptance criterion. For a solution s of OP, we put $f(s) = -score(s)$, effectively converting the problem to a minimization context. The rest of the equations can remain unchanged, i.e., energy change stays as $dE = f(s') - f(s)$.

Optimizing the score brings a new challenge that is not present in the case of VRP. Different solutions can share the same objective value, complicating the navigation in the solution space. For example, simply reordering customer visits in a solution might change its available slack, but the objective function does not capture the change. Vidal et al. [58] named this a *staircase aspect*, and to counter this issue in their solver, they used a weighted total solution cost as a secondary objective. On the other hand, state-of-the-art hybrid solver HALNS, described in Section 2.4.4, performs effectively without such addition.

Inspired by the work of Vidal et al., we introduce parameter ω that penalizes the distance in the objective function. The final form of the objective function for OP is $f(s) = -score(s) + \omega \cdot cost(s)$, where $0 \leq \omega < 1$.

4.1.1 Initial solution construction

Since the number of vehicles is limited in OP, the initial solution generation method used for VRP with an unlimited fleet (as described in Section 3.1) is not applicable. Inserting a single customer per vehicle would leave most customers absent, making the approach ineffective. OP allows feasible solutions that include empty routes. Therefore, we adopt a straightforward strategy and initialize all vehicles as empty at the beginning of the search. This approach remains consistent with the procedure, and the construction of the first non-trivial solution happens in the first iteration of LNS, where the REPAIR method begins the search process.

4.2 Destroy heuristic

We use the unchanged DESTROY procedure from Section 3.2. The original adjacent string removal can be applied to OP without modification. The heuristic is naturally compatible with solutions where only part of the customers are visited, which is ideal for OP. The slack introduced by this procedure brings enough diversity and prepares the ground for the critical search phase, which lies in the repair step. While one might consider enhancements such as score-aware seeding, where nodes with lower scores have a higher probability of being selected as pivots for removal, such modifications would go against the original simplicity of the method.

4.3 Repair heuristic

The repair step for OP is based on the original REPAIR procedure from Section 3.3. Changes to the procedure are made in multiple places; a pseudo-code to REPAIR_{OP} is presented in Figure 4.1. Unused lines from the original method are crossed out, while the additions are colored blue. In this section, we describe only the most important parts of the algorithm and the changes made. Detailed discussions are provided in corresponding subsection.

The method starts identically as for VRP, with the sorting methods used to define the order in which customers are inserted. However,

```

1: procedure REPAIROP( $s$ )
2:   pick sorting algorithm
3:   SORT( $A$ )
4:    $A_{\text{new}} = \emptyset$ 
5:   foreach  $c \in A$ ,  $c$  selected using Shaw method
6:      $p \leftarrow \text{none}$ 
7:     foreach  $r \in R$  such that  $r$  has enough capacity for  $c$ 
8:       foreach  $p_r \in r$ 
9:         if  $1 - \gamma < \mathcal{U}(0, 1)$  or  $\text{Cost}(r \text{ with } c \text{ at } p_r) > T_{\text{max}}$ 
10:          continue
11:         if  $p = \text{none}$  or  $\text{CostAT}(c, (r, p_r)) < \text{CostAT}(c, p)$ 
12:            $p \leftarrow (r, p_r)$ 
13:       if  $p \neq \text{none}$ 
14:         insert  $c$  at  $p$ 
15:       else
16:         if allowAbsence
17:            $A_{\text{new}} \leftarrow A_{\text{new}} \cup \{c\}$ 
18:         else
19:            $R \leftarrow R \cup \{\text{new route with } c\}$ 
20:    $A \leftarrow A_{\text{new}}$ 

```

Figure 4.1: Pseudo-code of the REPAIROP procedure.

the accompanying sort methods (used in lines 2 and 3) are redesigned to fit the problem. The rationale behind choosing the new methods follows in Section 4.3.1. Additionally, the absent customers are not inserted in the strict order given by sorting method, but randomness is introduced by using a *Shaw method* (line 5). This approach is described in detail in Section 4.3.2.

Each candidate insertion position p_r must be additionally checked against the time limit constraint on the route length T_{max} . This condition is added at line 9. The sum of the travel and service times of route r together with the candidate customer c inserted at p_r must not exceed the time limit.

The selection and assignment of the best insertion position remain unchanged (lines 11–14), continuing to rely on the lowest insertion

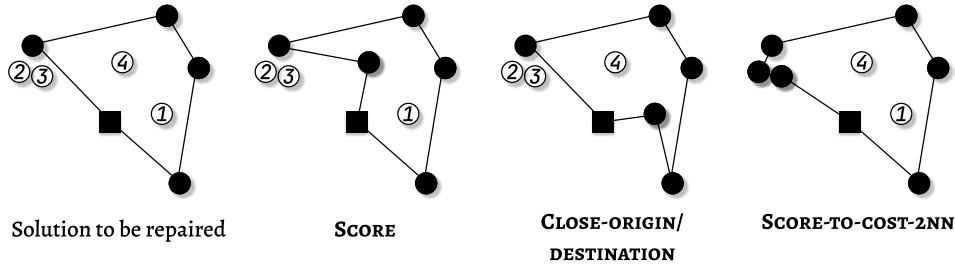


Figure 4.2: Example effect of different sorting methods in REPAIROP procedure.

cost combined with blinks. If no feasible position is found, the fleet size limit must be enforced. REPAIROP exclusively operates in a mode that allows customer absences, therefore the route creation (lines 18 and 19) is omitted.

4.3.1 Sorting methods

New objective and the differences in problem formulation create possibilities for new sorting methods. Some of the orderings used for VRP are not applicable. Namely, ordering customers by decreasing distance from the depot is not ideal, as visiting distant nodes significantly reduces the route slack. Given the limited time and need to select a subset of customers to serve, it is generally better to prioritize nodes located closer to the depot. For simplicity, we consider only operators that are also applicable to the TOPTW and CTOP variants. The following sorting methods are used for OP:

RANDOM shuffles the customers.

SCORE orders customers by decreasing score.

CLOSE-ORIGIN orders customers in increasing distance from the origin depot.

CLOSE-DESTINATION orders customers in increasing distance from the destination depot.

SCORE-TO-COST-KNN orders customers by the ratio between their score and the sum of distances to its k geographically nearest nodes. The precomputed adjacency table introduced in Section 3.2 can be utilized for efficient computation, as it contains the nodes sorted by their distance. The number of k neighbors is adjustable, creating more possibilities to include multiple methods. This method does not consider how the current solution looks like, making it as fast as the previous sortings. However, it effectively combines the score with the locality of the customers.

An example effect of the sorting methods is demonstrated in Figure 4.2. First, an input solution to the REPAIR is shown on the left. The solution serves just one route, with a black square depicting both the origin and destination depot. White-colored nodes are numbered from 1 to 4, and the number corresponds to their scores. The following three graphs depict repaired solutions, where the ordering SCORE-TO-COST-KNN with $k = 2$ resulted in the highest score increase of five in comparison to the other methods, increasing the score by four or one.

SCORE-TO-COST-STATIC orders customers by the ratio between their score and the lowest possible distance increase. The distance increase is calculated for each customer, considering each feasible insertion position and choosing the one with the smallest increase in cost. Values of this metric are calculated at the beginning of the REPAIR method and are not further updated after a successful insertion. This is consistent with the SISR repair strategy, which first orders the requests and then inserts them according to this static order. An alternative approach recalculates score-to-distance ratios after each insertion [23] is more common in OP literature but can significantly increase computational effort.

The weights of the sorting methods and the overall effect of the newly designed ones are addressed in Chapter 5.

4.3.2 Shaw removal inspired shuffle

To introduce more randomness and enhance search diversification, we designed a stochastic rank-based selection of nodes. The sorting

methods (with exception of RANDOM) rely on static values and maintain a rigid ordering. As a result, the list of absent customers tends to remain unchanged across iterations, making it highly unlikely for customers ranked lower in the ordering to be selected.

Shaw removal [59] is a destroy heuristic designed for LNS-based solvers of VRP. It selects a set of similar customers for removal based on a specified criterion. The customers are ranked in decreasing order and then sampled individually until the desired amount is selected.

In our approach, we use the Shaw removal to select requests from the ordered list of absent customers, replacing the simple iteration of the sorted set A . This effectively adds noise into the ordering. To avoid confusion with the original purpose of the heuristic, we refer to our adaptation as the *Shaw method*.

The Shaw method selects a customer $c \in A$. It is guided by a parameter $p \in [0, 1]$, controlling how much should be the original ordering respected. An index i from the ordered set A is sampled as follows:

$$i = \lfloor |A| \cdot (1 - \mathcal{U}(0, 1)^p) \rfloor, \quad (4.1)$$

where $\mathcal{U}(0, 1)$ is an uniform distribution. The customer $c = A[i]$ is subsequently selected for reinsertion and removed from the set of absent customers, i.e., $A = A \setminus \{c\}$. The probability of selecting the lower-ranked customers exponentially decreases. When $p = 0$, the highest-ranked customers are always selected, and in the case of $p = 1$, the choice is completely random. Note that for efficiency, if the initially picked sorting algorithm was RANDOM, the Shaw method can be omitted.

4.4 Parameter settings

We have introduced two new parameters: the distance weight in the objective function, ω , and the guiding parameter p for the Shaw method. Both can significantly influence the behavior of the method and need to be tuned. However, they also support default values that keep the original algorithm's behavior. Some parameters from SISR might also need adjustments to better capture the specifics of OP's. The weights of the sorting methods should also be reconsidered, with some methods

removed and others included. The following chapter explores these parameters in more detail and presents the tuning process used to optimize their values.

4.5 Adaptations to problem variants

This section describes how the previously presented methods were adjusted to handle the representative problem variants introduced in Section 2.2.1. Our algorithm for OP does not rely on very specific mechanisms and maintains high adaptability. The methods can be extended to new problem variants similar to those described in SISR for VRP.

4.5.1 TOPTW

Adding the time window constraints leads directly to the TOPTW variant. The algorithmic changes are the same as described for VRPTW in Section 3.6.1. Specifically, in the repair method, time slack calculations are used to verify the feasibility of insertions, and sorting methods TW-LENGTH, TW-START and TW-END are included.

4.5.2 CTOP

CTOP variant differs solely in capacity constraints, which affect only the REPAIROP method. Similar to CVRP, these constraints require an additional feasibility check when evaluating insertion positions within a route.

A new sorting method is created to employ the capacities of the vehicles. The tradeoff of customers' score and demand might lead to a helpful guide in reinsertion:

SCORE-TO-DEMAND orders customers by decreasing the ratio between their score and demand. This idea was previously used, for example, by Gunawan et al. [60] in the creation of initial solutions for CTOP.

5 Experiments

This chapter focuses on the experimental evaluation of our solver. The SISR algorithm [3] is highly reproducible due to its single destroy-and-repair method. A direct comparison with the original work’s results is presented on the same benchmark datasets. Additionally, we analyze the behavior of the fleet minimization procedure, a core component of the approach for some variants of the vehicle routing problem.

In the second part of this chapter, we evaluate the performance of our proposed modifications to the original SISR algorithm, designed to address the orienteering problems family. Three representative problem variants are studied. First, the parameters are tuned on a subset of problem instances, and finally, the achieved results are presented and compared to current state-of-the-art approaches.

The experiments were conducted within the MetaCentrum [61] distributed computing infrastructure in the Czech Republic. The system environment was based on Debian GNU/Linux 12 (x86_64). Computations ran on nodes with AMD EPYC 7543 @ 2.80 GHz CPUs and 2× 3.84 TB NVMe SSDs. The available RAM was shared across up to 32 CPU cores, with actual memory usage typically around 5%, indicating that memory was never a limiting factor.

The solver was implemented in C++. To ensure efficiency, each problem variant was implemented in a dedicated code branch. This avoided the extra cost of using a more general solver, which is often inefficient for heuristics required to perform only the necessary subset of operations. The code was compiled using GCC 12.2.0, targeting the C++20 standard with -O3 optimization enabled.

5.1 Classical Vehicle routing problems

The implementation was experimentally evaluated on the same benchmark instances used in the original paper. We followed the parameter settings and replicated the experiments using our implementation for comparison. Although runtime is not directly comparable due to differences in hardware, we report it to validate the behavior of our implementation. Each time value in this section is presented in minutes.

The number of iterations varies between problem variants and typically is scaled with the instance size allowing more iterations for bigger problems.

5.1.1 Datasets

In all datasets, the distance is Euclidean, usually without rounding, but rounding details are mentioned when it is used. In the VRPTW and PDPTW variants, the travel time between two nodes is equal to the distance value.

The CVRP benchmark and other CVRP instances used in this study are available through the CVRPLIB website¹. Similarly, data for VRPTW and PDPTW were obtained from the Transportation Optimization Portal of Sintef.no². Both websites serve as comprehensive resources for problem instances and provide the most up-to-date best-known solutions.

CVRP

A widely used benchmark set by Uchoa et al. [62] of 100 instances is used for CVRP. The customer counts range from 100 to 1000. It contains carefully generated instances to address limitations of previous datasets, creating a solid basis for evaluation. The instance definitions contain the positions of the nodes and their demands. The distances are rounded to the nearest whole number. At the time of writing this thesis, 58 out of the 100 instances were solved to optimality.

The authors of SISR additionally provided results on 10 older and smaller CVRP datasets but they recommended sticking with this benchmark set.

VRPTW

The benchmark set introduced by Gehring and Homberger [63] was used in the SISR paper to compare the performance to other VRPTW solvers. We have conducted our experiments on this set, as it is the only one we can compare to the original paper. The set contains 200 to

1. <http://vrp.atd-lab.inf.puc-rio.br>

2. <https://www.sintef.no/projectweb/top/>

1000 customers, totaling 300 instances. The problems can be divided into thirty subsets of 10 based on size and properties. It is a standard benchmark with a hierarchical objective: first, minimize the number of vehicles and then minimize the distance. There is no rounding during calculation, and the calculations should use double precision.

PDPTW

A widely used benchmark set by Li and Lim [64] was employed by the authors of SISR to evaluate the performance of the PDPTW variant. This dataset has descriptive properties similar to those of the VRPTW benchmark. Problem sizes include 200, 400, 600, 800, and 1000 pickup or delivery requests. Two instances from the 1000-request group are missing, resulting in a total of 298 available instances. The objective is hierarchical, with the primary goal of minimizing the number of vehicles, followed by minimizing the total distance.

5.1.2 Evaluation

In this section, the three previously mentioned sets are compared to the results from the original paper. An analysis of the fleet minimization procedure is also included, with focus on its convergence, along with a small proposed change. For some of the comparison between our result and the original we use the gap:

$$\text{gap} = \frac{(\text{our} - \text{original})}{\text{original}} \quad (5.1)$$

Alternatively we use the percentage gap, defined as $\text{gap}(\%) = 100 \cdot \text{gap}$.

CVRP

To verify the performance of the implementation and SISR algorithm, we start by examining the CVRP variant, as this represents the most basic version of the algorithm. Figure 5.1 compares the results on Uchoa's benchmark set between our implementation and the original results reported in the online appendix of the SISR paper. Each problem instance was run 50 times, the same as in the paper. The figure is divided into three subplots, each describing a different aggregated

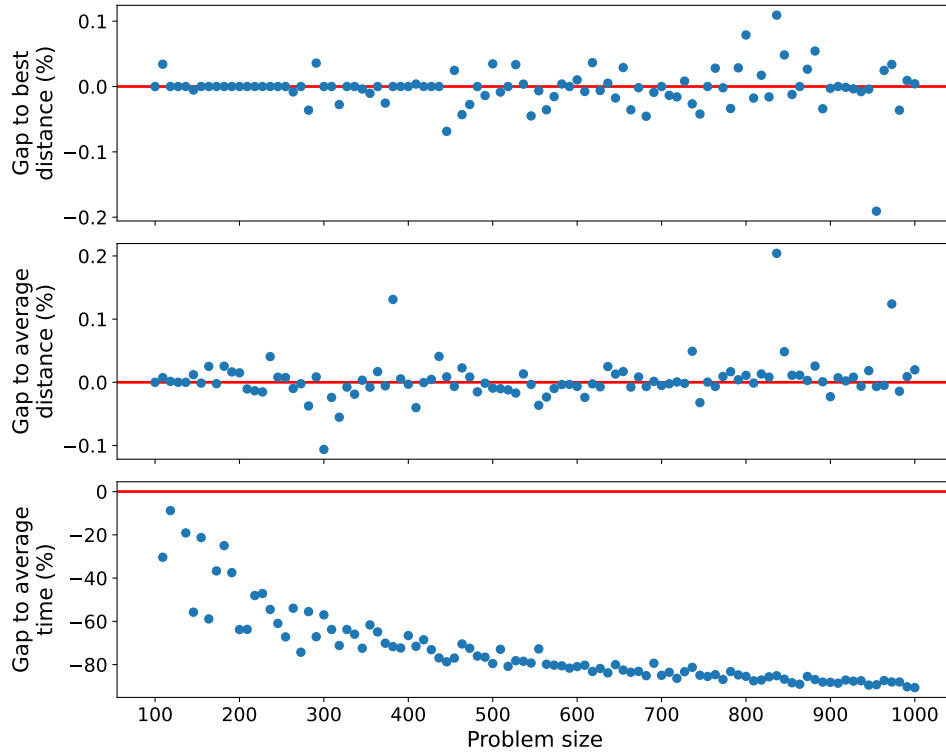


Figure 5.1: Comparison with the original SISR paper on the CVRP.

metric across instances: best distance, average distance, and average computation time. The X-axis shows the number of customers. The percentage gap on the Y-axis is calculated per instance for each metric. The red lines represent equal performance (zero percentage gap).

In the top part, which compares the best results, there are 35 instances where the exact same result was obtained. For 40 instances, our result is better (displayed below the line), and for 25 where it is worse (points above the line). We can conclude that those values are relatively balanced, and no approach has achieved a significantly better performance. The maximal absolute percentage gap is just 0.19%.

In the middle plot, the average distance is compared. Here, 48 instances lie above the line, 49 below, and 3 directly on it. The performance between the two solvers is again nearly balanced.

The computation time is shown at the bottom. Although the hardware is not directly comparable, a downward trend can be observed. Our implementation becomes increasingly faster as the problem size grows. For the 1000-customer instance, our solver runs more than 10 times faster. The advantage is less evident for smaller instances. For some, a known optimal solution is often reached, but we did not include an early-stopping criterion to terminate in such cases. One instance is excluded from the bottom part of the figure, as the algorithm swiftly converges to the optimum in all 50 runs. This results in our implementation reaching an average runtime 6 times larger than the original, which would skew the plot and make it unreadable.

Detailed results are available in a tabular form as a part of Appendix C.

VRPTW

The SISR paper reported the best results from only five runs of the VRPTW benchmark per instance. This enabled direct comparison with competitive approaches but may not capture possible solutions variation. The results were presented as mean of the best results over ten instances within subsets, which hides information about individual instances.

Reported distances should be interpreted with caution. In VRPTW, solutions with more vehicles typically result in shorter total distances. Even when a subset shows the same number of vehicles, there is no guarantee that the vehicle count is identical across all instances. We executed five runs per instance to compare with the numbers reported in the paper. The results are summarized in Table 5.1. The table shows the average fleet size for each of the 30 subsets. There are six subsets (e.g., R1, R2...) per problem size, each containing 10 instances. First, the best result per instance is selected and then averaged across the subset. Multiplying by 10 yields the total number of vehicles across a subset. Each cell shows values from the original paper (Orig) and from our implementation (Our). In 23 subsets, results are identical. The differences amount to only a few vehicles but grow with the problem size and are highlighted in bold. In the case of RC1, with 800 customers, the difference is the most evident. The likely explanation is that eight instances differed by only one vehicle, although this cannot

Size	200		400		600		800		1000	
	Orig	Our	Orig	Our	Orig	Our	Orig	Our	Orig	Our
R1	18.2	18.2	36.4	36.4	54.5	54.5	72.8	72.8	91.9	91.9
R2	4.0	4.0	8.0	8.0	11.0	11.0	15.0	15.0	19.0	19.0
C1	18.9	18.9	37.6	37.6	57.4	57.4	74.9	75.0	93.9	94.0
C2	6.0	6.0	11.8	12.0	17.4	17.5	23.4	23.5	29.1	29.2
RC1	18.0	18.0	36.0	36.0	55.0	55.0	72.8	72.0	90.0	90.0
RC2	4.3	4.3	8.4	8.4	11.4	11.4	15.4	15.4	18.2	18.2
Time	11.0	2.5	42.0	8.2	95.3	14.1	216.6	25.8	324.2	38.9

Table 5.1: Comparison of experiments on the VRPTW benchmark with the SISR paper, presenting the average number of vehicles per subset and the average computational time in minutes.

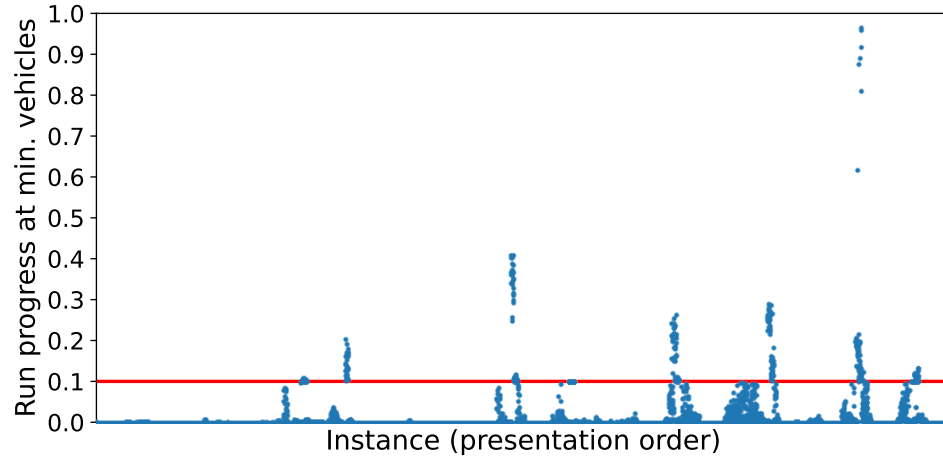
be confirmed. For completeness, we also compared distance across subsets with equal fleet sizes. The largest percentage gap was 0.027%. The last row of the table reports the average runtime per subset in minutes. The ratio between our times and the original behaves similarly to CVRP.

Due to incomplete results in the original paper and reasons of missing variety of runs, we have executed 30 runs on each instance. The complete tabular results from this extended experiment are included in Appendix C.

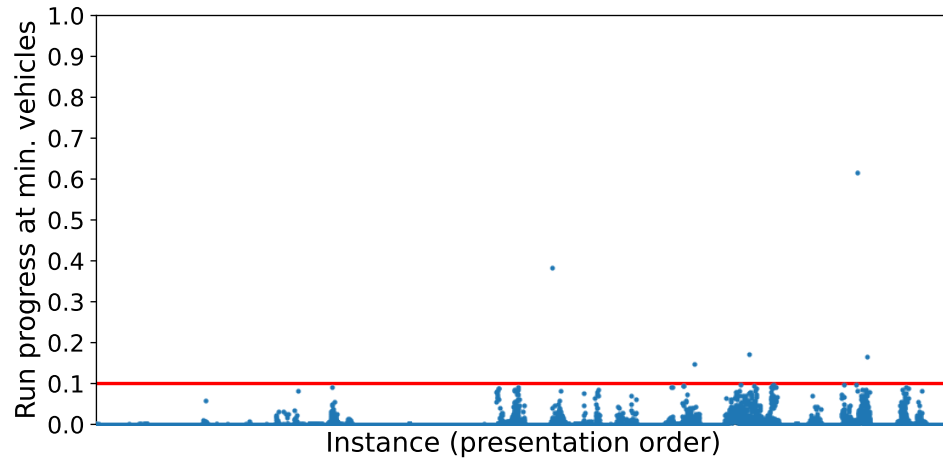
Analysis of fleet minimization procedure

The FLEETMINIMIZATION heuristic, presented in Section 3.4, is used in the first 10% iterations. The second phase minimizes the total distance in the remaining 90% iterations. The concept of absent customers from the original work successfully reached the performance of state-of-the-art algorithms on the fleet minimization task. We monitored how the solution progressed throughout the search to analyze convergence behavior.

Figure 5.2a shows the earliest iteration, in which the final lowest number of vehicles was reached. From that point, all subsequent iterations operated with a fleet of that size. The visualized data come



(a) Default



(b) Adjusted acceptance criterion

Figure 5.2: Normalized iterations at which the minimal fleet size was reached.

from our extended 30-run experiment on the VRPTW benchmark set. For presentation purposes, the instances on the X-axis are sorted by increasing customer count. As there are only five different instance sizes, ordering beyond that is arbitrary but consistent between subplots in the figure. The Y-axis shows a normalized iteration number, calculated as a fraction of the total iterations. The red horizontal line marks the boundary between the fleet and distance minimization phases. Most instances reached the minimal fleet within a small fraction of iterations. Others required more time during the first phase but eventually reached the minimum. There are multiple instances for which the fleet minimization happened shortly after the beginning of the second phase. In some cases, the fleet minimum is unexpectedly reached relatively late within the distance minimization.

Let us analyze the behavior of the absence list in the `FLEETMINIMIZATION` procedure. Consider a situation where only a few customers are absent from the solution. The solution is close to serving all requests and possibly removing a route. However, the remaining absent customers are too difficult to insert. Over the next iterations, their absence scores increase. At this stage, it becomes preferable to remove any customers from the solution and reintroduce those that have been absent for a long time. This is done according to the second part of the condition on line 9 in pseudo-code from Figure 3.6. In doing so, the algorithm may swap out several well-placed customers in favor of a single problematic one. This artificially reduces the absence score and satisfies the condition. The rationale behind the easier reinsertion later of those well-behaved customers is logical, but in some cases, a domino effect triggers it. The solution may quickly degrade as the algorithm selects customers with very low absence scores, disturbing well-structured routes. This phenomenon was observed, for example, on an instance with 600 requests, where the number of absent customers increased from one to 80 in just 70 iterations. The algorithm is generally robust and capable of recovering from such disruptions. Thanks to the large number of iterations, the absence counters balance out again potentially. This behavior still results in wasted effort and slower convergence. Only a subset of instances was affected by this behavior, but we could not identify any distinguishing features that would predict its occurrence.

Based on the procedure's behavior in problematic instances, we have modified the acceptance criterion. The condition on line 9 was changed to:

if $|A'| < |A|$ **or** $|A'| \cdot \text{SUMABS}(A') < |A| \cdot \text{SUMABS}(A)$

The first part remains unchanged, comparing the raw sizes of the absence sets between the current and neighboring solutions. In the second part, absence scores are weighted by the number of absent customers, aiming to rebalance the decision criterion. This penalizes solutions with both many absent customers and high cumulative absence scores.

Figure 5.2b presents the results of 30 runs on the VRPTW benchmark, incorporating the modified acceptance criterion. All parameters remained identical to the previous experiment. With few exceptions, the minimal fleet size is now reached during the first phase. However, for some instances, the minimum is reached notably later within the first phase. The numerical outcomes are largely similar. In two cases, the new procedure achieved a lower fleet size, conversely, in two other instances, the original procedure outperformed the new one by one vehicle. When comparing minimal distances (excluding the four instances with differing fleet sizes), the average percentage gap is 0.028% in favor of the original procedure. The most notable change is a reduction in total computation time by 9%. The highest speed-up observed on a single instance was 79%, whereas the greatest slowdown reached 30%. Such large fluctuations occurred in instances where the achieved fleet size differed. Serving fewer vehicles significantly reduced the available slack for reinsertion. Thanks to this, computationally expensive cost evaluation of reinsertion is less frequent during the repair phase. The method often failed to detect any feasible insertion positions in the first place.

In summary, the change in the acceptance criterion did not bring a significant improvement in the solution quality. It reliably eliminated the issue of non-convergence in the first phase for certain instances but at the cost of slightly worse performance in others. An approach that alternates between those strategies may be a promising direction to improve the algorithm's overall robustness.

Size	Original			Our		
	CNV	CTD	Time	CNV	CTD	Time
200	600	185257	6.3	601	183390	1.5
400	1133	444247	23.7	1136	432906	5.0
600	1629	906717	66.0	1629	889148	11.3
800	2119	1526559	140.1	2118	1501673	17.7
1000	2577	2276009	226.6	2576	2217682	31.2

Table 5.2: Comparison of experiments on the PDPTW benchmark with the SISR paper. Cumulative number of vehicles, cumulative total distances and average time in minutes.

PDPTW

For the PDPTW dataset, the SISR paper contains a simple table summarizing aggregate values of five groups, each corresponding to a different problem size. They report a cumulative number of vehicles, cumulative total distance, and average computational time. The solutions averaged are best of five runs. Each group of 60 (or 58) instances is represented by just three numbers. The results are displayed in Table 5.2. Due to the limited granularity, we can compare mainly the cumulative sums of vehicles. Compared to the original, our implementation found solutions requiring four additional vehicles across the groups. On the other hand, for two subsets, our solutions required one less vehicle. In terms of runtime, our implementation consistently outperforms the original, and the margin again increases with the instance size. For this dataset, the comparison is ever more limited. We have again executed 30 runs on each instance, and the results are included in the Appendix C.

The original SISR achieved impressive results, producing multiple new best-known solutions, mainly for the PDPTW variant. The authors attached exhaustive instance-wise results on the dataset by Li and Lim in the online appendix. However, those results are incomparable to ours, as they come from an unknown computational effort. Some of our solutions would have surpassed those reported in the original article if they were available when it was published (2020). Since then, many researchers actively worked on improving the best-

known solutions further, which is evident from the improvements posted on the Sintef website. It is also common practice to search for the new best-known results by using the existing ones as the initial solution. The involvement of companies like NVIDIA, with their GPU-accelerated solvers³ highlights the rapid progress in the field. As a result, no reportable improvements to the current best-known solutions for classical VRP have been made on our end.

5.2 Orienteering problems

In this section, we evaluate the performance of our approach for orienteering problems. First, the benchmark datasets used are described, followed by a parameter tuning process. Finally, comprehensive results are presented.

5.2.1 Datasets

It is common for the published problem instances to contain customer nodes that are completely unreachable from the depot. This issue should always be addressed before the actual solution process begins. Removing such requests is not difficult but crucial, as it can save significant computational resources. This phenomenon also occurs in several datasets we use in this section, and it is important to keep this in mind when working with the OP. There are other specifics related to the data, such as rounding, and we address these issues when they are encountered.

There is no dedicated site for the OP variants that actively manages the benchmark sets and maintains the best-known solutions. Some sites have repositories of data, but not on the scale of those previously mentioned for the VRP. The availability of particular solutions for analysis is also rare.

TOP

Commonly reported TOP benchmark instances in the literature were introduced by Chao et al. [65]. This set is insufficient for comprehen-

3. <https://www.nvidia.com/en-us/ai-data-science/products/cuopt/>

sive analysis, as many modern heuristic solvers have matched all the best-known solutions [53].

Dang et al. [49] created a larger benchmark set in 2013. After filtering out unreachable customers, the instances contain between 102 and 401 requests. These instances were derived from existing CVRP and TSP problems, with profits assigned as constant values, randomly selected, or based on customer demand. Each base instance is reused multiple times with varying fleet sizes (2 to 4 vehicles) and time limits. Although 333 instances were originally generated, only results for 82 non-trivial ones are commonly reported in the literature. One of the authors provides access to this data on their website⁴. We refer to this set as TOP-L, and it serves as the foundation for our experiments and comparison with other approaches.

More recently, Chaigneau et al. [66] introduced a large-scale benchmark set with instances containing between 1001 and 5394 customers and using 2 to 10 vehicles. The data generation process followed a similar approach as with TOP-L. This set contains 288 instances. Although reported as having at least 1001 customers, instances shrink after removing unreachable nodes. Some instances fall below 1000, or in rare cases, contain no serviceable customers at all. Preliminary analysis using our solver also revealed that in several instances, all reachable customers can be visited, which does not fit the philosophy of OP. The dataset is publicly available on GitHub⁵. We refer to this extended benchmark as TOP-XL and use it to diversify the tuning for OP parameters.

TOPTW

A base benchmark for the single-vehicle Orienteering Problem with Time Windows (OPTW) was introduced by Righini and Salani [67]. Montemanni et al. [68] extended this benchmark by introducing additional instances and varying the number of vehicles from 1 to 4, resulting in a total of 304 instances. We obtained the data from the Centre for Industrial Management at KU Leuven⁶. Two classical VRP datasets served as the basis for this set, whose characteristics depend

4. <https://www.hds.utc.fr/~moukrim/dokuwiki/en/top>

5. <https://github.com/CharlyChgn/VLS-Team-Orienteering-Problem>

6. <https://www.mech.kuleuven.be/en/cib/op>

on the original dataset from which it was derived. For example, distances are rounded down to either one or two decimal places. Such details are also provided on the aforementioned website. The customer capacities were directly transformed into profits. We use this set in our experimental evaluation, as it is a standard for evaluating TOPTW solvers. Problem sizes range from 48 to 288 customers, and in some instances, the number of vehicles is sufficient to serve all customers.

CTOP

We evaluate CTOP using the most recent and widely adopted benchmark dataset introduced by Tarantilis et al. [18]. The dataset contains three subsets of instances. The first subset contains 10 original instances in which all customers can be served. Subsets 2 and 3 use the node locations from the first subset but differ in problem specifications. These include varying fleet sizes (ranging from 6 to 8 vehicles), maximum route lengths, and vehicle capacities. The subsets contain 90 and 30 customers, respectively, with the number of customers ranging from 337 to 577. The authors randomly generated customer scores within an interval centered around their demands. The instance definitions were not publicly available but were kindly shared with us by Aldy Gunawan (personal communication, April 8, 2025), one of the authors of a recent CTOP algorithm [60].

5.2.2 Parameter tuning

Due to the large number of parameters in the SISR algorithm, an exhaustive grid search testing all combinations of parameters is not practically possible. The parameters of the original SISR were tuned for instances of similar size to those used in our OP datasets. In this thesis, we primarily focus on tuning of parameters introduced by the changes described in Chapter 4.

The parameter tuning is performed separately for the three previously introduced OP variants, as each has different characteristics. For each variant, a random subset of 20 instances is selected for tuning. Instances for which preliminary baseline tests consistently reached identical scores were excluded from the choice, as they provide limited insight. For TOP, the subset includes 10 instances from the TOP-L set

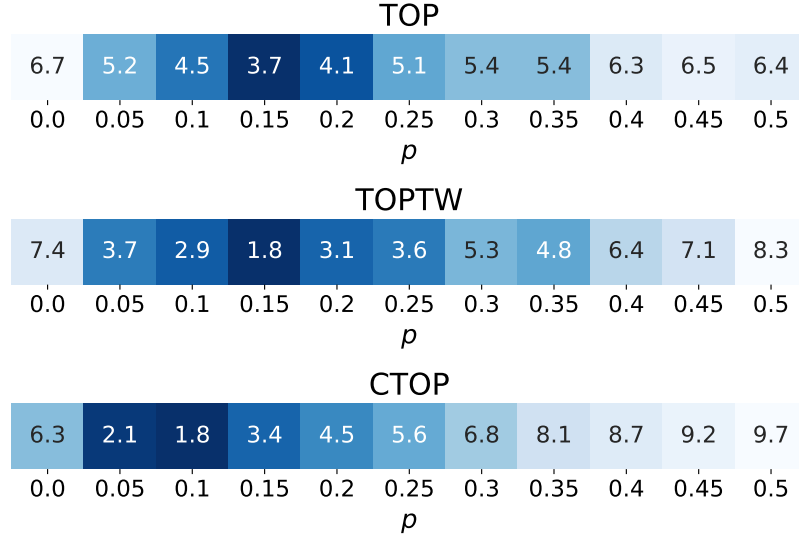


Figure 5.3: Performance for different values of Shaw method parameter.

and 10 from the TOP-XL set, restricted to those with less than 1000 customers.

We perform 20 independent runs per instance, which should be enough to capture the variability in performance. The quality of the parameter settings is compared using average ranks calculated from the best found solutions. In case of a tie, the average score across the runs is used.

Following the approach used in the original SISR paper, the number of iterations per instance is set according to the instance size. Larger problems require more iterations to reach high-quality solutions. This adjustment is identical for all three variants, we set the number of iterations as $maxIteration = |N| \cdot 25000$, where $|N|$ denotes the number of (reachable) customer nodes.

We also adjusted the initial temperature T_0 of the LNS, which was lowered to 50. This change was made because the scores in the OP datasets are typically much lower than the distances in the VRP.

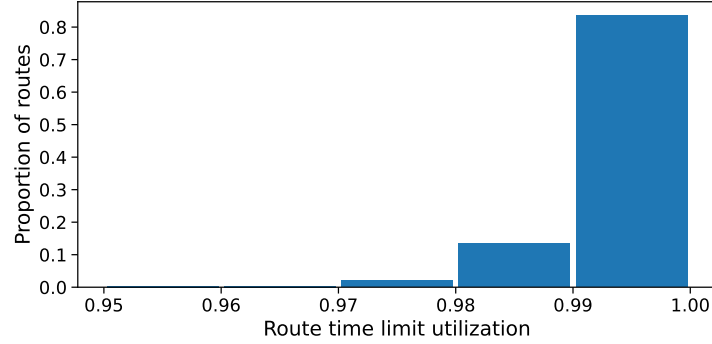
Shaw method

The first parameter considered for tuning is the Shaw method parameter p . It is tuned before all other parameters to establish a strong baseline, as the preliminary experiments indicated that the inclusion of the Shaw method significantly improves solution quality. Figure 5.3 presents the average ranks obtained on the tuning sets. Raising p slightly above zero is beneficial, with the lowest average ranks achieved at values of $p = 0.1$ for CTOP and $p = 0.15$ for TOP and TOPTW. Larger values lead to progressively worse performance. Based on these findings, we fix the value of p separately for each variant and proceed with tuning the remaining parameters.

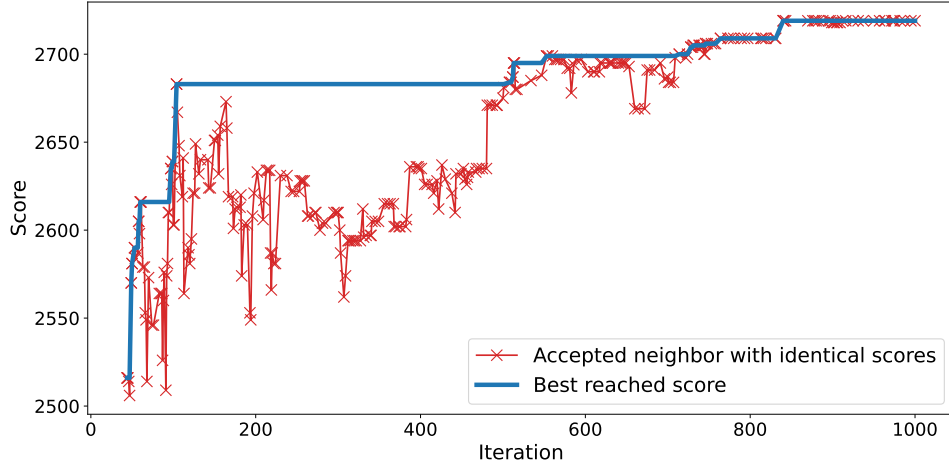
Distance weight

Another newly introduced parameter is the weight of distance penalization ω in the acceptance criterion of the LARGE NEIGHBORHOOD SEARCH. Its purpose is to guide the search to more efficient use of the time budget. While distance is not explicitly present in the objective function, it is implicitly important due to the route time constraint. Maximizing the use of this constraint should intuitively contribute to higher scores by creating space for serving more requests.

To see the relevance of distance utilization, we analyzed the fraction of the time limit used in the final solutions returned by the solver. For each route, we calculated the ratio of the traveled distance to the available time budget. Note that in this case, the distance and time are interchangeable. As shown in Figure 5.4a, over 80% of routes consume more than 99% of the available time. This supports the hypothesis that efficient use of distance is critical to solution quality, even though it is not directly optimized. The second part of the Figure 5.4b illustrates how the solution quality evolves during the search on the instance *bier127_gen3_m2* from the TOP-L set. The run lasted 1000 iterations, the first 40 iterations were excluded from the plot. Red cross marks represent accepted neighbor solutions that have the same score as the previously accepted solution. It highlights that solutions with equal scores can frequently be accepted in succession, even in earlier stages of the search, when the algorithm is more explorative. The blue



(a) Route time limit utilization by solutions of TOP instances.



(b) Score progression over time.

Figure 5.4: Analysis of solution behavior during the search.

line shows the best score achieved so far at each point, revealing a characteristic staircase pattern discussed earlier.

These observations motivated the introduction of the ω parameter to more effectively guide the search in such plateaus. Vidal et al. [58] set the penalization weight to 10^{-4} in their TOP solver. To evaluate its impact on our algorithm, we use four values: 0, 10^{-5} , 10^{-4} and 10^{-3} . We combine this with the newly introduced sorting methods, as described in the following section.

Sorting methods

The weights of the base sorting methods are copied from the original SISR for VRP, as the roles of these methods in the OP variants mostly correspond to their counterparts. Methods taken directly from VRP and their weights are the following: RANDOM=4, CLOSE-ORIGIN=1, CLOSE-DESTINATION=1. An exception is made when the starting and destination depot locations are identical. In that case, the CLOSE-DESTINATION is set to 0. The DEMAND method was replaced by SCORE in SISR-OP, but its weight of 4 is kept to preserve the original ratio among the methods. When adding new sorting methods, it is important to consider that it will lower the relative probability of selecting those base methods.

We combine parameter ω and the new sorting methods into a grid search. From the SCORE-TO-COST-KNN, we derived two methods with two different values of k . 2NN with $k = 2$ was added to capture tight relations to neighbors, while 5NN with $k = 5$ considers broader region. These are evaluated in combination with the SCORE-TO-COST-STATIC (STAT) method. Each of the three sorting methods (STAT, 2NN, 5NN) is assigned a binary weight, creating eight sorting configurations in total. Combined with values of ω (see page 57), this experiment has 32 different setups.

The average ranks obtained are presented in Figure 5.5. We now analyze the results variant by variant. For TOP, the best average ranks are obtained when STAT was excluded (i.e., its weight is set to zero). This sorting method has a clear drawback, placing a strong emphasis on the single most suitable insertion position for each node. If that position is taken before, the next option might be much worse, but the request must still be inserted. The combination with overall lowest average rank does not include 5NN sorting, although 5NN proved to be useful in some cases. The combination of 2NN and 5NN achieved the most stable average ranks across all values of ω . The distance weight $\omega = 10^{-5}$ achieves the best average ranks. Overall, we select the combination $\omega = 10^{-5}$ and 2NN & 5NN sorting for sake of generality.

TOPTW is not as strongly affected by the same drawback of STAT as TOP, but this sorting still does not contribute to the solution quality. Drawback is less significant because many insertions are infeasible due to the stricter constraints, so the nodes have higher chance of being inserted on their former best place. Additionally, to work properly

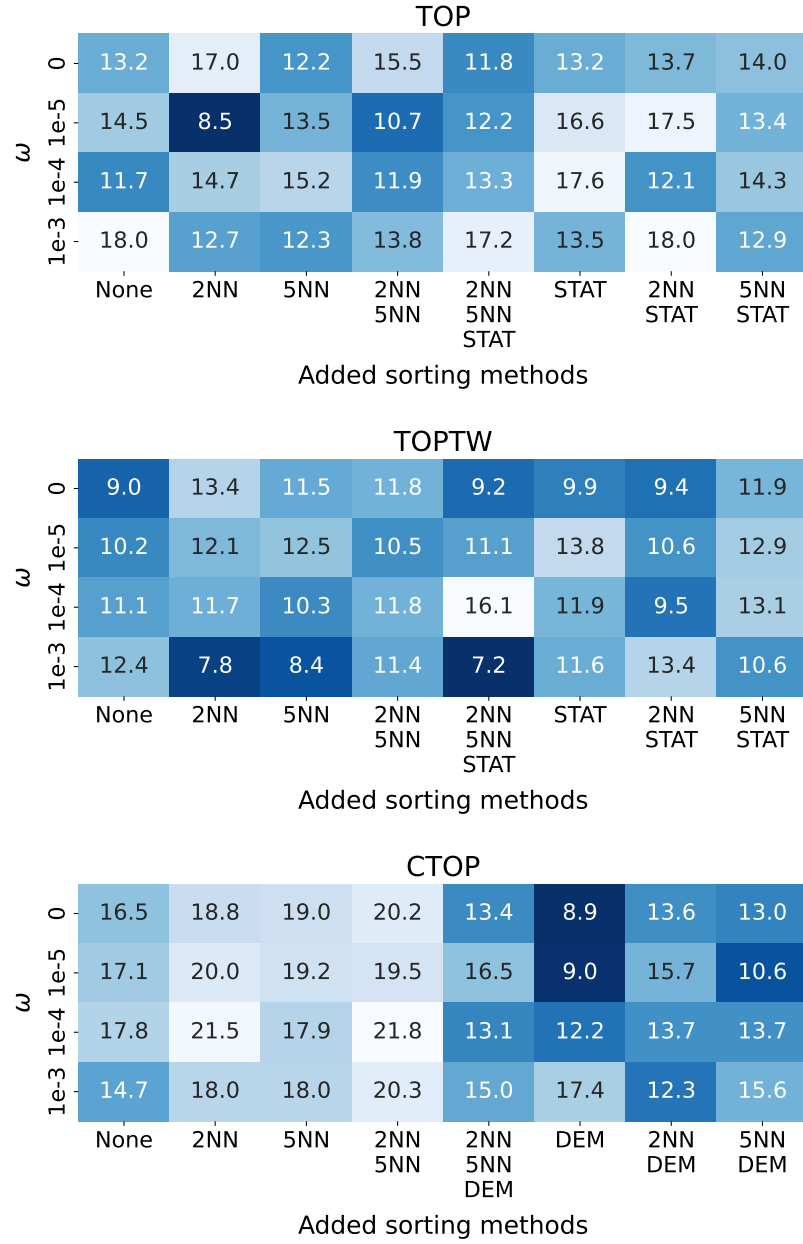


Figure 5.5: Effect of the sorting methods and distance weight on performance on tuning set.

with variants, it is important that STAT only considers the feasible insertion locations. The highest value of ω reaches the best results and no other clear pattern emerges. This dataset contains the smallest instances, which may be exhausted, providing not enough variance for effective tuning. Overall, the best setup is with $\omega = 10^{-3}$ and all three sortings active.

For CTOP, we used the SCORE-TO-DEMAND (DEM) method in place of STAT in order to avoid expanding the grid search. This decision is supported by the results from the previous variants, which indicated that STAT does not provide a significant improvement in performance. The addition of DEM sorting method clearly benefits the search. The addition of the other methods worsens the performance. This suggests that increasing the weight of DEM could be beneficial. The acceptance weight ω yielded slightly better results when set to zero, but we opted for a small positive value $\omega = 10^{-5}$. This decision was motivated by the limited size of the test set, which may limit generality, and the previous variants in which nonzero ω proved beneficial.

5.2.3 Evaluation

In this section, we refer to our approach as SISR-OP to avoid confusion with the original paper and results for VRP. We use the parameter settings discussed in the previous section, and the *maxIteration* is set again to $|N| \cdot 25000$, where $|N|$ is size of set of customer nodes. This setting has shown to be a reliable choice for producing a fair comparison with the competitive methods. Other experimental setups, such as the number of independent runs per instance, vary across benchmarks and are discussed separately for each case. The algorithm parameters are summarized in Table B.2.

Detailed results including previous works are provided in tables located in Appendix C. In these tables, BKS denotes the best-known solutions. An improvement over the BKS by our algorithm is highlighted in gray and marked with a star (*).

TOP

We followed the usual setting in the literature and performed twenty independent runs on each instance for the TOP-L benchmark set. Our

	PSOiA	PMA	HALNS	LNS2	SISR-OP
# BKS	69	76	77	54	60
# Highest means	7	7	27	14	40
Avg. gap-mean (%)	0.506	0.429	0.302	0.687	0.450
Avg. gap-best (%)	0.047	0.007	0.007	0.141	0.190
Avg. time (s)	11031	1004.1	783.4	20.6	221.3

Table 5.3: Performance comparison over TOP benchmark.

method is compared to four existing algorithms. The compared methods are Particle swarm optimization inspired algorithm (PSOiA) by Dang et al. [49], Pareto mimic algorithm (PMA) by Ke et al. [69], Hybrid adaptive large neighborhood search (HALNS) by Hammami et al. [53] (described in Section 2.4.4), and Large neighborhood search (LNS2) by Chaigneau et al. [66]. Note that PSOiA and PMA report results based on just ten runs per instance, making the mean performance more important in interpreting their results. Reported computation time is the average per run, except only for HALNS, where the lowest time required to identify the best solution is shown.

The aggregated results are presented in Table 5.3. For each algorithm, the table contains the number of best-known solutions found, the number of instances where it achieved the best mean score, the average percentage gap of the best solution to the BKS, and the average percentage gap of the mean solution to the BKS. The top result for each metric is highlighted in bold. HALNS achieves the strongest overall performance across most metrics, with PMA following closely. SISR-OP performs competitively and even surpasses existing approaches on several instances, discovering two new best-known solutions.

When considering the dataset as a whole, SISR-OP has greater performance variability than other algorithms, reflecting less overall stability. This is illustrated in Figure 5.6, which shows the 95% confidence intervals of the percentage gap to the BKS for each algorithm. The wider intervals for SISR-OP indicate that its performance varies more across the benchmark set. In contrast, SISR-OP achieves the best average score 40 times on individual instances (out of 86), more than any other method. This indicates that while SISR-OP may not always

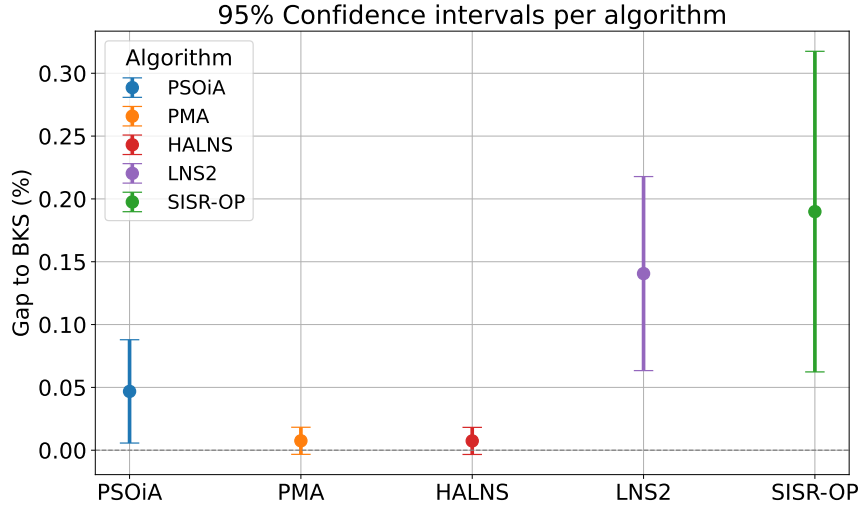


Figure 5.6: Interval plot of algorithms for TOP-L dataset.

find the absolute best solution, it provides consistently strong average results across many instances

The mean computational times are not entirely comparable because of differing experimental setups, but SISR-OP reaches competitive values. Detailed tabular results for each instance are shown in Appendix C.

TOPTW

We compare our approach with two state-of-the-art methods from the TOPTW literature: Evolution strategy approach (ES) by Karabulut et al. [70] and Multi-start iterated local search (MSILS) by Amarouche et al. [71]. These methods report best and average results based on five runs. While many more papers have benchmarked this dataset, their performance is lower and omitted here for brevity. For additional comparisons, interested reader may refer to the work of Karabulut et al.

Figure 5.7 presents an interval plot comparing the algorithms across different fleet sizes, showing the percentage gaps of the best solutions relative to the BKS. The performance of SISR-OP and MSILS is on par, while ES exhibits greater variability and significantly larger

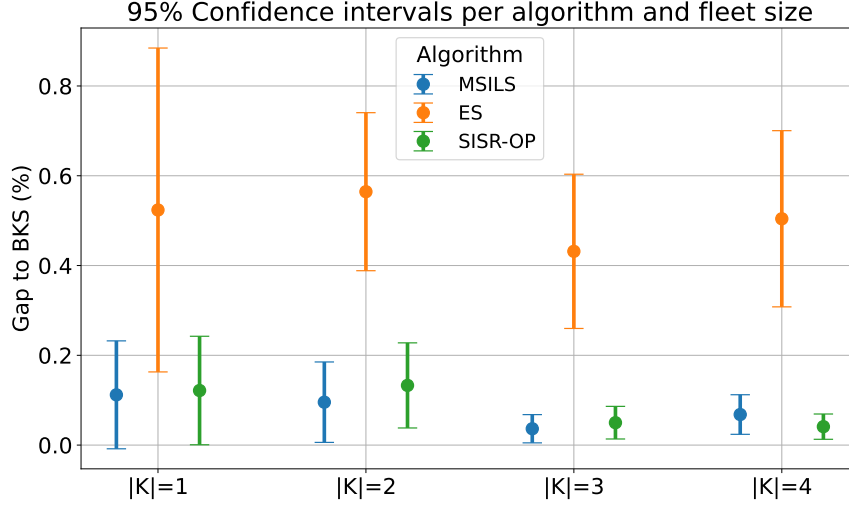


Figure 5.7: Interval plot of algorithms for TOPTW, grouped by fleet size.

gaps. Counter-intuitively, the percentage gaps and the widths of the confidence intervals decrease with increasing fleet size. This can be explained by the fact that starting from $|K| = 2$, all customers, for some instances, can be served. The algorithms are not guaranteed to find such solutions, so we left all instances included in the comparison. For $|K| = 4$, some instances degraded in such a way that SISR-OP found complete solutions in the very first iteration. Overall, SISR-OP performs at a high level compared to state-of-the-art methods on this benchmark set, with an average percentage gap to the best-known solutions of just 0.087%. Our method matched 257 out of 304 BKS and even improved 14 best-known solutions. The detailed results in the appendix are organized into four tables C.9 - C.12 according to the fleet size.

CTOP

We compare our method with three approaches developed for the CTOP benchmark set: Variable space search with tabu search (VSS-Tabu) and Variable space search with simulated annealing (VSS-SA)

	BiF&F-s	VSS-Tabu	VSS-SA	HALNS	SISR-OP
subset 2					
# BKS	17	61	59	68	40
Avg. gap (%)	0.399	0.034	0.058	0.023	0.382
subset 3					
# BKS	21	16	24	20	0
Avg. gap (%)	0.027	0.019	0.007	0.030	0.665

Table 5.4: Performance comparison on CTOP benchmark.

by Ben-Said et al. [72], and Bi-level filter-and-fan method (BiF&F-s) by Tarantilis et al. [18] and Hybrid adaptive large neighborhood search (HALNS) by Hammami [54]. They all report results based on the best of 10 runs. Following historical conventions, computation time is declared using a time to best protocol, i.e., the computational time required to reach the best solution. The other methods do not report mean solution values.

The CTOP dataset is the most challenging among studied orienteering problem benchmarks due to the larger instance and fleet sizes. SISR-OP found four new best-known solutions in the second subset. Table 5.4 shows subset-wise aggregated results. As the problem’s complexity increases, our algorithm starts to lag behind. The performance gap is especially evident in subset 3, where our algorithm matched none of the BKS values. Although the subsets use the same instance definitions and fleet sizes, in subset 3, the capacities and time limits are significantly raised. This change allows for solutions with longer routes and more customers can be served.

One notable observation is that the mean results of our approach are relatively close to the best results obtained across our runs, indicating lower variance. However, the mean values for other methods are not available, so this behavior cannot be compared directly. Figure 5.8 presents interval plots of the percentage gaps between the best results and the best-known solutions (BKS). Our method matches the performance of BiF&F-s on the second subset. As the non-overlapping confidence intervals indicate, the other methods generally perform better. Detailed tabular results are provided in Appendix C.

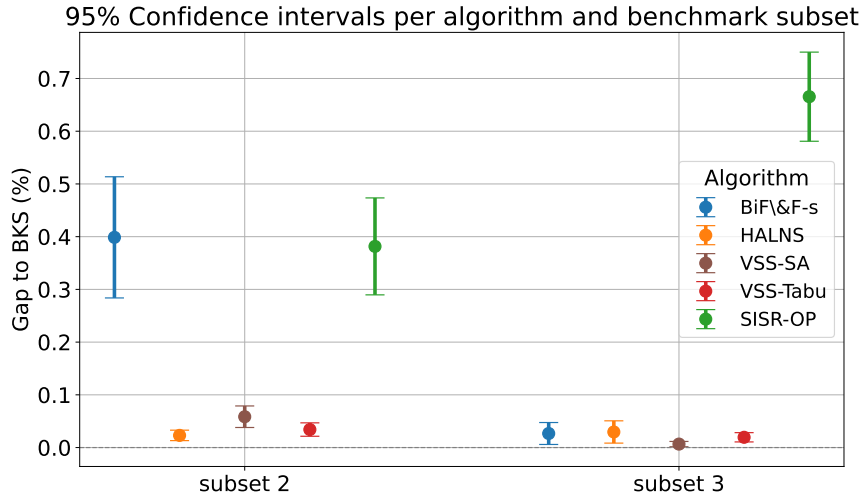


Figure 5.8: Interval plot of algorithms for CTOP, grouped by subsets.

An interesting observation is the distribution of the computational times. SISR-OP is stable without significant fluctuations, reaching a maximum average of 10 minutes per instance. On the other hand, the competing methods exceed 10000 seconds for some instances, while others are solved much faster. Matching such variability is impossible with our linear configuration of iterations without doing it individually for each instance.

To better understand what our method might be missing or to validate the data, it would help to have access to reference solutions. Unfortunately, we were not able to find any of them online. Future improvements to SISR-OP could include increasing the iteration count or tuning core parameters such as the average number of removed customers, $\bar{c}$.

6 Conclusion

In this thesis, we studied and implemented a SISR algorithm for two related but distinct families of routing problems. We reviewed exact and heuristic solution approaches found in the literature. An effective C++ solver is the main contribution of this thesis, as the available resources for these problems are scarce. A detailed description of the algorithm was presented based on the original paper. We extended the explanations to contain more detail and practical insights encountered during the implementation phase.

SISR was originally designed for VRP. We proposed and implemented its adaptation to OP, showing that the algorithm can handle a problem with an additional decision. We implemented the algorithm for three representative OP variants. Most of the approach required only minimal changes. The critical difference lies in the repair procedure, which was redesigned to handle the specifics of OP.

We evaluated the implemented methods to demonstrate their capability. For classical VRP, our implementation successfully reproduced the results reported in the original paper. We used the same parameters and experimental setup to compare performance on three representative variants, namely CVRP, VRPTW, and PDPTW. We also examined the fleet minimization procedure in more detail and identified potential improvements. A direct runtime comparison is limited due to differences in hardware and environment, but our implementation performed consistently faster. Notably, as the problem instances increased in size, we achieved higher speedups, up to 10× on the largest instances.

For OP variants, we gathered available datasets that were suitable for evaluation. They often included non-standard formats, unusable customer locations, and other important differences that have been addressed. Subsets of the data were used to fine-tune the newly introduced parameters. Our introduction of an additional stochastic component during the repair phase proved beneficial for OP. Additionally, considering the traveled distance in the acceptance criterion also positively impacted the search. After parameter tuning, our implementation consistently matched the existing state-of-the-art results

across the tested instances. We discovered 20 new best-known solutions distributed across all three studied OP variants.

Vehicle routing is an active research area, future work could include testing the implementation on new, larger benchmark datasets. With the proven capabilities of the solver, extending it to other problem variants is a natural next step. The adaptation of SISR to OP variants shows strong potential for further improvements. Deeper parameter tuning could be beneficial, especially for those not adjusted in this thesis.

Bibliography

1. CORDEAU, J-F; GENDREAU, M.; LAPORTE, G.; POTVIN, J-Y; SEMET, F. A Guide to Vehicle Routing Heuristics. *The Journal of the Operational Research Society*. 2002, vol. 53, no. 5, pp. 512–522.
2. LAPORTE, Gilbert. Fifty Years of Vehicle Routing. *Transportation Science*. 2009, vol. 43, pp. 408–416.
3. CHRISTIAENS, Jan; VANDEN BERGHE, Greet. *Slack Induction by String Removals for Vehicle Routing Problems*. Vol. 54. Institute for Operations Research and the Management Sciences, 2020. No. 2.
4. DANTZIG, G. B.; RAMSER, J. H. The Truck Dispatching Problem. *Management Science*. 1959, vol. 6, no. 1, pp. 80–91.
5. GOLDEN, Bruce; LEVY, L.; VOHRA, Rakesh. The orienteering problem. *Naval Research Logistics*. 1987, vol. 34, pp. 307–318.
6. TOTH, Paolo; VIGO, Daniele; TOTH, Paolo; VIGO, Daniele. *Vehicle Routing: Problems, Methods, and Applications, Second Edition*. USA: Society for Industrial and Applied Mathematics, 2014.
7. JOZEFOWIEZ, Nicolas; SEMET, Frédéric; TALBI, El-Ghazali. Multi-objective vehicle routing problems. *European Journal of Operational Research*. 2008, vol. 189, no. 2, pp. 293–309.
8. VIDAL, Thibaut; LAPORTE, Gilbert; MATL, Piotr. A concise guide to existing and emerging vehicle routing problem variants. *European Journal of Operational Research*. 2020, vol. 286, no. 2, pp. 401–416.
9. BRAEKERS, Kris; RAMAEKERS, Katrien; VAN NIEUWENHUYSE, Inneke. The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*. 2016, vol. 99, pp. 300–313.
10. NAGATA, Yuichi; BRÄYSY, Olli; DULLAERT, Wout. A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows. *Computers & Operations Research*. 2010, vol. 37, no. 4, pp. 724–737.

11. CURTOIS, Timothy; LANDA-SILVA, Dario; QU, Yi; LAESANKLANG, Wasakorn. Large neighbourhood search with adaptive guided ejection search for the pickup and delivery problem with time windows. *EURO Journal on Transportation and Logistics*. 2018, vol. 7, no. 2, pp. 151–192.
12. CORBERÁN, Ángel; EGGLESE, Richard; HASLE, Geir; PLANA, Isaac; SANCHIS, José María. Arc routing problems: A review of the past, present, and future. *Networks*. 2021, vol. 77, no. 1, pp. 88–115.
13. FEILLET, Dominique; DEJAX, Pierre; GENDREAU, Michel. Traveling Salesman Problems with Profits. *Transportation Science*. 2005, vol. 39, no. 2, pp. 188–205.
14. GUNAWAN, Aldy; LAU, Hoong Chuin; VANSTEENWEGEN, Pieter. Orienteering Problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research*. 2016, vol. 255, no. 2, pp. 315–332.
15. BOUSSIER, Sylvain; FEILLET, Dominique; GENDREAU, Michel. An exact algorithm for team orienteering problems. *4OR quarterly journal of the Belgian, French and Italian Operations Research Societies*. 2007, vol. 5, pp. 211–230.
16. VANSTEENWEGEN, Pieter; SOUFFRIAU, Wouter; VANDEN BERGHE, Greet; VAN OUDHEUSDEN, Dirk. Iterated local search for the team orienteering problem with time windows. *Computers & Operations Research*. 2009, vol. 36, no. 12, pp. 3281–3290.
17. TANG, Hao; MILLER-HOOKS, Elise. A TABU search heuristic for the team orienteering problem. *Computers & Operations Research*. 2005, vol. 32, no. 6, pp. 1379–1407.
18. TARANTILIS, C.D.; STAVROPOULOU, F.; REPOUSSIS, P.P. The Capacitated Team Orienteering Problem: A Bi-level Filter-and-Fan method. *European Journal of Operational Research*. 2013, vol. 224, no. 1, pp. 65–78.
19. COSTA, Luciano; CONTARDO, Claudio; DESAULNIERS, Guy. Exact Branch-Price-and-Cut Algorithms for Vehicle Routing. *Transportation Science*. 2019, vol. 53, no. 4, pp. 946–985.

20. PESSOA, A.; SADYKOV, R.; UCHOA, E., et al. A generic exact solver for vehicle routing and related problems. *Mathematical Programming*. 2020, vol. 183, pp. 483–523.
21. YOU, Zhengzhong; YANG, Yu; WANG, Xinshang; YIN, Wotao. Two-Stage Learning to Branch in Branch-Price-and-Cut Algorithms for Solving Vehicle Routing Problems Exactly [Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4630549]. 2023. To be published, visited on 2025-05-13.
22. LENSTRA, J. K.; KAN, A. H. G. Rinnooy. Complexity of vehicle routing and scheduling problems. *Networks*. 1981, vol. 11, no. 2, pp. 221–227.
23. LAPORTE, Gilbert; MARTELLO, Silvano. The selective travelling salesman problem. *Discrete Applied Mathematics*. 1990, vol. 26, no. 2, pp. 193–207.
24. MORTEZA KESHTKARAN Koorush Ziarati, Andrea Bettinelli; VIGO, Daniele. Enhanced exact solution methods for the Team Orienteering Problem. *International Journal of Production Research*. 2016, vol. 54, no. 2, pp. 591–601.
25. HASLE, Geir; KLOSTER, Oddvar. Industrial Vehicle Routing. *Geometric Modelling, Numerical Simulation, and Optimization: Applied Mathematics at SINTEF*. 2007, pp. 397–435.
26. SAVELSBERGH, M. W. P. Local search in routing problems with time windows. *Annals of Operations Research*. 1985, vol. 4, no. 1, pp. 285–305.
27. BALDACCI, Roberto; BARTOLINI, Enrico; MINGOZZI, Aristide. An Exact Algorithm for the Pickup and Delivery Problem with Time Windows. *Operations Research*. 2011, vol. 59, no. 2, pp. 414–426.
28. CLARKE, G.; WRIGHT, J. W. Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*. 1964, vol. 12, no. 4, pp. 568–581.

29. REYES-RUBIANO, L.; JUAN, A.A.; BAYLISS, C.; PANADERO, J.; FAULIN, J.; COPADO, P. A Biased-Randomized Learnheuristic for Solving the Team Orienteering Problem with Dynamic Rewards. *Transportation Research Procedia*. 2020, vol. 47, pp. 680–687.
30. VIDAL, Thibaut; CRAINIC, Teodor Gabriel; GENDREAU, Michel; PRINS, Christian. Heuristics for multi-attribute vehicle routing problems: A survey and synthesis. *European Journal of Operational Research*. 2013, vol. 231, no. 1, pp. 1–21.
31. VAN BREEDAM, Alex. Improvement heuristics for the Vehicle Routing Problem based on simulated annealing. *European Journal of Operational Research*. 1995, vol. 86, no. 3, pp. 480–490.
32. LOURENÇO, Helena R.; MARTIN, Olivier C.; STÜTZLE, Thomas. Iterated Local Search: Framework and Applications. In: *Handbook of Metaheuristics*. Ed. by GENDREAU, Michel; POTVIN, Jean-Yves. Boston, MA: Springer US, 2010, pp. 363–397.
33. GUNAWAN, Aldy; LAU, Hoong; VANSTEENWEGEN, Pieter; LU, Kun. Well-tuned algorithms for the Team Orienteering Problem with Time Windows. *Journal of the Operational Research Society*. 2017, vol. 68.
34. KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by Simulated Annealing. *Science*. 1983, vol. 220, no. 4598, pp. 671–680.
35. LIN, Shih-Wei. Solving the team orienteering problem using effective multi-start simulated annealing. *Applied Soft Computing*. 2013, vol. 13, no. 2, pp. 1064–1073.
36. LIN, Shih-Wei; YU, Vincent F. A simulated annealing heuristic for the team orienteering problem with time windows. *European Journal of Operational Research*. 2012, vol. 217, no. 1, pp. 94–107.
37. CORDEAU, J-F; LAPORTE, G.; MERCIER, A. A Unified Tabu Search Heuristic for Vehicle Routing Problems with Time Windows. *The Journal of the Operational Research Society*. 2001, vol. 52, no. 8, pp. 928–936.

38. LIN, Shen. Computer Solutions of the Traveling Salesman Problem. *Bell System Technical Journal*. 1965, vol. 44, no. 10, pp. 2245–2269.
39. SHAW, Paul. A new local search algorithm providing high quality solutions to vehicle routing problems. *APES Group, Dept of Computer Science, University of Strathclyde, Glasgow, Scotland, UK*. 1997, vol. 46.
40. SCHRIMPF, Gerhard; SCHNEIDER, Johannes; STAMM-WILBRANDT, Hermann; DUECK, Gunter. Record Breaking Optimization Results Using the Ruin and Recreate Principle. *Journal of Computational Physics*. 2000, vol. 159, no. 2, pp. 139–171.
41. SCHMID, Verena; EHMKE, Jan Fabian. An Effective Large Neighborhood Search for the Team Orienteering Problem with Time Windows. In: *Computational Logistics*. Cham: Springer International Publishing, 2017, pp. 3–18.
42. ROPKE, Stefan; PISINGER, David. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*. 2006, vol. 40, no. 4, pp. 455–472.
43. PISINGER, David; ROPKE, Stefan. A general heuristic for vehicle routing problems. *Computers & Operations Research*. 2007, vol. 34, no. 8, pp. 2403–2435.
44. PULJIĆ, Krunoslav; MANGER, Robert. Comparison of eight evolutionary crossover operators for the vehicle routing problem. *Mathematical Communications*. 2013, vol. 18, pp. 359–375.
45. LAU, H. C. W.; CHAN, T. M.; TSUI, W. T.; PANG, W. K. Application of Genetic Algorithms to Solve the Multidepot Vehicle Routing Problem. *IEEE Transactions on Automation Science and Engineering*. 2010, vol. 7, no. 2, pp. 383–392.
46. NALEPA, Jakub; BLOCHO, Mirosław. Adaptive memetic algorithm for minimizing distance in the vehicle routing problem with time windows. *Soft Computing*. 2015, vol. 20, pp. 2309–2327.
47. PATRICK, Siarry. *Metaheuristics*. Springer, 2016.

48. VERBEECK, C.; SÖRENSEN, K.; AGHEZZAF, E.-H.; VANSTEEN-
WEGEN, P. A fast solution method for the time-dependent ori-
enteering problem. *European Journal of Operational Research*. 2014,
vol. 236, no. 2, pp. 419–432.
49. DANG, Duc-Cuong; GUIBADJ, Rym Nesrine; MOUKRIM, Aziz.
An effective PSO-inspired algorithm for the team orienteering
problem. *European Journal of Operational Research*. 2013, vol. 229,
no. 2, pp. 332–344.
50. VIDAL, Thibaut; CRAINIC, Teodor Gabriel; GENDREAU, Michel;
LAHRICHI, Nadia; REI, Walter. A Hybrid Genetic Algorithm for
Multidepot and Periodic Vehicle Routing Problems. *Operations
Research*. 2012, vol. 60, pp. 611–624.
51. VIDAL, Thibaut; CRAINIC, Teodor Gabriel; GENDREAU, Michel;
PRINS, Christian. A hybrid genetic algorithm with adaptive di-
versity management for a large class of vehicle routing problems
with time-windows. *Computers & Operations Research*. 2013, vol. 40,
no. 1, pp. 475–489.
52. VIDAL, Thibaut; CRAINIC, Teodor Gabriel; GENDREAU, Michel;
PRINS, Christian. A unified solution framework for multi-attribute
vehicle routing problems. *European Journal of Operational Research*.
2014, vol. 234, no. 3, pp. 658–673.
53. HAMMAMI, Farouk; REKIK, Monia; COELHO, Leandro C. A
hybrid adaptive large neighborhood search heuristic for the team
orienteering problem. *Computers & Operations Research*. 2020,
vol. 123, p. 105034.
54. HAMMAMI, Farouk. An efficient hybrid adaptive large neigh-
borhood search method for the capacitated team orienteering
problem. *Expert Systems with Applications*. 2024, vol. 249, p. 123561.
55. WOUDA, Niels A.; LAN, Leon; KOOL, Wouter. PyVRP: A High-
Performance VRP Solver Package. *INFORMS Journal on Comput-
ing*. 2024, vol. 36, no. 4, pp. 943–955.
56. BRESINA, John L. Heuristic-biased stochastic sampling. In: *Pro-
ceedings of the Thirteenth National Conference on Artificial Intelligence
- Volume 1*. Portland, Oregon: AAAI Press, 1996, pp. 271–278.
AAAI'96.

57. SAVELSBERGH, Martin. The Vehicle Routing Problem with Time Windows: Minimizing Route Duration. *INFORMS Journal on Computing*. 1992, vol. 4, pp. 146–154.
58. VIDAL, Thibaut; MACULAN, Nelson; OCHI, Luiz Satoru; PENNA, Puca Huachi Vaz. Large Neighborhoods with Implicit Customer Selection for Vehicle Routing Problems with Profits. *Transportation Science*. 2016, vol. 50, no. 2, pp. 720–734.
59. SHAW, Paul. Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems. In: MAHER, Michael; PUGET, Jean-Francois (eds.). *Principles and Practice of Constraint Programming — CP98*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 417–431.
60. GUNAWAN, Aldy; ZHU, Jiahui; NG, Kien Ming. The Capacitated Team Orienteering Problem: a hybrid Simulated Annealing and Iterated Local Search Approach. In: *Proceedings of the 13th International Conference on the Practice and Theory of Automated Timetabling - PATAT 2021: Volume II*. PATAT, 2021, pp. 299–303.
61. METACENTRUM. *MetaCentrum* [online] [Available from: <https://metavo.metacentrum.cz/en/index.html>]. 2025. CESNET, visited on 2025-05-11.
62. UCHOA, Eduardo; PECIN, Diego; PESSOA, Artur; POGGI, Marcus; VIDAL, Thibaut; SUBRAMANIAN, Anand. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*. 2017, vol. 257, no. 3, pp. 845–858.
63. GEHRING, Hermann; HOMBERGER, Jörg. Parallelization of a Two-Phase Metaheuristic for Routing Problems with Time Windows. *J. Heuristics*. 2002, vol. 8, pp. 251–276.
64. LI, H.; LIM, A. A metaheuristic for the pickup and delivery problem with time windows. In: *Proceedings 13th IEEE International Conference on Tools with Artificial Intelligence. ICTAI 2001*. 2001, pp. 160–167.
65. CHAO, I-Ming; GOLDEN, Bruce L.; WASIL, Edward A. The team orienteering problem. *European Journal of Operational Research*. 1996, vol. 88, no. 3, pp. 464–474.

66. CHAIGNEAU, Charly; BOSTEL, Nathalie; GRIMAULT, Axel. A Large Neighborhood Search-based approach to tackle the very large scale Team Orienteering Problem in industrial context. *Computers & Operations Research*. 2025, vol. 176, p. 106954.
67. RIGHINI, Giovanni; SALANI, Matteo. Decremental state space relaxation strategies and initialization heuristics for solving the Orienteering Problem with Time Windows with dynamic programming. *Computers & Operations Research*. 2009, vol. 36, no. 4, pp. 1191–1203.
68. MONTEMANNI, Roberto; GAMBARDELLA, Luca Maria. Ant colony system for team orienteering problems with time windows. *Foundations of Computing and Decision Sciences*. 2009, vol. 34, pp. 287–306.
69. KE, Liangjun; ZHAI, Laipeng; LI, Jing; CHAN, Felix T.S. Pareto mimic algorithm: An approach to the team orienteering problem. *Omega*. 2016, vol. 61, pp. 155–166.
70. KARABULUT, Korhan; TASGETIREN, M. Fatih. An evolution strategy approach to the team orienteering problem with time windows. *Computers & Industrial Engineering*. 2020, vol. 139, 106–109.
71. AMAROUCHE, Youcef; GUIBADJ, Rym Nesrine; CHAALAL, Elhadja; MOUKRIM, Aziz. Effective neighborhood search with optimal splitting and adaptive memory for the team orienteering problem with time windows. *Computers & Operations Research*. 2020, vol. 123, p. 105039.
72. BEN-SAID, Asma; EL-HAJJ, Racha; MOUKRIM, Aziz. A variable space search heuristic for the Capacitated Team Orienteering Problem. *Journal of Heuristics*. 2019, vol. 25, no. 2, pp. 273–303.
73. ERDELIĆ, Tomislav; CARIĆ, Tonči. A Survey on the Electric Vehicle Routing Problem: Variants and Solution Approaches. *Journal of Advanced Transportation*. 2019, vol. 2019, no. 1, p. 5075671.

A Archive

Files submitted into the master thesis archive in the Information System of Masaryk University:

- *thesis.pdf*: Text of the thesis in PDF format.
- *thesis.zip*: Source codes of the implemented solvers.
- *data.zip*: Datasets used for evaluation.

B SISR parameters

This appendix contains Tables B.1 and B.2 with overview of parameters used for experimental evaluation. The default values for CVRP come from the SISR [3] paper.

<i>maxIteration</i>	The number of iterations is static and configured before execution. For example, for a CVRP benchmark with 100 customers, $3 * 10^7$ iterations were performed.
<i>maxIterationFleet</i>	When solving variants with the hierarchical objective, iterations performed in the first phase use 10% of <i>maxIteration</i> , leaving remaining 90% for the distance minimization.
$T_0 = 100, T_{\text{final}} = 1$	The static value of T_0 is set in a reasonable range, which has worked well on the studied benchmarks. This approach is likely chosen due to a naïve initial solution generation in SISR. In literature, it is more common to set the initial temperature based on the initial solution, as some problem instances might vary in properties [42, 73].
$\bar{c} = 10$	As mentioned earlier, $\bar{c}$ represents roughly the average number of removed customers. Lower values were found insufficient to introduce enough slack, while values above 15 resulted in highly divergent neighbors that are hard to repair into improving solutions.
$l_{\text{max}} = 10$	Routes in VRP solutions are usually longer, Equation 3.5 is evaluated to l_{max} , except during the first few iterations of <code>DISTANCEMINIMIZATION(s)</code> . This parameter affects not only the length of the removed strings but also impacts the number of removed strings.
$\alpha = 0.5$	The <code>STRING</code> and <code>SPLITSTRING</code> operators are executed with equal probability.
$\beta = 0.01$	With this value, the removed strings of customers in <code>SPLITSTRING</code> operator are typically far from each other but close to the depot on both ends of the route.
$\gamma = 0.01$	The best insertion position is blinked over with a probability of 1%. This value is a compromise, as smaller instances inclined toward 0% in experiments, while 5% was more suitable for larger instances.

Table B.1: Summary of used parameters for CVRP.

B. SISR PARAMETERS

$maxIteration = 25000 \cdot N $, where $ N $ is number of customer nodes.
$T_0 = 50, T_{final} = 1$
$\bar{c} = 10$
$l_{max} = 10$
$\alpha = 0.5$
$\beta = 0.01$
$\gamma = 0.01$
$\omega = 10^{-5}$ for TOP and CTOP
$\omega = 10^{-3}$ for TOPTW
$p = 0.15$ for TOP and TOPTW
$p = 0.10$ for CTOP

Table B.2: Summary of used parameters for orienteering problems.

C Detailed tabular results

This appendix provides detailed tabular results from the experimental evaluation presented in Chapter 5. Each table compares a specific problem variant, with results aggregated on a per-instance level. The number of runs varies depending on the experimental setup, both the best and average values are reported.

- Table C.1: CVRP; original SISR vs our implementation.
- Table C.2: VRPTW; size 200 and 400
- Table C.3: VRPTW; size 600 and 800
- Table C.4: VRPTW; size 1000
- Table C.5: PDPTW; size 200 and 400
- Table C.6: PDPTW; size 600 and 800
- Table C.7: PDPTW; size 1000
- Table C.8: TOP-L
- Table C.9: TOPTW; 1 vehicle
- Table C.10: TOPTW; 2 vehicles
- Table C.11: TOPTW; 3 vehicles
- Table C.12: TOPTW; 4 vehicles
- Table C.13: CTOP; subset 2
- Table C.14: CTOP; subset 3

C. DETAILED TABULAR RESULTS

Instance	Avg	original Best	Time(min)	Avg	our Best	Time(min)	Gap to Avg (%)
X-n101-k25	27591	27591	0.8	27591	27591	1.1	0
X-n106-k14	26381.5	26362	1.3	26383.4	26371	0.9	0.007
X-n110-k13	14971.1	14971	1	14971.3	14971	0.9	0.001
X-n115-k10	12747	12747	0.2	12747	12747	1.3	0
X-n120-k6	13332	13332	1.6	13332	13332	1.3	0
X-n125-k30	55556.3	55542	3.1	55563	55539	1.4	0.012
X-n129-k18	28948.8	28940	1.5	28948.4	28940	1.2	-0.001
X-n134-k13	10940.1	10916	2.8	10942.9	10916	1.2	0.025
X-n139-k10	13595.4	13590	2	13595.1	13590	1.3	-0.002
X-n143-k7	15705.8	15700	2.1	15709.8	15700	1.6	0.025
X-n148-k46	43469.2	43448	2.8	43476.3	43448	1.8	0.016
X-n153-k22	21229.4	21220	5.6	21232.6	21220	2	0.015
X-n157-k13	16878.6	16876	3.7	16876.8	16876	1.3	-0.011
X-n162-k11	14157.1	14138	3.4	14155.2	14138	1.8	-0.013
X-n167-k10	20560.8	20557	3.2	20557.7	20557	1.7	-0.015
X-n172-k51	45619.2	45607	5.3	45637.8	45607	2.4	0.041
X-n176-k26	47849.6	47812	5.2	47853.5	47812	2	0.008
X-n181-k23	25579.8	25569	5.5	25581.7	25569	1.8	0.008
X-n186-k15	24178.4	24149	4	24176.1	24147	1.8	-0.01
X-n190-k8	16984.9	16980	9.1	16984.5	16980	2.3	-0.002
X-n195-k51	44298.5	44241	6.1	44281.9	44225	2.7	-0.037
X-n200-k36	58636.1	58578	6.7	58641.1	58599	2.2	0.008
X-n204-k19	19662.3	19565	4.9	19641.4	19565	2.1	-0.106
X-n209-k16	30669.4	30656	6	30662.1	30656	2.2	-0.024
X-n214-k11	10908.6	10873	8.7	10902.6	10870	2.5	-0.055
X-n219-k73	117650.4	117595	8	117641.5	117595	2.9	-0.008
X-n223-k34	40529.9	40448	7.6	40522.3	40448	2.6	-0.019
X-n228-k23	25790.9	25744	10.5	25791.8	25743	2.9	0.003
X-n233-k16	19269.7	19232	8.1	19268.2	19230	3.1	-0.008
X-n237-k14	27089.7	27042	7.2	27094.3	27042	2.5	0.017
X-n242-k48	82884.4	82775	9.9	82879.7	82754	3	-0.006
X-n247-k47	37323.2	37274	18.4	37372.1	37274	5.2	0.131
X-n251-k28	38791	38687	9.8	38793.1	38687	2.7	0.005
X-n256-k16	18888.9	18880	11.5	18888.3	18880	3.8	-0.003
X-n261-k13	26642.3	26558	11.8	26631.6	26559	3.4	-0.04
X-n266-k58	75617.8	75478	10.8	75617.5	75478	3.4	0
X-n270-k35	35362.2	35323	11.4	35363.8	35323	3.1	0.005
X-n275-k28	21268.6	21245	13.3	21277.3	21245	3.1	0.041
X-n280-k17	33628.1	33529	17.7	33631	33506	3.8	0.009
X-n284-k15	20286.6	20240	15.3	20285.3	20245	3.5	-0.007
X-n289-k60	95352.2	95233	14.3	95373.9	95192	4.2	0.023
X-n294-k50	47274.5	47210	14.7	47278.4	47197	4	0.008
X-n298-k31	34276	34234	14.5	34270.8	34234	3.5	-0.015
X-n303-k21	21776.5	21751	17.3	21776.2	21748	4.1	-0.001
X-n308-k13	26207.7	25931	25.7	26205.3	25940	5.3	-0.009
X-n313-k71	94182.4	94063	18.9	94172.8	94055	5.1	-0.01
X-n317-k53	78392.4	78355	22	78383	78355	4.2	-0.012
X-n322-k28	29927.6	29849	16.9	29922.5	29859	3.7	-0.017
X-n327-k20	27631.4	27608	21.6	27635.1	27609	4.7	0.013
X-n331-k15	31128.2	31122	20.4	31127.2	31108	4.2	-0.003
X-n336-k84	139373.4	139209	22.8	139322.7	139200	6.2	-0.036
X-n344-k43	42158.5	42079	21.5	42148.6	42064	4.3	-0.023
X-n351-k40	25982.1	25938	26.5	25979.5	25934	5.2	-0.01
X-n359-k29	51577.8	51505	23.1	51576	51507	4.5	-0.004

C. DETAILED TABULAR RESULTS

(Continued from previous page)

Instance	Avg	original Best	Time(min)	Avg	our Best	Time(min)	Gap to Avg (%)
X-n367-k17	22833.4	22814	36.1	22832.6	22814	6.6	-0.003
X-n376-k94	147783.6	147721	32	147774	147736	6.1	-0.006
X-n384-k52	66107.4	65963	25.9	66091.6	65958	5.1	-0.024
X-n393-k38	38394.1	38331	30.4	38393.1	38345	5.1	-0.003
X-n401-k29	66248.5	66189	38	66243.8	66185	6.9	-0.007
X-n411-k19	19768.5	19731	58.4	19773.4	19732	9.4	0.025
X-n420-k130	107879.2	107817	47.9	107893.3	107798	9.6	0.013
X-n429-k61	65593.6	65485	35	65604.8	65504	6.1	0.017
X-n439-k37	36473.8	36426	42.1	36471.1	36413	6.9	-0.008
X-n449-k29	55411.2	55272	38	55415.9	55271	6.4	0.008
X-n459-k26	24242.2	24175	56.5	24240.6	24164	8.4	-0.007
X-n469-k138	222227.1	221984	48	222230.5	221964	9.9	0.002
X-n480-k70	89559.2	89458	50.5	89554.7	89458	7.6	-0.005
X-n491-k59	66645.5	66517	51.4	66643.9	66508	8.5	-0.002
X-n502-k39	69274.7	69243	60.9	69275.1	69232	8.3	0.001
X-n513-k21	24292.1	24238	77.1	24291.7	24240	12.9	-0.002
X-n524-k137	154807.2	154651	151.4	154883.3	154610	28.4	0.049
X-n536-k96	95173.2	95006	74.7	95142.7	94966	11.3	-0.032
X-n548-k50	86798	86710	64.5	86798.3	86710	9.4	0
X-n561-k42	42868.1	42774	73.8	42865.3	42786	11.3	-0.007
X-n573-k30	50804.6	50737	113	50809.3	50736	14.8	0.009
X-n586-k159	190600.7	190484	86.3	190632.7	190420	14.5	0.017
X-n599-k92	108688.6	108548	75.4	108693	108579	11.5	0.004
X-n613-k62	59731.3	59585	88.1	59737.9	59632	12.8	0.011
X-n627-k43	62317.1	62219	89.3	62316.3	62208	11.1	-0.001
X-n641-k35	63850.3	63750	92.5	63858.7	63761	11.9	0.013
X-n655-k131	106844.6	106813	109.6	106853.3	106796	15.7	0.008
X-n670-k126	146720.4	146451	198.9	147019.9	146611	29.6	0.204
X-n685-k75	68369	68271	135.1	68402.1	68304	17.9	0.048
X-n701-k44	82065.4	81974	122.5	82074.5	81964	14.2	0.011
X-n716-k35	43483.8	43426	158.3	43488.7	43426	17.3	0.011
X-n733-k159	136389.3	136255	143.2	136393.3	136291	20.8	0.003
X-n749-k98	77509.2	77380	146.3	77529.1	77422	19.1	0.026
X-n766-k71	114761.1	114590	174.4	114762.1	114551	20.8	0.001
X-n783-k48	72660.7	72492	170.2	72644.2	72490	20.1	-0.023
X-n801-k40	73436.7	73347	137.1	73442	73347	15.6	0.007
X-n819-k171	158423	158305	172.5	158426.6	158303	22.2	0.002
X-n837-k142	193976.9	193824	166.8	193992.7	193817	20.6	0.008
X-n856-k95	89131.3	89050	160	89125.8	89043	20.1	-0.006
X-n876-k59	99483.2	99388	217.4	99501.5	99384	22.9	0.018
X-n895-k37	54085.8	53993	212.5	54082.2	53890	22.7	-0.007
X-n916-k207	329509.5	329299	215.3	329493.5	329380	27.2	-0.005
X-n936-k151	133117.3	133014	412.7	133282.4	133059	49.4	0.124
X-n957-k87	85620	85546	202.4	85607.8	85515	24.2	-0.014
X-n979-k58	119120.4	119065	276.6	119131.1	119076	26.9	0.009
X-n1001-k43	72528.1	72415	284.3	72542.5	72418	26.6	0.02

Table C.1: Comparison on CVRP instances of original paper and our implementation.

C. DETAILED TABULAR RESULTS

200 requests					400 requests				
Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time	Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time
C1_2_1	20.00	20	2704.57	2.23	C1_4_1	40.00	40	7152.06	7.55
C1_2_10	18.00	18	2643.55	1.60	C1_4_10	36.00	36	6861.17	3.36
C1_2_2	18.00	18	2917.89	1.42	C1_4_2	36.00	36	7698.56	3.06
C1_2_3	18.00	18	2707.35	1.61	C1_4_3	36.00	36	7069.87	5.00
C1_2_4	18.00	18	2643.61	1.72	C1_4_4	36.00	36	6805.84	3.56
C1_2_5	20.00	20	2702.05	2.23	C1_4_5	40.00	40	7152.06	7.79
C1_2_6	20.00	20	2701.04	2.32	C1_4_6	40.00	40	7153.45	8.00
C1_2_7	20.00	20	2701.04	2.23	C1_4_7	39.00	39	7421.56	7.01
C1_2_8	19.00	19	2775.48	1.99	C1_4_8	37.00	37	7347.23	4.94
C1_2_9	18.00	18	2687.83	1.49	C1_4_9	36.00	36	7054.44	3.17
C2_2_1	6.00	6	1931.44	2.40	C2_4_1	12.00	12	4116.14	10.13
C2_2_10	6.00	6	1806.58	3.29	C2_4_10	11.95	11	3891.89	11.23
C2_2_2	6.00	6	1863.16	3.01	C2_4_2	12.00	12	3930.05	13.30
C2_2_3	6.00	6	1775.08	3.70	C2_4_3	12.00	12	3772.46	15.21
C2_2_4	6.00	6	1703.43	3.81	C2_4_4	12.00	12	3541.23	17.12
C2_2_5	6.00	6	1878.85	2.77	C2_4_5	12.00	12	3938.69	10.90
C2_2_6	6.00	6	1857.35	3.03	C2_4_6	12.00	12	3875.94	11.39
C2_2_7	6.00	6	1849.46	3.03	C2_4_7	12.00	12	3894.45	11.30
C2_2_8	6.00	6	1820.53	3.06	C2_4_8	12.00	12	3795.88	10.79
C2_2_9	6.00	6	1830.05	3.19	C2_4_9	12.00	12	3867.27	11.67
R1_2_1	20.00	20	4784.11	2.17	R1_4_1	40.00	40	10386.15	7.56
R1_2_10	18.00	18	3301.18	1.57	R1_4_10	36.00	36	8120.95	3.53
R1_2_2	18.00	18	4045.19	1.44	R1_4_2	36.00	36	8949.64	3.40
R1_2_3	18.00	18	3382.11	1.61	R1_4_3	36.00	36	7823.42	3.73
R1_2_4	18.00	18	3065.60	1.75	R1_4_4	36.00	36	7290.53	3.92
R1_2_5	18.00	18	4108.31	1.31	R1_4_5	36.00	36	9242.97	3.11
R1_2_6	18.00	18	3583.14	1.52	R1_4_6	36.00	36	8405.57	3.53
R1_2_7	18.00	18	3150.11	1.66	R1_4_7	36.00	36	7628.51	3.82
R1_2_8	18.00	18	2959.71	1.75	R1_4_8	36.00	36	7273.76	3.95
R1_2_9	18.00	18	3762.57	1.41	R1_4_9	36.00	36	8748.94	3.28
R2_2_1	4.00	4	4483.16	2.52	R2_4_1	8.00	8	9288.20	9.17
R2_2_10	4.00	4	2666.10	3.46	R2_4_10	8.00	8	5813.08	11.19
R2_2_2	4.00	4	3623.04	3.14	R2_4_2	8.00	8	7610.90	12.16
R2_2_3	4.00	4	2880.75	3.96	R2_4_3	8.00	8	5925.51	14.32
R2_2_4	4.00	4	1981.30	4.37	R2_4_4	8.00	8	4278.59	15.78
R2_2_5	4.00	4	3366.79	2.92	R2_4_5	8.00	8	7151.46	9.83
R2_2_6	4.00	4	2913.81	3.58	R2_4_6	8.00	8	6130.80	12.50
R2_2_7	4.00	4	2451.14	4.21	R2_4_7	8.00	8	5056.12	14.02
R2_2_8	4.00	4	1849.87	4.58	R2_4_8	8.00	8	4017.45	15.90
R2_2_9	4.00	4	3100.24	3.28	R2_4_9	8.00	8	6415.00	10.57
RC1_2_1	18.00	18	3619.60	1.19	RC1_4_1	36.00	36	8606.21	2.98
RC1_2_10	18.00	18	3000.30	1.44	RC1_4_10	36.00	36	7600.70	3.50
RC1_2_2	18.00	18	3249.59	1.32	RC1_4_2	36.00	36	7912.16	3.30
RC1_2_3	18.00	18	3008.76	1.43	RC1_4_3	36.00	36	7541.81	3.50
RC1_2_4	18.00	18	2852.62	1.49	RC1_4_4	36.00	36	7318.04	3.60
RC1_2_5	18.00	18	3379.98	1.29	RC1_4_5	36.00	36	8201.98	3.14
RC1_2_6	18.00	18	3324.80	1.28	RC1_4_6	36.00	36	8205.35	3.14
RC1_2_7	18.00	18	3192.95	1.35	RC1_4_7	36.00	36	7961.18	3.29
RC1_2_8	18.00	18	3087.72	1.41	RC1_4_8	36.00	36	7791.97	3.41
RC1_2_9	18.00	18	3082.70	1.40	RC1_4_9	36.00	36	7759.54	3.38
RC2_2_1	6.00	6	3099.76	2.84	RC2_4_1	11.00	11	6703.98	10.60
RC2_2_10	4.00	4	2015.61	3.57	RC2_4_10	8.00	8	4293.14	12.68
RC2_2_2	5.00	5	2825.24	3.52	RC2_4_2	9.00	9	6219.40	12.44
RC2_2_3	4.00	4	2602.11	3.55	RC2_4_3	8.00	8	4935.47	13.48
RC2_2_4	4.00	4	2039.27	3.94	RC2_4_4	8.00	8	3651.08	15.02
RC2_2_5	4.00	4	2911.46	2.81	RC2_4_5	8.18	8	6772.46	9.71
RC2_2_6	4.00	4	2876.02	2.70	RC2_4_6	8.00	8	5854.96	9.91
RC2_2_7	4.00	4	2528.63	3.09	RC2_4_7	8.00	8	5400.15	10.54
RC2_2_8	4.00	4	2296.16	3.43	RC2_4_8	8.00	8	4823.90	11.42
RC2_2_9	4.00	4	2180.44	3.39	RC2_4_9	8.00	8	4576.75	11.53

Table C.2: Detailed results on VRPTW dataset with 200 and 400 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

C. DETAILED TABULAR RESULTS

600 requests					800 requests				
Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time	Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time
C1_6_1	60.00	60	14095.64	14.34	C1_8_1	80.00	80	25184.38	29.72
C1_6_10	56.00	56	13667.89	8.18	C1_8_10	72.00	72	24101.21	8.73
C1_6_2	56.00	56	14163.31	7.42	C1_8_2	72.29	72	26715.16	9.26
C1_6_3	56.00	56	13779.38	16.86	C1_8_3	72.00	72	24261.80	32.33
C1_6_4	56.00	56	13598.39	15.78	C1_8_4	72.00	72	23851.94	33.70
C1_6_5	60.00	60	14085.72	14.60	C1_8_5	80.00	80	25166.28	30.46
C1_6_6	60.00	60	14089.66	15.05	C1_8_6	79.93	79	27397.13	31.21
C1_6_7	57.98	57	16422.00	11.61	C1_8_7	77.49	77	26640.24	26.09
C1_6_8	56.00	56	14480.77	7.79	C1_8_8	74.00	74	25301.88	16.32
C1_6_9	56.00	56	13726.03	7.79	C1_8_9	72.00	72	24398.13	8.41
C2_6_1	18.00	18	7774.16	21.43	C2_8_1	24.00	24	11662.08	39.12
C2_6_10	17.00	17	7272.83	15.89	C2_8_10	23.00	23	10988.92	37.01
C2_6_2	17.48	17	8304.11	18.86	C2_8_2	23.64	23	12364.71	36.82
C2_6_3	17.88	17	7538.07	35.23	C2_8_3	23.93	23	11427.86	74.08
C2_6_4	17.00	17	6929.58	34.60	C2_8_4	23.00	23	10784.71	83.68
C2_6_5	18.00	18	7575.20	22.31	C2_8_5	24.00	24	11425.67	38.36
C2_6_6	18.00	18	7473.33	22.62	C2_8_6	24.00	24	11348.76	39.72
C2_6_7	18.00	18	7520.47	23.18	C2_8_7	24.00	24	11372.15	42.10
C2_6_8	17.00	17	7587.23	15.63	C2_8_8	23.00	23	11319.73	34.84
C2_6_9	17.57	17	8098.17	19.37	C2_8_9	23.36	23	11704.87	38.68
R1_6_1	59.00	59	21440.75	13.91	R1_8_1	80.00	80	36859.27	24.05
R1_6_10	54.00	54	17745.39	5.58	R1_8_10	72.00	72	31150.15	9.77
R1_6_2	54.00	54	18705.46	5.67	R1_8_2	72.00	72	32475.11	10.00
R1_6_3	54.00	54	16975.15	6.05	R1_8_3	72.00	72	29438.66	10.70
R1_6_4	54.00	54	15793.67	6.19	R1_8_4	72.00	72	27868.23	11.22
R1_6_5	54.00	54	19576.58	5.23	R1_8_5	72.00	72	33805.74	9.18
R1_6_6	54.00	54	17931.30	5.79	R1_8_6	72.00	72	31033.80	10.26
R1_6_7	54.00	54	16582.99	6.11	R1_8_7	72.00	72	28924.77	10.89
R1_6_8	54.00	54	15639.56	6.22	R1_8_8	72.00	72	27720.55	11.14
R1_6_9	54.00	54	18722.05	5.36	R1_8_9	72.00	72	32476.32	9.42
R2_6_1	11.00	11	18259.43	13.14	R2_8_1	15.00	15	28259.03	26.21
R2_6_10	11.00	11	12256.39	15.85	R2_8_10	15.00	15	20572.89	31.45
R2_6_2	11.00	11	14859.10	16.96	R2_8_2	15.00	15	22936.96	36.03
R2_6_3	11.00	11	11227.78	20.19	R2_8_3	15.00	15	17770.55	41.69
R2_6_4	11.00	11	8049.90	22.98	R2_8_4	15.00	15	13270.45	48.28
R2_6_5	11.00	11	15207.11	14.27	R2_8_5	15.00	15	24505.41	28.85
R2_6_6	11.00	11	12586.19	17.74	R2_8_6	15.00	15	20543.28	37.39
R2_6_7	11.00	11	10109.44	20.78	R2_8_7	15.00	15	16695.78	43.81
R2_6_8	11.00	11	7585.04	23.39	R2_8_8	15.00	15	12741.42	48.89
R2_6_9	11.00	11	13549.31	15.17	R2_8_9	15.00	15	22438.98	30.01
RC1_6_1	55.00	55	17081.85	5.33	RC1_8_1	72.07	72	31075.75	7.77
RC1_6_10	55.00	55	15709.48	6.04	RC1_8_10	72.00	72	28522.95	8.50
RC1_6_2	55.00	55	15965.17	5.91	RC1_8_2	72.16	72	28825.23	8.50
RC1_6_3	55.00	55	15229.68	6.30	RC1_8_3	72.00	72	27695.77	8.59
RC1_6_4	55.00	55	14797.20	6.51	RC1_8_4	72.02	72	26807.57	8.95
RC1_6_5	55.00	55	16593.23	5.58	RC1_8_5	72.00	72	29904.64	7.98
RC1_6_6	55.00	55	16593.77	5.60	RC1_8_6	72.00	72	29832.30	8.04
RC1_6_7	55.00	55	16144.87	5.72	RC1_8_7	72.00	72	29355.55	8.21
RC1_6_8	55.00	55	15954.69	5.95	RC1_8_8	72.00	72	28876.94	8.43
RC1_6_9	55.00	55	15862.42	5.91	RC1_8_9	72.00	72	28833.79	8.36
RC2_6_1	14.00	14	13410.77	22.26	RC2_8_1	18.02	18	21068.54	38.06
RC2_6_10	11.00	11	9121.74	13.17	RC2_8_10	15.00	15	14527.35	32.16
RC2_6_2	12.00	12	11624.31	22.06	RC2_8_2	16.00	16	18255.01	39.70
RC2_6_3	11.00	11	9485.68	14.64	RC2_8_3	15.00	15	14539.10	36.94
RC2_6_4	11.00	11	7087.36	16.78	RC2_8_4	15.00	15	11040.25	41.24
RC2_6_5	11.05	11	13043.64	11.16	RC2_8_5	15.00	15	19319.14	26.19
RC2_6_6	11.00	11	12033.46	11.16	RC2_8_6	15.00	15	18253.90	27.04
RC2_6_7	11.00	11	10808.64	11.97	RC2_8_7	15.00	15	16918.73	28.76
RC2_6_8	11.00	11	10064.98	12.55	RC2_8_8	15.00	15	15823.22	31.51
RC2_6_9	11.00	11	9680.09	12.72	RC2_8_9	15.00	15	15396.77	30.63

Table C.3: Detailed results on VRPTW dataset with 600 and 800 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

C. DETAILED TABULAR RESULTS

Instance	1000 requests			
	Avg vehicles	Best vehicles	Best Distance	Avg Time
C1_10_1	100.00	100	42478.95	52.73
C1_10_2	90.00	90	42270.59	13.22
C1_10_3	90.00	90	40213.05	59.93
C1_10_4	90.00	90	39485.38	39.68
C1_10_5	100.00	100	42469.18	53.78
C1_10_6	99.88	99	44009.11	53.81
C1_10_7	97.33	97	43398.48	45.00
C1_10_8	92.97	92	43409.27	28.93
C1_10_9	90.00	90	40477.42	13.51
C1_10_10	90.00	90	39919.59	14.33
C2_10_1	30.00	30	16879.24	73.08
C2_10_2	29.07	29	17168.23	62.64
C2_10_3	29.70	29	16388.56	137.08
C2_10_4	28.95	28	15687.38	162.15
C2_10_5	30.00	30	16562.19	69.27
C2_10_6	29.88	29	17183.96	67.50
C2_10_7	30.00	30	16431.84	73.85
C2_10_8	29.00	29	16215.80	59.15
C2_10_9	29.02	29	16422.22	68.17
C2_10_10	28.10	28	16035.04	45.04
R1_10_1	100.00	100	53651.73	32.69
R1_10_2	91.00	91	48699.82	13.54
R1_10_3	91.00	91	44909.12	14.06
R1_10_4	91.00	91	42635.67	14.24
R1_10_5	91.00	91	51176.16	12.27
R1_10_6	91.00	91	47304.20	13.55
R1_10_7	91.00	91	44262.03	14.09
R1_10_8	91.00	91	42427.57	14.30
R1_10_9	91.00	91	49829.16	12.44
R1_10_10	91.00	91	47982.60	12.67
R2_10_1	19.00	19	42548.67	45.43
R2_10_2	19.00	19	33718.34	57.23
R2_10_3	19.00	19	25081.64	67.84
R2_10_4	19.00	19	17987.68	78.29
R2_10_5	19.00	19	36646.79	47.10
R2_10_6	19.00	19	30251.37	59.00
R2_10_7	19.00	19	23367.64	69.84
R2_10_8	19.00	19	17548.36	76.92
R2_10_9	19.00	19	33365.21	49.95
R2_10_10	19.00	19	30525.22	50.39
RC1_10_1	90.00	90	46165.15	12.61
RC1_10_2	90.00	90	43916.24	14.02
RC1_10_3	90.00	90	42360.17	14.51
RC1_10_4	90.00	90	41481.27	14.85
RC1_10_5	90.00	90	45331.18	12.98
RC1_10_6	90.00	90	45144.54	13.09
RC1_10_7	90.00	90	44653.37	13.41
RC1_10_8	90.00	90	44143.46	13.86
RC1_10_9	90.00	90	44044.22	13.84
RC1_10_10	90.00	90	43706.57	13.86
RC2_10_1	20.00	20	30396.29	58.67
RC2_10_2	18.00	18	26466.68	32.06
RC2_10_3	18.00	18	20077.10	36.83
RC2_10_4	18.00	18	15769.10	43.30
RC2_10_5	18.00	18	27296.63	28.24
RC2_10_6	18.00	18	27093.39	28.55
RC2_10_7	18.00	18	25394.10	30.02
RC2_10_8	18.00	18	23932.60	30.86
RC2_10_9	18.00	18	23230.65	31.53
RC2_10_10	18.00	18	22090.46	31.27

Table C.4: Detailed results on VRPTW dataset with 1000 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

C. DETAILED TABULAR RESULTS

200 requests					400 requests				
Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time	Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time
LC1_2_1	20.00	20	2704.57	0.68	LC1_4_1	40.00	40	7152.06	2.58
LC1_2_10	17.00	17	2954.14	0.76	LC1_4_10	35.03	35	7388.61	3.00
LC1_2_2	19.00	19	2764.56	0.71	LC1_4_2	38.00	38	8007.79	2.85
LC1_2_3	17.00	17	3129.53	0.77	LC1_4_3	32.00	32	8765.36	2.95
LC1_2_4	17.00	17	2693.41	1.01	LC1_4_4	30.00	30	6842.37	3.73
LC1_2_5	20.00	20	2702.05	0.67	LC1_4_5	40.00	40	7150.00	2.62
LC1_2_6	20.00	20	2701.04	0.69	LC1_4_6	40.00	40	7154.02	2.70
LC1_2_7	20.00	20	2701.04	0.69	LC1_4_7	40.00	40	7149.43	2.61
LC1_2_8	19.23	19	3378.32	0.68	LC1_4_8	38.97	38	8639.68	2.65
LC1_2_9	18.00	18	2724.24	0.70	LC1_4_9	36.00	36	7451.20	2.71
LC2_2_1	6.00	6	1931.44	0.76	LC2_4_1	12.00	12	4116.33	2.70
LC2_2_10	6.00	6	1817.45	1.95	LC2_4_10	12.00	12	3828.44	5.25
LC2_2_2	6.00	6	1881.40	1.39	LC2_4_2	12.00	12	4144.29	3.83
LC2_2_3	6.00	6	1844.33	1.88	LC2_4_3	12.00	12	4401.08	5.77
LC2_2_4	6.00	6	1767.12	2.83	LC2_4_4	12.00	12	4038.00	9.02
LC2_2_5	6.00	6	1891.21	1.00	LC2_4_5	12.00	12	4030.63	3.22
LC2_2_6	6.00	6	1857.78	1.30	LC2_4_6	12.00	12	3900.29	3.83
LC2_2_7	6.00	6	1850.13	1.16	LC2_4_7	12.00	12	3962.51	3.96
LC2_2_8	6.00	6	1824.34	1.57	LC2_4_8	12.00	12	3844.45	4.25
LC2_2_9	6.00	6	1854.21	1.42	LC2_4_9	12.00	12	4188.93	4.46
LR1_2_1	20.00	20	4819.12	0.68	LR1_4_1	40.00	40	10639.75	2.51
LR1_2_10	11.00	11	3670.51	0.77	LR1_4_10	20.00	20	8235.68	2.68
LR1_2_2	17.00	17	4621.21	0.77	LR1_4_2	30.57	30	11025.54	2.81
LR1_2_3	14.00	14	4402.38	0.86	LR1_4_3	22.00	22	9245.48	3.26
LR1_2_4	10.00	10	3027.17	1.27	LR1_4_4	16.00	16	6657.97	4.50
LR1_2_5	16.00	16	4760.18	0.65	LR1_4_5	28.33	28	11491.92	2.27
LR1_2_6	13.07	13	4800.94	0.77	LR1_4_6	24.00	24	9816.60	2.83
LR1_2_7	12.00	12	3550.61	0.92	LR1_4_7	18.30	18	8794.57	3.22
LR1_2_8	9.00	9	2759.32	1.25	LR1_4_8	14.00	14	5839.03	4.86
LR1_2_9	14.00	14	4343.86	0.68	LR1_4_9	24.00	24	9866.18	2.41
LR2_2_1	5.00	5	4073.10	1.08	LR2_4_1	8.00	8	9726.88	3.83
LR2_2_10	3.00	3	3254.83	2.21	LR2_4_10	5.00	5	7614.23	6.03
LR2_2_2	4.00	4	3796.00	2.17	LR2_4_2	7.00	7	9405.40	6.12
LR2_2_3	4.00	4	3098.36	3.72	LR2_4_3	5.93	5	10605.63	10.25
LR2_2_4	3.00	3	2486.00	6.91	LR2_4_4	4.00	4	6391.71	13.26
LR2_2_5	4.00	4	3438.39	1.69	LR2_4_5	6.00	6	9947.99	4.56
LR2_2_6	3.10	3	4457.95	1.59	LR2_4_6	5.00	5	9012.31	5.50
LR2_2_7	3.00	3	3098.35	3.23	LR2_4_7	4.80	4	8069.05	12.44
LR2_2_8	2.00	2	2460.49	3.63	LR2_4_8	4.00	4	5268.78	22.77
LR2_2_9	3.00	3	3922.11	1.41	LR2_4_9	6.00	6	7926.07	5.72
LRC1_2_1	19.00	19	3606.06	0.67	LRC1_4_1	36.00	36	9124.52	2.29
LRC1_2_10	12.00	12	2951.90	0.85	LRC1_4_10	23.00	23	7086.93	2.81
LRC1_2_2	15.00	15	3671.02	0.73	LRC1_4_2	31.00	31	8346.06	2.93
LRC1_2_3	13.00	13	3154.92	0.99	LRC1_4_3	24.03	24	7805.16	3.61
LRC1_2_4	10.00	10	2631.82	1.47	LRC1_4_4	19.00	19	5803.39	5.48
LRC1_2_5	16.00	16	3715.81	0.70	LRC1_4_5	32.00	32	8847.40	2.36
LRC1_2_6	16.00	16	3572.16	0.67	LRC1_4_6	30.00	30	8394.47	2.32
LRC1_2_7	14.00	14	3666.34	0.71	LRC1_4_7	28.00	28	8121.69	2.52
LRC1_2_8	13.00	13	3145.74	0.74	LRC1_4_8	26.80	26	7951.70	2.66
LRC1_2_9	13.00	13	3157.34	0.76	LRC1_4_9	25.03	25	8057.36	2.55
LRC2_2_1	6.00	6	3595.18	1.01	LRC2_4_1	11.97	11	10104.29	3.52
LRC2_2_10	3.00	3	2438.14	2.14	LRC2_4_10	6.00	6	5546.55	7.25
LRC2_2_2	5.00	5	3158.25	1.71	LRC2_4_2	10.00	10	7159.95	6.80
LRC2_2_3	4.00	4	2881.99	2.34	LRC2_4_3	8.00	8	6434.83	10.73
LRC2_2_4	3.00	3	2845.86	5.21	LRC2_4_4	5.00	5	5281.82	20.21
LRC2_2_5	5.00	5	2776.93	1.50	LRC2_4_5	10.00	10	7312.67	4.37
LRC2_2_6	5.00	5	2707.96	1.66	LRC2_4_6	9.00	9	6337.08	4.56
LRC2_2_7	4.00	4	3010.68	1.53	LRC2_4_7	8.00	8	6327.98	5.18
LRC2_2_8	4.00	4	2399.89	2.43	LRC2_4_8	7.00	7	5847.12	5.71
LRC2_2_9	4.00	4	2208.49	2.41	LRC2_4_9	6.03	6	6467.76	5.28

Table C.5: Detailed results on PDPTW dataset with 200 and 400 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

C. DETAILED TABULAR RESULTS

600 requests					800 requests				
Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time	Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time
LC1_6_1	60.00	60	14095.64	5.47	LC1_8_1	80.00	80	25184.38	9.52
LC1_6_10	52.03	52	14743.21	6.69	LC1_8_10	69.93	69	26862.39	11.22
LC1_6_2	57.00	57	15048.16	6.08	LC1_8_2	77.00	77	26826.60	10.46
LC1_6_3	49.30	49	15134.40	6.43	LC1_8_3	63.03	63	26722.56	11.09
LC1_6_4	47.00	47	13323.95	8.21	LC1_8_4	59.53	59	22763.42	14.11
LC1_6_5	60.00	60	14086.30	5.53	LC1_8_5	80.00	80	25211.22	9.76
LC1_6_6	60.00	60	14090.79	5.73	LC1_8_6	80.00	80	25164.25	10.02
LC1_6_7	60.00	60	14083.76	5.61	LC1_8_7	80.00	80	25158.38	9.71
LC1_6_8	58.00	58	14880.70	5.96	LC1_8_8	77.90	77	27449.31	10.17
LC1_6_9	53.93	53	15481.13	5.76	LC1_8_9	72.13	72	25942.67	9.80
LC2_6_1	19.00	19	7977.98	5.45	LC2_8_1	24.00	24	11687.06	10.04
LC2_6_10	17.03	17	7995.21	8.65	LC2_8_10	23.70	23	12245.11	14.28
LC2_6_2	18.03	18	10002.75	7.47	LC2_8_2	24.00	24	13713.13	12.90
LC2_6_3	17.93	17	8750.32	11.19	LC2_8_3	24.00	24	13045.65	17.68
LC2_6_4	17.13	17	7863.84	14.97	LC2_8_4	24.00	24	12285.68	24.74
LC2_6_5	18.87	18	9388.47	6.22	LC2_8_5	25.00	25	12326.73	11.01
LC2_6_6	18.00	18	8884.32	7.16	LC2_8_6	24.07	24	12645.71	12.34
LC2_6_7	18.90	18	10015.85	7.54	LC2_8_7	25.00	25	11854.44	12.97
LC2_6_8	18.00	18	7579.93	8.10	LC2_8_8	24.00	24	11454.33	13.24
LC2_6_9	18.00	18	8795.51	8.16	LC2_8_9	24.00	24	11629.41	14.38
LR1_6_1	59.00	59	22821.65	5.74	LR1_8_1	80.00	80	39291.32	9.97
LR1_6_10	25.97	25	20069.15	5.48	LR1_8_10	31.00	31	30484.12	9.22
LR1_6_2	45.00	45	20146.24	7.34	LR1_8_2	59.00	59	34148.68	12.27
LR1_6_3	37.00	37	17851.98	9.61	LR1_8_3	44.00	44	29305.10	16.51
LR1_6_4	28.00	28	13136.08	13.93	LR1_8_4	25.00	25	20725.76	20.21
LR1_6_5	37.43	37	24442.86	5.03	LR1_8_5	48.47	48	41214.06	8.59
LR1_6_6	31.17	31	21574.41	6.37	LR1_8_6	38.87	38	38036.17	11.27
LR1_6_7	24.07	24	17122.71	7.63	LR1_8_7	30.00	30	27710.75	13.77
LR1_6_8	18.00	18	11955.15	10.42	LR1_8_8	20.00	20	19650.51	17.97
LR1_6_9	31.47	31	21835.87	5.02	LR1_8_9	40.27	40	38380.57	8.67
LR2_6_1	11.00	11	22140.32	8.01	LR2_8_1	14.30	14	43364.72	12.47
LR2_6_10	7.00	7	16610.51	13.00	LR2_8_10	8.50	8	31028.58	19.16
LR2_6_2	9.00	9	21623.47	12.55	LR2_8_2	11.63	11	37374.68	22.35
LR2_6_3	7.10	7	17502.49	19.40	LR2_8_3	9.00	9	27037.99	35.20
LR2_6_4	6.00	6	10642.57	49.12	LR2_8_4	6.00	6	21008.64	59.79
LR2_6_5	8.93	8	23786.39	9.90	LR2_8_5	11.00	11	36085.40	15.69
LR2_6_6	7.03	7	19849.86	12.56	LR2_8_6	9.00	9	29966.28	26.60
LR2_6_7	6.00	6	15007.50	22.67	LR2_8_7	7.00	7	26023.24	34.54
LR2_6_8	5.00	5	10674.64	48.06	LR2_8_8	5.00	5	18525.51	59.09
LR2_6_9	8.00	8	18466.99	10.91	LR2_8_9	10.00	10	31259.75	17.02
LRC1_6_1	52.00	52	18293.72	5.08	LRC1_8_1	66.00	66	32263.88	7.96
LRC1_6_10	29.20	29	14192.31	5.63	LRC1_8_10	40.00	40	23989.11	9.47
LRC1_6_2	43.00	43	16531.71	6.88	LRC1_8_2	56.00	56	27894.32	11.76
LRC1_6_3	36.00	36	13987.06	9.69	LRC1_8_3	48.00	48	24375.72	17.87
LRC1_6_4	25.00	25	10807.04	15.73	LRC1_8_4	34.00	34	18223.14	28.01
LRC1_6_5	46.00	46	16910.12	5.44	LRC1_8_5	58.00	58	31284.88	8.40
LRC1_6_6	41.83	41	19025.36	4.96	LRC1_8_6	54.73	54	29165.36	7.98
LRC1_6_7	37.20	37	15990.37	5.16	LRC1_8_7	50.87	50	29119.48	8.40
LRC1_6_8	33.03	33	15390.79	5.46	LRC1_8_8	45.97	45	26171.79	9.74
LRC1_6_9	33.97	33	15962.17	5.38	LRC1_8_9	44.20	44	25062.82	8.85
LRC2_6_1	16.00	16	14624.16	8.03	LRC2_8_1	20.00	20	23119.30	12.58
LRC2_6_10	7.27	7	13299.91	11.93	LRC2_8_10	9.00	9	20276.25	17.67
LRC2_6_2	13.00	13	13922.47	12.94	LRC2_8_2	17.00	17	22297.37	23.66
LRC2_6_3	10.00	10	12458.11	25.24	LRC2_8_3	14.73	14	20401.46	46.51
LRC2_6_4	7.00	7	9955.76	52.06	LRC2_8_4	11.00	11	14819.47	99.63
LRC2_6_5	13.00	13	14642.76	10.13	LRC2_8_5	16.00	16	23904.69	14.41
LRC2_6_6	12.00	12	15217.01	9.01	LRC2_8_6	15.00	15	22659.45	14.95
LRC2_6_7	10.00	10	14599.24	10.22	LRC2_8_7	13.80	13	25919.74	16.23
LRC2_6_8	9.00	9	13577.83	10.77	LRC2_8_8	11.60	11	23683.69	16.83
LRC2_6_9	9.00	9	13031.65	10.71	LRC2_8_9	10.87	10	23280.93	17.89

Table C.6: Detailed results on PDPTW dataset with 600 and 800 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

C. DETAILED TABULAR RESULTS

1000 requests				
Instance	Avg vehicles	Best vehicles	Best Distance	Avg Time
LC1_10_1	100.00	100	42488.66	16.36
LC1_10_2	94.03	94	44551.24	18.91
LC1_10_3	80.07	80	43534.97	20.57
LC1_10_4	73.43	73	37646.27	25.28
LC1_10_5	100.00	100	42477.41	16.76
LC1_10_6	101.00	101	42838.39	16.25
LC1_10_7	100.00	100	42854.99	17.27
LC1_10_8	98.00	98	42976.11	17.53
LC1_10_9	91.03	91	42628.84	16.92
LC1_10_10	87.97	87	42643.83	19.14
LC2_10_1	30.00	30	16879.24	16.86
LC2_10_2	30.00	30	21549.77	23.13
LC2_10_3	29.97	29	20317.31	30.91
LC2_10_4	29.00	29	17901.58	39.90
LC2_10_5	31.00	31	17137.53	17.55
LC2_10_6	30.97	30	19957.27	20.73
LC2_10_7	31.00	31	18519.68	21.27
LC2_10_8	30.00	30	17015.05	21.56
LC2_10_9	30.10	30	18368.63	23.37
LC2_10_10	29.07	29	17179.49	22.92
LR1_10_1	100.00	100	56744.91	17.12
LR1_10_2	80.00	80	49395.80	23.39
LR1_10_3	54.00	54	41567.04	30.91
LR1_10_4	27.00	27	31207.23	31.79
LR1_10_5	58.67	58	63018.59	14.72
LR1_10_6	47.17	47	51299.16	20.97
LR1_10_7	35.00	35	39051.55	24.81
LR1_10_8	25.00	25	28972.39	33.93
LR1_10_9	48.13	48	54377.84	14.72
LR1_10_10	38.53	38	47584.47	15.20
LR2_10_1	17.87	17	67867.86	21.42
LR2_10_2	14.00	14	56441.77	38.12
LR2_10_3	10.70	10	45285.23	63.81
LR2_10_4	8.00	8	27280.85	134.81
LR2_10_5	13.17	13	58730.61	24.65
LR2_10_6	11.00	11	48986.83	44.61
LR2_10_7	8.73	8	41231.56	72.33
LR2_10_8	6.00	6	27348.12	103.62
LR2_10_9	12.13	12	53358.03	29.63
LR2_10_10	10.27	10	47141.37	32.34
LRC1_10_1	82.00	82	49117.82	15.09
LRC1_10_2	71.77	71	45592.28	23.87
LRC1_10_3	53.00	53	35644.85	31.32
LRC1_10_4	40.00	40	27306.58	45.50
LRC1_10_5	72.03	72	50649.05	14.84
LRC1_10_6	67.73	67	45562.40	14.14
LRC1_10_7	60.47	60	41980.57	14.61
LRC1_10_8	56.03	56	40665.50	15.45
LRC1_10_9	53.17	53	39518.56	14.74
LRC1_10_10	47.77	47	37073.93	15.43
LRC2_10_1	22.00	22	34726.88	22.08
LRC2_10_2	19.83	19	39414.35	40.32
LRC2_10_3	16.00	16	27369.43	81.91
LRC2_10_4	11.00	11	23576.18	179.07
LRC2_10_5	16.87	16	42530.76	25.66
LRC2_10_6	17.00	17	30910.86	25.26
LRC2_10_7	15.13	15	33754.98	27.06
LRC2_10_10	11.00	11	29982.46	33.58

Table C.7: Detailed results on PDPTW dataset with 1000 requests. Best distance refers to the minimum distance achieved with the best vehicles count.

Instance	BKS	Best	PSOiA Mean	Time(s)	Best	PMA Mean	Time(s)	Best	HALNS Mean	Time(s)	Best	LNS2 Mean	Time(s)	Best	SISR-OP Mean	Time(s)
bier127_gen1_m2	106	106	104.8	1153.9	106	105	673.5	106	105.5	207.2	106	105.3	7.2	105	105	30
bier127_gen1_m3	103	103	102.4	591.9	103	102.5	470.1	103	102.6	209.6	103	102.2	10.5	103	103	32.3
bier127_gen2_m2	5464	5464	5446.8	1132.6	5464	5454.9	398.8	5464	5460.2	301.3	5464	5442.4	7.6	5464	5446.6	27
bier127_gen2_m3	5393	5393	5376.2	648.1	5393	5386.9	359.3	5393	5390.1	227.5	5393	5392.6	5.5	5391	5389.5	30.5
bier127_gen2_m4	5123	5122	5119.2	657.6	5123	5120.2	383.8	5123	5121	356	5122	5121.1	7.1	5122	5120.9	24.8
bier127_gen3_m2	2885	2885	2884.3	1301.3	2885	2884.7	296.7	2885	2884.5	185	2885	2872.8	6.2	2885	2884.6	32.8
bier127_gen3_m3	2706	2706	2703.8	711.7	2706	2705.2	509.6	2706	2705.6	69.4	2706	2703.7	6.2	2706	2704.4	32.1
bier127_gen3_m4	2402	2402	2384.6	680.8	2402	2398.6	227.7	2402	2399.4	222.4	2402	2402	5.7	2402	2401.9	25.6
cmt101c_m3	1300	1300	1299	111.1	1300	1299	42	1300	1299.4	23.1	1300	1288	4.8	1300	1300	20.3
cmt151b_m3	1385	1385	1373.8	754	1385	1374.5	169.9	1385	1384	164.7	1385	1382.5	10	1385	1383.8	61.9
cmt151c_m2	1964	1963	1962	1799.6	1964	1962	368.5	1964	1962.2	132	1964	1960.6	14	1962	1956.2	45.2
cmt151c_m3	1916	1916	1909.1	1376.2	1916	1909.2	441.5	1916	1914.4	355.6	1911	1909	12.1	1909	1909	48.7
cmt151c_m4	1880	1880	1875.6	881.1	1880	1877.6	826.5	1880	1878.2	107.9	1880	1875.5	9.4	1880	1880	53.9
cmt200b_m2	2096	2096	2088.2	4181	2096	2086.8	669.4	2096	2090.2	183.3	2096	2094.2	17.2	2096	2079.4	130.4
cmt200b_m3	2019	2019	2005	2711.7	2019	2009.4	1351.7	2019	2004.4	386.3	2019	2011.5	21.7	2019	2018.2	136.5
cmt200b_m4	1894	1894	1889.7	1515.2	1894	1891.6	974.5	1894	1891	324.3	1894	1892.8	12.8	1894	1894	137.5
cmt200c_m2	2818	2818	2810.1	7320.3	2818	2810.6	1048.3	2818	2810	368.4	2809	2801.3	22.2	2809	2797.8	93.6
cmt200c_m3	2766	2766	2751.2	4217.3	2766	2751.8	1200	2766	2751.8	273	2766	2756.3	16.7	2766	2766	103
cmt200c_m4	2712	2712	2700.6	3004.1	2712	2703.3	1411.4	2712	2704.4	395.9	2711	2703.3	23.2	2712	2710.3	107.4
eil101b_m3	916	916	913.8	134.4	916	914.6	160.6	916	915.8	69.4	916	910.5	5.9	916	916	24.5
eil101c_m2	1305	1305	1304.8	452.8	1305	1304.8	250.5	1305	1304.7	153.8	1305	1305	4.7	1305	1303.7	19.2
eil101c_m3	1251	1251	1244.1	227.6	1251	1244.2	95	1251	1248.6	116.2	1251	1247.4	10.2	1251	1249.5	19.7
gil262_gen1_m3	101	101	100.9	1769.3	101	100.2	482.6	101	100.8	709.7	101	100.9	13.3	101	100.5	157.1
gil262_gen1_m4	78	78	77.1	155.8	78	77	123.5	78	78	77	78	77.9	11.6	78	77.5	22.1
gil262_gen2_m2	7498	7498	7457.8	7356.6	7498	7458.4	742.1	7498	7466.2	387	7498	7444.5	16.1	7498	7497.3	305.1
gil262_gen2_m3	5615	5615	5608.2	3304.6	5615	5604.9	1163.8	5615	5609.7	352.4	5614	5590.1	22.6	5615	5608.9	164.8
gil262_gen3_m2	7183	7183	7182.8	9129.3	7183	7180	284.2	7183	7182.5	532.1	7183	7168.4	14.6	7181	7181	299.2
gil262_gen3_m4	2507	2507	2499.8	276.4	2507	2500.1	308.8	2507	2501.2	227.2	2507	2505.2	11.1	2507	2507	20
gil262a_m2	4078	4078	4056.4	5907.3	4078	4066.3	2100	4078	4068.8	451.8	4038	4016.2	17.9	4078	4078	224.2
gil262a_m4	3175	3175	3174.2	271.8	3175	3175	222.6	3175	3175	133.8	3175	3175	10.7	3175	3175	31.6
gil262b_m2	8081	8081	8061.1	7473.2	8081	8074.1	1267.8	8081	8078.5	545.2	8072	8071.8	15.8	8081	8072.7	259.3
gil262b_m3	7585	7585	7574.9	7276.8	7585	7566.6	1027.2	7585	7569	463.2	7544	7411.9	29.9	7585	7571.1	267.9
gil262b_m4	6781	6781	6742	4878.6	6781	6756.7	912.6	6781	6761.3	329.1	6780	6760.6	31.6	6781	6765.9	246.6
gil262c_m2	11030	11030	11020	27500.9	11030	11016.5	1309	11030	11022.4	731.2	11008	10934.9	29.8	11019	10923.6	192.6
gil262c_m3*	10758	10757	10714.6	14553.8	10757	10715.2	1375.6	10757	10713.1	650.9	10755	10683.7	31.5	10758	10722	207.7
gil262c_m4	10281	10281	10259.4	8472	10281	10267.3	1997	10281	10262.8	516.5	10280	10264.5	31.7	10280	10271.8	227.2
gr229_gen1_m4	223	223	220.8	11922	223	220.3	46.6	223	221	344.3	222	220.7	23.2	221	220.2	54.9
gr229_gen2_m3	11566	11566	11551.3	14197.2	11566	11557.8	1665.3	11566	11559.2	441.8	11566	11563.2	20.2	11566	11566	44
gr229_gen2_m4	11359	11355	11255.3	18799.5	11355	11328.1	2272	11355	11355	377.3	11359	11334.5	20.6	11359	11195	54.2
gr229_gen3_m3	8056	8056	8051.6	14090.1	8056	8052.2	1065.3	8056	8055.4	938.8	8056	8047.5	19.4	8056	8054.8	41.7
gr229_gen3_m4	7660	7621	7600	11399.7	7651	7610.5	781.5	7651	7622.9	828.7	7660	7610.8	19.8	7660	7627.4	50.8
kroA150_gen2_m2	4335	4335	4334.4	893	4335	4335	495.9	4335	4333.2	392.3	4335	4335	7.7	4335	4335	60.8
kroA150_gen3_m3	2726	2726	2719.6	538	2726	2725.2	597.2	2726	2725.8	168.8	2710	2664.2	14.7	2726	2726	42.1
kroA200_gen1_m4	81	81	80.4	560.3	81	80.6	503.4	81	81	114.6	81	80.5	9.8	81	81	45.8
kroB200_gen1_m2	111	111	110.4	2344.5	111	110.3	663.9	111	111	295.3	111	111	10.3	111	110.5	131.1
kroB200_gen2_m2	6185	6185	6182.2	3467.3	6185	6183.4	426.7	6185	6184.8	438.3	6185	6181.5	12.2	6185	6185	129.7
kroB200_gen2_m4	4944	4944	4942.2	640.7	4944	4942.4	582.4	4944	4942.5	360.4	4944	4944	8.5	4944	4944	37.1

C. DETAILED TABULAR RESULTS

(Continued from previous page)

Instance	BKS	Best	PSOiA Mean	Time(s)	Best	PMA Mean	Time(s)	Best	HALNS Mean	Time(s)	Best	LNS2 Mean	Time(s)	Best	SISR-OP Mean	Time(s)
kroB200_gen3_m2	4765	4765	4757.8	6306.6	4765	4762.4	513.9	4765	4765	470.1	4703	4669.5	17.8	4765	4765	135.9
kroB200_gen3_m3	3028	3028	3016	1713.9	3028	3022.4	741.5	3028	3022.9	547.2	3028	3014.8	9.1	3028	3022.1	69.3
lin318_gen1_m2	180	180	170.1	20667.2	180	175.3	1168.3	180	174.5	2702.8	180	173.7	26.9	180	178.2	528.3
lin318_gen1_m3	149	149	148.6	9014.6	149	147.9	721.4	149	148.4	203.5	148	148	16.2	149	148.2	275.7
lin318_gen2_m2	9544	9544	9533.8	23804.8	9544	9537.5	1924.6	9544	9539.2	1962.1	9544	9494	21.1	9544	9533.2	539.3
lin318_gen2_m3	7807	7786	7782.1	9773.6	7807	7769.6	1413.5	7807	7775.8	1600.6	7807	7741	28.3	7807	7781.6	276.2
lin318_gen3_m2	7936	7936	7905.6	44029	7936	7923.3	1547.3	7936	7919.2	581.3	7936	7842.4	20	7922	7871.2	527.9
lin318_gen3_m4	3797	3797	3796.4	1446.3	3797	3795.5	970.7	3797	3796	128.8	3797	3796.1	22.2	3797	3797	48.6
pr136_gen1_m2	63	63	62.7	451.1	63	62.8	107.5	63	62.9	70.7	63	63	8.1	63	63	43.6
pr136_gen2_m2	3646	3641	3631.8	601.3	3646	3637.4	281.6	3646	3642.8	95.6	3646	3639.1	6.4	3646	3643.5	45.1
pr264_gen1_m4	107	107	106.6	503.1	107	106.7	289.8	107	106.6	195.8	107	106.7	12.1	107	107	22.9
pr264_gen2_m2	6635	6635	6634.2	2048.2	6635	6632	719	6635	6634.4	643	6631	6627.1	15.7	6588	6542.7	14.8
pr264_gen2_m3	6420	6420	6410.7	938.4	6420	6417	859	6420	6418	527.3	6420	6400.5	11.3	6420	6419	24.5
pr264_gen2_m4	5584	5584	5564.5	590.8	5584	5565.1	663.2	5584	5566.2	294.7	5584	5566	26	5567	5564.7	19.4
pr264_gen3_m3	2772	2772	2770	1037.5	2772	2769.8	922.5	2772	2770	1490.8	2772	2761.7	16.5	2772	2772	25
pr299_gen1_m2	139	139	138.5	4775.9	139	138.3	573.4	139	138.8	355.3	139	138.2	14.5	138	137.2	257.1
pr299_gen1_m3	111	111	110.1	1303.7	111	109.2	506	111	110.2	413.8	111	109.2	33.5	110	109.3	71.1
pr299_gen1_m4	84	84	83.6	383.5	84	83.6	340.3	84	83.4	191.3	84	83.8	34.3	84	83.2	28.5
pr299_gen2_m3	6018	6018	5966.7	1446	6018	5979.2	909.7	6018	5978.5	690.4	6005	5961.6	32.5	6018	6014.9	69.3
pr299_gen2_m4	4457	4457	4453	593.4	4457	4455.2	767.7	4457	4454.8	257.8	4457	4446.2	32.5	4457	4457	27.2
pr299_gen3_m2	5729	5729	5728.6	11872.5	5729	5709.8	1489.5	5729	5729	692.1	5728	5715.9	41.8	5729	5729	259.2
pr299_gen3_m3	3655	3655	3611	2705.8	3655	3611.6	1058.2	3655	3648.2	644.4	3655	3607.8	31.1	3655	3654.4	70.5
pr299_gen3_m4	2268	2268	2258	455.6	2268	2261.8	402.4	2268	2262.3	271.9	2268	2183.7	29.3	2268	2267.8	25.7
rat195_gen2_m2	5148	5148	5145.6	2157	5148	5145.9	886.6	5148	5147.6	335.5	5145	5135.8	17.7	5148	5145.7	119.4
rat195_gen3_m3	2574	2574	2571.2	721.8	2574	2569.9	369.5	2574	2570.2	276.8	2574	2574	15.3	2574	2574	34.6
rd400_gen1_m2	233	230	227.8	56767.3	232	228.5	3066.6	232	227.9	3925.6	233	230.7	51.4	224	221.1	1079.3
rd400_gen1_m3	224	222	221.7	62476.1	224	221.3	2844	224	221.4	4178	223	221.7	48.8	218	215.7	1099.7
rd400_gen1_m4	213	213	210.6	34744.8	213	209.9	1985.4	213	210.8	3000.9	212	210	40.6	209	207.1	1091
rd400_gen2_m2*	13047	12993	12787.5	77049.2	13045	12873	3220.6	13045	12872.4	4035.1	13018	12873.1	49.3	13047	12942.3	1088.6
rd400_gen2_m3	12646	12645	12372.1	53707.1	12645	12543.9	2852.7	12646	12555.8	2814.6	12362	12299.4	52	12588	12564.9	1112.3
rd400_gen2_m4	12032	12032	11953.5	42001.6	12032	11969.7	3299.3	12032	11981.2	4070.4	12031	11895.6	50.7	12032	12007	1122.8
rd400_gen3_m2	12431	12428	12274.1	96178.7	12431	12312.2	2418.4	12431	12308	2814.7	12291	12240.5	47.5	12348	12297.2	1065.7
rd400_gen3_m3	11639	11639	11629.5	68074.8	11639	11549.8	3500	11639	11609.8	3811.2	11579	11291.6	46.6	11613	11543.6	1097.3
rd400_gen3_m4	10436	10417	10383.1	48462.8	10436	10345.4	3500	10436	10392.6	3615.5	10374	10283	46.8	10418	10254.6	1107.4
ts225_gen2_m2	5859	5859	5858.5	2998.4	5859	5859	759.6	5859	5858.6	686.5	5859	5728.1	18.5	5859	5839.8	168.3

Table C.8: Results on TOP-L dataset.

C. DETAILED TABULAR RESULTS

Instance	BKS	Best	MSILS			Best	ES		Best	SISR-OP	
			Mean	Time(s)			Mean	Time(s)		Mean	Time(s)
c101	320	320	320	9.5	320	320	0	320	320	26.3	26.3
c102	360	360	360	29.9	360	360	0	360	360	24.7	24.7
c103	400	400	400	47.5	400	400	0.2	400	400	24.8	24.8
c104	420	420	420	89.4	420	420	0.5	420	420	27.2	27.2
c105	340	340	340	13.2	340	340	0	340	340	26.6	26.6
c106	340	340	340	18.8	340	340	0	340	340	26.5	26.5
c107	370	370	370	19.7	370	370	0.4	370	370	25.1	25.1
c108	370	370	370	25.7	370	370	0	370	370	27.1	27.1
c109	380	380	380	38.3	380	380	0	380	380	25	25
c201	870	870	870	39.1	860	860	20.5	870	870	24.3	24.3
c202	930	930	930	67.7	930	930	14.3	930	930	24.6	24.6
c203	960	960	960	83	960	960	0.2	960	960	26.8	26.8
c204	980	980	980	169	970	970	0.2	980	976	29	29
c205	910	910	910	99	900	900	0	910	910	24.4	24.4
c206	930	930	930	65.8	930	928	81.8	930	930	24.3	24.3
c207	930	930	930	69	930	930	30.7	930	930	30.1	30.1
c208	950	950	950	50.4	950	948	17.5	950	950	26.2	26.2
r101	198	198	198	4.7	198	198	3.4	198	198	25.1	25.1
r102	286	286	286	17.4	286	286	0	286	286	26.6	26.6
r103	293	293	293	29.1	293	293	0	293	293	25.6	25.6
r104	303	303	303	42.2	303	303	5.4	303	303	25.7	25.7
r105	247	247	247	16.8	247	247	0	247	247	25	25
r106	293	293	293	23.6	293	293	0	293	293	26.1	26.1
r107	299	299	299	27.9	297	297	0.1	299	299	30.5	30.5
r108	308	308	308	37.2	308	308	0.4	308	308	25.4	25.4
r109	277	277	277	26.1	277	277	0.1	277	277	28.1	28.1
r110	284	284	284	23.9	284	284	7.5	284	284	26.3	26.3
r111	297	297	297	29.5	297	297	0.2	297	297	26.2	26.2
r112	298	298	298	31.7	298	298	3.5	298	298	27.4	27.4
r201	797	797	797	61.6	795	795	6.1	797	797	24.3	24.3
r202	930	930	927.4	99.3	923	922	88.7	930	929.2	22.7	22.7
r203	1028	1028	1028	156.3	1025	1015.8	115	1028	1026.4	23.5	23.5
r204	1093	1093	1093	266.1	1090	1081.8	26.1	1093	1092.8	25.1	25.1
r205	953	953	953	107.1	953	953	13.5	953	953	22.9	22.9
r206	1032	1032	1032	134.5	1029	1029	43.3	1032	1032	23.7	23.7
r207	1078	1078	1078	136.2	1077	1077	1.7	1078	1076	24.4	24.4
r208	1118	1118	1118	352.1	1097	1095.4	55.1	1111	1104.4	25.4	25.4
r209	962	962	962	164.5	958	949.6	87.4	961	958.2	24.3	24.3
r210	1002	1002	1001.4	158.3	991	983	69.7	1002	1001.2	23.9	23.9
r211	1051	1051	1051	202	1049	1048.4	4.8	1051	1050.8	25.4	25.4
rc101	219	219	219	10.3	219	219	0.1	219	219	25.9	25.9
rc102	266	266	266	23	266	266	0.1	266	266	25.3	25.3
rc103	266	266	266	19.9	266	266	0.1	266	266	25	25
rc104	301	301	301	26.6	301	301	0.1	301	301	25.1	25.1
rc105	244	244	244	13	244	244	3.2	244	244	24.7	24.7
rc106	252	252	252	17.4	252	252	0.5	252	252	27.1	27.1
rc107	277	277	277	21.7	277	277	0	277	277	27.7	27.7
rc108	298	298	298	28.3	298	298	0	298	298	25.1	25.1
rc201	795	795	795	43.9	795	795	4.8	795	795	23.3	23.3
rc202	938	938	938	86.8	934	928.2	27.8	938	938	24.2	24.2
rc203	1003	1003	1002.4	248.6	990	987.6	62.3	1003	1003	23.9	23.9
rc204	1143	1143	1139.4	256.1	1122	1117.8	162.2	1143	1137.2	25.3	25.3
rc205	859	859	859	67.1	859	858.4	1.1	859	859	24.8	24.8
rc206	899	899	896.6	73.4	895	890.8	24.9	899	896.6	23.7	23.7
rc207	983	983	983	119.1	983	983	2.3	983	983	24.4	24.4
rc208	1058	1058	1055	159.4	1057	1051.4	5.8	1058	1057.8	26.3	26.3
pr01	308	308	308	12	308	308	0	308	308	5.4	5.4
pr02	404	404	404	37.2	404	404	103.1	404	404	23.4	23.4
pr03	394	394	394	48.2	394	392.8	0.6	394	394	55.8	55.8
pr04	489	489	489	158.2	489	488.2	81.6	489	489	102.1	102.1
pr05	595	595	595	172.6	592	592	68.5	595	595	174.1	174.1
pr06	591	591	590.4	331.7	583	576.6	16.3	591	591	253	253
pr07	298	298	298	12	298	298	0.1	298	298	12.9	12.9
pr08	463	463	463	61.8	463	463	0.1	463	463	55.3	55.3
pr09	493	493	493	106.7	493	493	30.5	493	493	131.4	131.4
pr10	594	594	594	371.6	594	569.2	106.3	594	594	256.4	256.4
pr11	353	353	353	10.6	353	353	3.7	353	353	5.4	5.4
pr12	442	441	441	37.4	441	441	0.6	441	441	24.4	24.4
pr13	467	467	467	96.1	457	457	13.2	467	467	58.6	58.6
pr14	567	555	555	209.2	552	552	6.6	555	554.2	108.9	108.9
pr15	708	708	708	430.6	707	688	54.2	708	708	188.9	188.9
pr16	674	650	650	444.2	650	620.4	21.7	650	650	271	271
pr17	362	362	362	15	362	361.6	53.8	362	362	13.3	13.3
pr18	539	539	539	81.6	539	537.6	55.5	539	539	57.9	57.9
pr19	562	560	554.2	279.7	490	482.6	120.5	560	559.2	141	141
pr20	667	652	651.2	445.4	648	645.8	7.1	652	651.6	275.1	275.1

Table C.9: Results on TOPTW dataset with fleet size $|K| = 1$. 90

C. DETAILED TABULAR RESULTS

Instance	BKS	Best	MSILS		Best	ES		Best	SISR-OP	
			Mean	Time(s)		Mean	Time(s)		Mean	Time(s)
c101	590	590	590	17.1	590	590	0.2	590	590	29.2
c102	660	660	660	22.8	660	660	135.2	660	660	33
c103	720	720	720	41.6	720	718	37.5	720	720	31.1
c104	760	760	760	71.1	760	760	0.3	760	760	29.7
c105	640	640	640	25.7	640	640	0	640	640	30.7
c106	620	620	620	14.3	620	620	0	620	620	28.8
c107	670	670	670	17.7	670	670	0	670	670	28.6
c108	680	680	680	19.2	680	680	0.1	680	680	31.3
c109	720	720	720	41.9	720	720	0.2	720	720	31.3
c201	1460	1460	1458	54.3	1450	1450	2.6	1460	1460	21.5
c202	1470	1470	1470	84.6	1470	1460	67.6	1470	1470	25
c203	1480	1480	1478	129.2	1470	1466	12.5	1480	1478	26.8
c204	1490	1490	1490	225.5	1480	1478	47.2	1490	1486	28
c205	1470	1470	1470	36.4	1470	1470	93.1	1470	1470	23.1
c206	1480	1480	1480	27.1	1480	1480	12.5	1480	1480	22.8
c207	1490	1490	1486	65.5	1480	1480	29.8	1490	1486	24.8
c208	1490	1490	1490	57.6	1480	1480	2.5	1490	1490	24.9
r101	349	349	349	12.5	349	349	0.1	349	349	28.7
r102	508	508	508	18	508	508	0.3	508	508	30.8
r103	522	522	521.6	24.8	520	520	2.1	522	522	30.9
r104	552	552	552	34.1	549	547.2	3.3	552	551	31.5
r105	453	453	453	17	447	447	2.4	453	453	29.5
r106	529	529	529	19.9	529	529	0.2	529	529	30.8
r107	538	538	538	29.6	538	538	0.2	538	538	31.4
r108	560	560	560	32.1	560	558.8	39	560	560	34.7
r109	506	506	506	20.5	506	506	2.1	506	506	32.5
r110	525	525	525	21	525	525	2.4	525	525	32
r111	544	544	544	24.3	541	541	4.3	544	544	33.3
r112	544	544	544	31.8	544	542.2	14.8	544	544	31.3
r201	1260	1260	1254.6	95	1254	1247.2	107.2	1260	1258.8	20.8
r202	1353	1353	1351.6	228.9	1347	1341.6	103.3	1353	1351.4	18.4
r203	1431	1430	1425.6	371.6	1416	1413.8	55.7	1431	1427.4	19.4
r204	1458	1458	1458	1.7	1458	1458	43.3	1458	1458	14.7
r205	1402	1402	1396.6	275.8	1383	1380.2	138.7	1394	1391	19.2
r206	1452	1452	1450.8	297.6	1440	1438.2	128.9	1451	1448.4	16.7
r207	1458	1458	1458	2.7	1458	1455.4	89.6	1458	1458	15.3
r208	1458	1458	1458	0.2	1458	1458	0.4	1458	1458	14.4
r209*	1424	1423	1419.4	344.9	1417	1404	157.3	1424	1419.6	20.4
r210	1438	1438	1432	431.5	1420	1415.4	107.4	1430	1428	17.4
r211	1458	1458	1458	17.5	1458	1457	94.6	1458	1458	15.1
rc101	427	427	427	13.9	427	427	0.1	427	427	28.6
rc102	505	505	504.8	22.3	505	503.4	63.7	505	505	29.4
rc103	524	524	524	18.7	523	523	1	524	524	33.6
rc104	575	575	574.8	30.2	574	573	19.5	574	574	34.4
rc105	480	480	480	16.4	480	480	78.8	480	480	30.2
rc106	483	483	482.8	18.6	483	483	64.5	483	483	29.5
rc107	534	534	534	19.3	529	529	7.3	534	534	30.8
rc108	556	556	556	27.1	553	553	5.9	554	554	32.6
rc201	1386	1386	1384.6	64.2	1375	1369.6	84.1	1386	1383	22
rc202*	1524	1523	1519.8	136.9	1511	1503.6	65.3	1524	1523.4	20.3
rc203	1639	1639	1636.6	197	1629	1605.8	101.1	1638	1634.4	19.8
rc204	1718	1718	1717	249.4	1706	1701.6	115.6	1712	1711.2	16.3
rc205	1462	1462	1461.8	126.3	1458	1427.4	105.8	1462	1462	21.4
rc206	1552	1552	1546.2	132.9	1538	1521.8	112.5	1550	1545.6	20
rc207	1609	1607	1604.4	176	1587	1569	111.6	1601	1601	20.4
rc208*	1707	1705	1696	292.5	1691	1683.4	190.6	1707	1695.8	17.1
pr01	502	502	502	9.3	502	502	0.3	502	502	5.8
pr02	715	713	713	48	715	713	40.6	713	713	26.3
pr03	742	742	742	50	742	726.6	42.6	742	742	66.7
pr04	928	928	928	95.5	913	902.8	132.9	928	928	125.5
pr05	1103	1103	1093.4	181.8	1090	1086.4	95.3	1101	1095	222.1
pr06	1076	1076	1074.2	382	1070	1050.4	88.7	1076	1074.2	321.5
pr07	566	566	566	24.4	566	566	4.1	566	566	14.6
pr08	834	834	834	56.2	829	829	8.6	834	833.2	65.5
pr09	909	909	909	166.2	907	904	98.1	909	907.2	166.3
pr10	1145	1145	1137	230.8	1128	1091.6	43.9	1145	1143.4	331
pr11	566	566	565.6	24.4	566	564.8	16	566	565.2	5.5
pr12	774	768	766.2	50.5	765	764.4	12.5	766	764.4	27.5
pr13	845	845	840.2	113.4	845	840.4	73.5	845	841.4	68
pr14	1017	996	994.8	251.4	988	981.8	20	995	994.2	133.8
pr15	1238	1237	1230.2	383.6	1222	1194.8	131.9	1238	1226.2	239.8
pr16	1231	1199	1190.6	666.2	1183	1162.6	130	1199	1197.8	356.4
pr17	652	646	646	22.2	646	645.2	95.7	646	646	14.7
pr18	955	955	955	181.5	945	933.2	58.3	955	954.8	69
pr19*	1042	1041	1040.2	334.1	1019	1015.4	146.2	1042	1038.2	178.7
pr20	1251	1251	1246.8	514	1207	1187.2	76.8	1249	1246.4	364

Table C.10: Results on TOPTW dataset with fleet size $|K| = 2$. 91

C. DETAILED TABULAR RESULTS

Instance	BKS	Best	MSILS		Best	ES		Best	SISR-OP	
			Mean	Time(s)		Mean	Time(s)		Mean	Time(s)
c101	810	810	810	30.9	810	810	1	810	810	31.1
c102	920	920	920	38.5	920	920	16.1	920	920	30.7
c103	990	990	986	77.3	980	980	32.2	990	984	32.2
c104	1030	1030	1030	52.1	1030	1022	6	1030	1026	35.7
c105	870	870	870	18.7	870	870	24.9	870	870	30.2
c106	870	870	870	30.2	870	870	26.6	870	870	30.5
c107	910	910	910	21.4	910	910	3.1	910	910	29.7
c108	920	920	920	37.4	920	916	46	920	920	31.8
c109	970	970	970	51.5	970	964	4.2	970	970	31.4
c201	1810	1810	1810	0.7	1810	1806	22.2	1810	1810	12.9
c202	1810	1810	1810	1.1	1810	1808	18.8	1810	1810	14.4
c203	1810	1810	1810	1.6	1810	1808	37.8	1810	1810	17
c204	1810	1810	1810	1.8	1810	1810	51.7	1810	1810	14.7
c205	1810	1810	1810	0.3	1810	1810	4.8	1810	1810	11.2
c206	1810	1810	1810	0.2	1810	1810	2.5	1810	1810	13.7
c207	1810	1810	1810	0.2	1810	1810	1.9	1810	1810	12
c208	1810	1810	1810	0.1	1810	1810	1.3	1810	1810	12.2
r101	484	484	484	15.3	481	481	0.1	484	484	34.5
r102	694	694	694	21.1	691	691	4.9	694	694	32.3
r103	747	747	747	29.6	740	740	7.6	747	747	35.1
r104	778	778	778	33.3	778	778	90.3	778	778	34.8
r105	620	620	620	12	620	619.2	1.8	620	620	32
r106	729	729	729	30.6	729	728	9.3	729	729	34.4
r107	760	760	760	43.6	759	759	38.7	760	759.4	34.3
r108	797	797	797	27.2	797	797	2.8	797	797	35.8
r109	710	710	710	32.7	710	706.4	5.5	710	710	35.8
r110	737	737	736.8	22.6	736	736	9.9	737	737	36.5
r111	774	774	774	27.8	774	773.2	45.6	774	773.4	34.3
r112	776	776	776	33.8	776	775.6	72.8	776	776	35.1
r201	1450	1450	1442	204.4	1440	1435.8	58.4	1443	1441.6	16.8
r202	1458	1458	1458	0.6	1458	1457.4	21.6	1458	1458	16.2
r203	1458	1458	1458	0.1	1458	1458	0.3	1458	1458	16.4
r204	1458	1458	1458	0.1	1458	1458	0	1458	1458	17.3
r205	1458	1458	1458	0.1	1458	1458	0	1458	1458	13
r206	1458	1458	1458	0.1	1458	1458	0	1458	1458	14.9
r207	1458	1458	1458	0.1	1458	1458	0	1458	1458	17.2
r208	1458	1458	1458	0.1	1458	1458	0	1458	1458	17.6
r209	1458	1458	1458	0.1	1458	1458	0	1458	1458	13.8
r210	1458	1458	1458	0.1	1458	1458	0	1458	1458	14.9
r211	1458	1458	1458	0.1	1458	1458	0	1458	1458	17.7
rc101	621	621	621	16.5	621	616.8	5.9	621	621	32.2
rc102	714	714	712.8	15	710	710	10	714	714	32.5
rc103	764	764	760.6	33.8	764	757	13.7	764	759.4	33.3
rc104	835	835	835	39.5	834	833.2	29.8	834	834	34.6
rc105	682	682	682	18.2	682	682	2.4	682	682	33.8
rc106	706	706	705.4	25.4	706	705.8	17.8	706	706	32.8
rc107	773	773	773	27.5	768	768	17.7	773	773	34.5
rc108	795	795	795	32.8	795	792.8	77.5	795	795	34.3
rc201	1698	1698	1694	120.5	1698	1685.8	94.7	1698	1698	14.7
rc202	1724	1724	1724	13.9	1724	1720.4	89.9	1724	1724	14.2
rc203	1724	1724	1724	0.1	1724	1724	0.7	1724	1724	14.9
rc204	1724	1724	1724	0.2	1724	1724	0	1724	1724	16.6
rc205	1719	1719	1710	156.9	1709	1705.4	64.7	1719	1718.4	14.3
rc206	1724	1724	1724	0.2	1724	1724	0.6	1724	1724	13.1
rc207	1724	1724	1724	0.2	1724	1724	0.5	1724	1724	14.7
rc208	1724	1724	1724	0.1	1724	1724	0	1724	1724	15.7
pr01	622	622	620.8	16.6	616	616	23.7	622	622	5.4
pr02	947	945	943.8	61.4	941	940.2	34.2	944	942.8	26
pr03	1014	1014	1012.6	110.3	1010	1008.6	56	1013	1011.8	70.4
pr04	1298	1296	1286.8	122.7	1269	1266.6	62.9	1288	1277.8	138.4
pr05	1500	1500	1488.8	291.5	1478	1468.8	147.9	1499	1491.2	247.6
pr06*	1516	1515	1514.6	351.2	1505	1481.4	169	1516	1513.2	378.8
pr07	744	744	744	19.5	744	741.2	74.5	744	743.8	14
pr08	1142	1141	1137	120.5	1135	1129.2	133	1140	1140	70
pr09	1282	1276	1271.4	168.7	1273	1269.2	111.5	1282	1278.2	188.6
pr10*	1584	1582	1577.8	424.4	1574	1556.4	146.6	1584	1579.6	388.7
pr11	654	654	654	28.5	654	654	12.2	654	654	5.1
pr12*	1004	1000	997.6	71	990	987.4	95.5	1004	997	27.3
pr13	1159	1159	1155	143.3	1147	1143.2	115.5	1159	1156.8	71.4
pr14	1375	1375	1365.4	335.5	1339	1331.2	152	1375	1364.8	147
pr15	1694	1694	1683.4	601.5	1654	1636.8	106.3	1688	1681.6	271.2
pr16	1668	1651	1638.8	736.1	1629	1606.2	183	1652	1645.2	420.3
pr17*	843	841	838.8	39.7	837	828.2	112.2	843	841.2	13.4
pr18	1289	1289	1279	123.4	1256	1243.8	161.3	1285	1279.6	72.5
pr19	1428	1428	1420.4	365.7	1391	1372.8	137.2	1426	1420.8	199.9
pr20	1722	1722	1705.6	1049.1	1671	1658.6	162.2	1722	1713	421.3

Table C.11: Results on TOPTW dataset with fleet size $|K| = 3$. 92

C. DETAILED TABULAR RESULTS

Instance	BKS	Best	MSILS			Best	ES		Best	SISR-OP	
			Mean	Time(s)			Mean	Time(s)		Mean	Time(s)
c101	1020	1020	1020	20.8		1020	1020	53	1020	1020	31.4
c102	1150	1150	1150	30		1140	1136	51.6	1150	1150	32.5
c103	1210	1210	1208	60.6		1210	1196	49.2	1210	1208	32.3
c104	1260	1260	1260	70.4		1260	1254	53.4	1260	1256	34.7
c105	1070	1060	1060	26.9		1060	1058	18.8	1070	1062	29.8
c106	1080	1080	1078	40.2		1080	1068	63.2	1080	1078	31.4
c107	1120	1120	1120	34.8		1110	1110	16.8	1120	1120	30.2
c108	1140	1140	1132	57.1		1130	1126	98.8	1130	1130	32.5
c109	1190	1190	1190	61.9		1190	1184	55.4	1190	1190	32.4
c201	1810	1810	1810	0		1810	1810	0	1810	1810	11
c202	1810	1810	1810	0.1		1810	1810	0	1810	1810	12.1
c203	1810	1810	1810	0.1		1810	1810	0	1810	1810	14.2
c204	1810	1810	1810	0.1		1810	1810	0	1810	1810	17
c205	1810	1810	1810	0.1		1810	1810	0	1810	1810	11
c206	1810	1810	1810	0.1		1810	1810	0	1810	1810	12
c207	1810	1810	1810	0.1		1810	1810	0	1810	1810	11.9
c208	1810	1810	1810	0.1		1810	1810	0	1810	1810	12.9
r101	611	611	611	16		608	608	14	611	611	33.8
r102	843	843	840	29.4		837	837	47.9	843	842	36.3
r103	928	928	928	41.4		928	928	5.5	928	928	37.8
r104	975	975	975	50.5		972	969.6	51.5	975	974.8	35.4
r105	778	778	776	22.4		778	776.6	147.2	778	778	39.2
r106	906	906	905	23.5		906	903.8	24.7	906	906	36.4
r107	950	950	950	39.2		950	947.2	57.5	950	950	34.3
r108	995	995	994.6	52.3		991	991	79.3	994	993	38
r109	885	885	885	37.2		885	885	131.5	885	885	37.2
r110	915	915	915	26.9		915	907.6	50.8	915	915	38.6
r111	953	952	950.8	38.9		945	944.4	12.2	953	952.4	36.2
r112	974	974	973.6	41.3		971	967.4	69.1	974	973.4	37.3
r201	1458	1458	1458	0.1		1458	1458	0	1458	1458	13.8
r202	1458	1458	1458	0.1		1458	1458	0	1458	1458	14.6
r203	1458	1458	1458	0.1		1458	1458	0	1458	1458	17.6
r204	1458	1458	1458	0.1		1458	1458	0	1458	1458	17.8
r205	1458	1458	1458	0.1		1458	1458	0	1458	1458	15
r206	1458	1458	1458	0.1		1458	1458	0	1458	1458	16.9
r207	1458	1458	1458	0.1		1458	1458	0	1458	1458	19.8
r208	1458	1458	1458	0.1		1458	1458	0	1458	1458	16.2
r209	1458	1458	1458	0.1		1458	1458	0	1458	1458	15.6
r210	1458	1458	1458	0.1		1458	1458	0	1458	1458	16
r211	1458	1458	1458	0.1		1458	1458	0	1458	1458	16.3
rc101	811	811	811	25.9		808	808	1.6	811	811	32.1
rc102	909	908	904.4	26.5		902	902	38.1	908	908	35.5
rc103	975	975	975	39		972	971	103.6	975	975	35.6
rc104	1065	1065	1065	28.8		1065	1064.6	107.5	1065	1065	34.6
rc105	875	875	875	24.6		875	875	64.5	875	875	33.7
rc106	909	909	909	33.7		909	908.8	37.6	909	909	33.5
rc107	987	987	986	46.9		980	975.8	75.9	987	985	34.9
rc108	1025	1025	1025	28.9		1023	1021.6	60.8	1025	1025	35.3
rc201	1724	1724	1724	0.1		1724	1724	0.1	1724	1724	12.7
rc202	1724	1724	1724	0.1		1724	1724	0	1724	1724	14.1
rc203	1724	1724	1724	0.1		1724	1724	0	1724	1724	16.2
rc204	1724	1724	1724	0.1		1724	1724	0	1724	1724	17
rc205	1724	1724	1724	0.1		1724	1724	0	1724	1724	13
rc206	1724	1724	1724	0.1		1724	1724	0	1724	1724	13.5
rc207	1724	1724	1724	0.1		1724	1724	0	1724	1724	15.5
rc208	1724	1724	1724	0.1		1724	1724	0	1724	1724	15.8
pr01	657	657	657	0		657	657	0.5	657	657	4.5
pr02	1083	1083	1078.2	85		1075	1067.8	85.5	1081	1079.8	23.6
pr03	1248	1247	1239.4	75.9		1247	1236	145	1247	1244.6	69.1
pr04	1595	1595	1587	242.5		1566	1549.8	176.8	1595	1582.8	141.5
pr05	1859	1858	1841.2	482.1		1840	1818.6	138.6	1854	1849.2	264.2
pr06	1898	1894	1883.8	487.7		1861	1851.4	222.2	1894	1891	416.6
pr07	876	876	872.4	36.7		872	866.4	142.3	876	876	12.6
pr08	1392	1390	1378.8	142.2		1392	1372.6	102.6	1390	1382.4	69.5
pr09	1626	1622	1615.2	246.8		1603	1585.8	151.9	1623	1620.2	199.4
pr10*	1970	1965	1956.4	613.4		1936	1887.8	211.8	1970	1958.8	418.2
pr11	657	657	657	0		657	657	0.2	657	657	4.7
pr12	1135	1133	1129.4	120.1		1117	1108.6	105.4	1130	1128.4	24.6
pr13*	1396	1392	1385.8	170.5		1377	1350.4	148.5	1396	1391.2	68.3
pr14*	1694	1688	1682.6	473.1		1664	1638.6	245	1694	1682.6	152.6
pr15*	2088	2085	2063.4	669.6		2007	1999	171.7	2088	2081.2	284.9
pr16	2065	2056	2040	908.9		1999	1967.6	197.1	2060	2047.6	455.7
pr17	936	936	932	95.3		931	927.2	227.1	934	933.6	11.2
pr18	1558	1554	1541.2	171.6		1496	1489.4	79.8	1557	1543	69.7
pr19*	1782	1780	1763.6	540.6		1750	1723.8	166.7	1782	1778.8	209.2
pr20*	2125	2098	2085.8	1007.3		2082	2027.8	261.5	2125	2112	459.6

Table C.12: Results on TOPTW dataset with fleet size $|K| = 4$. 93

C. DETAILED TABULAR RESULTS

(Continued from previous page)

Instance	BKS	BiF&F-s		VSS-Tabu		VSS-SA		HALNS		Best	SISR-OP	
		Best	Time(s)	Best	Time(s)	Best	Time(s)	Best	Time(s)		Mean	Time(s)
P75	1913	1910	7882	1913	5123	1913	1671	1912	3214	1893	1888.9	442.1
P76	1948	1947	8065	1947	4125	1948	2342	1947	3164	1925	1922.6	528.5
P77	1844	1843	3754	1844	808	1844	977	1844	2214	1833	1829.9	196
P78	1850	1848	10204	1848	1601	1850	1365	1849	1433	1827	1824.3	302.2
P79	1873	1870	5774	1873	2662	1871	3101	1870	3083	1853	1849.1	398
P80	1920	1919	1410	1919	6215	1920	3755	1920	2831	1898	1893.9	552
P81	2041	2032	1236	2039	1286	2041	763	2040	1167	2026	2022.2	163
P82	2068	2065	5641	2066	2121	2068	1078	2066	1983	2050	2046.8	224.1
P83	2098	2097	3173	2097	2724	2098	1649	2098	1562	2080	2073.4	286.4
P84	2131	2127	103	2130	3317	2131	1627	2130	1116	2113	2108.1	358.4
P85	2163	2155	7914	2162	3764	2163	2252	2161	1339	2141	2137.8	437.8
P86	2205	2204	1658	2205	6426	2204	2289	2205	2078	2186	2178	550.1
P87	2064	2063	3762	2063	889	2064	886	2064	1329	2053	2046.6	193.9
P88	2074	2070	3589	2072	1974	2074	2000	2072	784	2049	2046.7	302.8
P89	2118	2115	17380	2116	2693	2118	2515	2115	2849	2097	2092.3	400
P90	2173	2172	15678	2171	3326	2173	3197	2173	2901	2145	2141.9	539.2

Table C.13: Results on CTOP subset 2.

Instance	BKS	BiF&F-s		VSS-Tabu		VSS-SA		HALNS		Best	SISR-OP	
		Best	Time(s)	Best	Time(s)	Best	Time(s)	Best	Time(s)		Mean	Time(s)
P1	2530	2530	136	2530	178	2530	122	2530	101	2523	2521.5	144.7
P2	2614	2614	752	2613	2703	2613	897	2614	755	2598	2593.9	202.8
P3	2520	2520	319	2520	6320	2520	2246	2520	406	2503	2499.2	269.9
P4	2603	2603	1844	2603	2755	2603	6341	2603	1123	2585	2581.2	331.7
P5	2696	2696	6956	2696	5675	2696	6799	2695	3892	2679	2675.1	410.2
P6	2932	2932	7050	2931	16945	2931	7305	2931	6813	2909	2904.2	485.4
P7	2489	2489	187	2489	2097	2489	262	2489	99	2475	2473.4	165
P8	2623	2621	11461	2622	3121	2623	3846	2622	5801	2600	2598.7	262.4
P9	2782	2775	11315	2780	12326	2782	5697	2776	6193	2751	2743.1	365.1
P10	2963	2959	10222	2962	8460	2963	10388	2958	5948	2930	2922.5	500.1
P11	2860	2859	153	2860	2235	2860	1518	2860	729	2851	2847.5	139.7
P12	2965	2965	652	2965	2853	2965	1946	2965	868	2949	2947.3	187.7
P13	2860	2860	270	2860	2530	2860	2845	2860	389	2843	2841.1	254.7
P14	2954	2954	556	2954	4768	2954	1572	2954	507	2937	2932.9	315.4
P15	3075	3075	6881	3074	7201	3074	6259	3075	6783	3055	3052.8	388.2
P16	3353	3353	7411	3351	11492	3352	10810	3352	3609	3324	3321.1	464.7
P17	2814	2814	3306	2812	2429	2813	2760	2813	2016	2799	2795.4	159.4
P18	2973	2971	1425	2972	5321	2973	2772	2973	1907	2957	2952.9	254.5
P19	3163	3163	13864	3162	8331	3163	6937	3160	7013	3136	3126.7	344.5
P20	3383	3381	14379	3383	11735	3383	8549	3378	7142	3347	3341.9	484.6
P21	3159	3159	1745	3158	1462	3159	1576	3159	824	3148	3146.2	124.9
P22	3291	3291	583	3291	2850	3291	3018	3291	1401	3278	3274.2	174.2
P23	3190	3190	461	3190	4288	3190	3143	3190	616	3174	3170.7	243.5
P24	3289	3289	426	3289	283	3289	314	3289	297	3271	3269.6	302.7
P25	3434	3434	7418	3434	4145	3434	4396	3434	5964	3415	3410.7	374.9
P26	3749	3749	1007	3748	14090	3748	9342	3749	1430	3723	3717.7	467.1
P27	3117	3116	189	3117	3090	3117	2248	3117	2046	3100	3098.1	141.7
P28	3303	3302	3455	3303	5691	3303	5120	3303	3583	3284	3279.4	241.5
P29	3529	3525	14499	3528	7370	3529	11119	3526	7134	3496	3489.9	328.2
P30	3783	3783	11663	3781	10154	3783	9865	3783	7147	3745	3741.2	449.9

Table C.14: Results on CTOP subset 3.